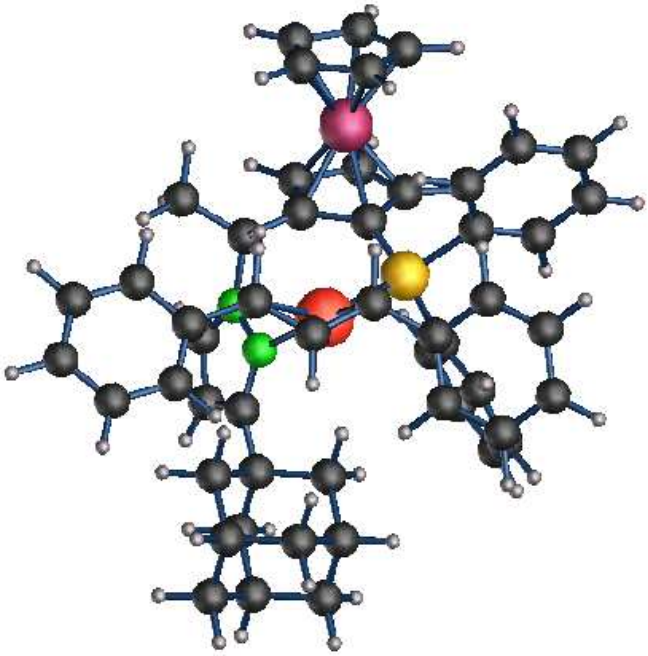
Manual for the Projector Augmented Wave Method

Version 2.0



Peter E. Blöchl, Clausthal University of Technology
(October 10, 2026)

1

[1]The title picture shows the a chiral Pd complex with P,N ligands, a highly enantio-selective catalyst for allylic amination [1].

Contents

1 Initial remarks **1**

2 What is the projector augmented wave method? **1**

3 How the code is built **3**

4 Input data structure **5**
4.1 Syntax rules for the input data 6
4.2 Extended notation for atoms 8

5 Before starting... **8**
5.1 The PAW directory 8
5.2 The project data structure 9

6 The simulation code **10**
6.1 How to perform a simulation 10
6.1.1 Input files 10
6.1.2 Execute the simulation code 10
6.1.3 Terminate the simulation 12
6.2 The control input file “CNTL” 12
6.2.1 Examples for the control input file “CNTL” 13
6.2.2 Argument keywords for the control input file “CNTL” . . 14
6.2.3 !CONTROL 14
6.2.4 !CONTROL!FILES 14
6.2.5 !CONTROL!FILES!FILE 15
6.2.6 !CONTROL!GENERIC 18
6.2.7 !CONTROL!DFT 21
6.2.8 !CONTROL!DFT!NTBO 25
6.2.9 !CONTROL!FOURIER 26
6.2.10 !CONTROL!PSIDYN 28
6.2.11 !CONTROL!PSIDYN!AUTO 33
6.2.12 !CONTROL!PSIDYN!THERMOSTAT 34
6.2.13 !CONTROL!RDYN 36
6.2.14 !CONTROL!RDYN!AUTO 41
6.2.15 !CONTROL!RDYN!WARMUP 42
6.2.16 !CONTROL!RDYN!THERMOSTAT 43
6.2.17 !CONTROL!MERMIN 45

6.2.18 !CONTROL!MERMIN!DIAL 49
6.2.19 !CONTROL!CELL 49
6.2.20 !CONTROL!QM-MM 53
6.2.21 !CONTROL!QM-MM!AUTO 54
6.2.22 !CONTROL!QM-MM!THERMOSTAT 55
6.2.23 !CONTROL!COSMO 57
6.2.24 !CONTROL!ANALYSE 59
6.2.25 !CONTROL!ANALYSE!TRA 60
6.2.26 !CONTROL!ANALYSE!HYPERFINE 61
6.2.27 !CONTROL!ANALYSE!CORELEVELS 63
6.2.28 !CONTROL!ANALYSE!WAVE 64
6.2.29 !CONTROL!ANALYSE!DENSITY 65
6.2.30 !CONTROL!ANALYSE!POTENTIAL 68
6.2.31 !CONTROL!ANALYSE!EELS 68
6.3 The structure input file “STRC” 71
6.3.1 Example for the structure input file “STRC” 71
6.3.2 Argument description for the structure input file “STRC” . 73
6.3.3 !STRUCTURE 73
6.3.4 !STRUCTURE!GENERIC 73
6.3.5 !STRUCTURE!LATTICE 74
6.3.6 !STRUCTURE!KPOINTS 74
6.3.7 !STRUCTURE!SPECIES 76
6.3.8 !STRUCTURE!SPECIES!DFTU 79
6.3.9 !STRUCTURE!SPECIES!AUGMENT 80
6.3.10 !STRUCTURE!SPECIES!AUGMENT!POT 83
6.3.11 !STRUCTURE!SPECIES!AUGMENT!CORE 85
6.3.12 !STRUCTURE!SPECIES!AUGMENT!GRID 86
6.3.13 !STRUCTURE!SPECIES!NTBO 87
6.3.14 !STRUCTURE!ATOM 90
6.3.15 !STRUCTURE!OCCUPATIONS 91
6.3.16 !STRUCTURE!OCCUPATIONS!STATE 92
6.3.17 !STRUCTURE!ISOLATE 93
6.3.18 !STRUCTURE!CONFINE 94
6.3.19 !STRUCTURE!QM-MM 96
6.3.20 !STRUCTURE!QM-MM!ATOM 96
6.3.21 !STRUCTURE!QM-MM!BOND 98
6.3.22 !STRUCTURE!QM-MM!LINK 99
6.3.23 !STRUCTURE!COSMO 100

6.3.24 !STRUCTURE!COSMO!ATOM . . . 101
6.3.25 !STRUCTURE!GROUP . . . 101
6.3.26 !STRUCTURE!VEXT . . . 102
6.3.27 !STRUCTURE!VEXT!UNBIND . . . 102
6.3.28 !STRUCTURE!CONSTRAINTS . . . 103
6.3.29 !STRUCTURE!CONSTRAINTS!RIGID . . . 103
6.3.30 !STRUCTURE!CONSTRAINTS!FREEZE . . . 104
6.3.31 !STRUCTURE!CONSTRAINTS!TRANSLATION . . . 104
6.3.32 !STRUCTURE!CONSTRAINTS!ROTATION . . . 105
6.3.33 !STRUCTURE!CONSTRAINTS!ORIENTATION . . . 106
6.3.34 !STRUCTURE!CONSTRAINTS!BOND . . . 106
6.3.35 !STRUCTURE!CONSTRAINTS!ANGLE . . . 109
6.3.36 !STRUCTURE!CONSTRAINTS!TORSION . . . 111
6.3.37 !STRUCTURE!CONSTRAINTS!COGSEP . . . 114
6.3.38 !STRUCTURE!CONSTRAINTS!MIDPLANE . . . 116
6.3.39 !STRUCTURE!CONSTRAINTS!LINEAR . . . 118
6.3.40 !STRUCTURE!CONSTRAINTS!LINEAR!ATOM . . . 120
6.3.41 !STRUCTURE!ORBPOT . . . 121
6.3.42 !STRUCTURE!ORBPOT!POT . . . 121

7 The Energy Grab Tool: paw_grab 124
7.1 Command . . . 124
7.2 Example for the control input file . . . 124
7.3 Argument description for the control input file . . . 125
7.3.1 !GCNTL . . . 125
7.3.2 !GCNTL!GENERIC . . . 125
7.3.3 !GCNTL!SUBSTANCE . . . 125
7.3.4 !GCNTL!REACTION . . . 126
7.3.5 !GCNTL!REACTION!FROM . . . 126
7.3.6 !GCNTL!REACTION!TO . . . 127

8 The Atomic Structure Analysis Tool: “paw_strc” 127
8.1 Command . . . 127

9 The Wave Function and Density Analysis Tool: “paw_wave” 129
9.1 Command . . . 129
9.2 Example for the control input file . . . 129
9.3 Argument description for the control input file . . . 130

9.3.1 !WCNTL . . . 130
9.3.2 !WCNTL!FILES . . . 130
9.3.3 !WCNTL!FILES!FILE . . . 130
9.3.4 !WCNTL!GENERIC . . . 132
9.3.5 !WCNTL!VIEWBOX . . . 132
9.3.6 !WCNTL!PLANE . . . 133
9.4 View data using OPENDX[13] . . . 133
9.5 View data using FreeWRL . . . 135
9.6 Contour plots using Gnuplot . . . 136
9.7 Rubbersheets using Gnuplot . . . 139

10 Wave function converter for CryMolCAD: "paw_cmcwave" 142

11 Band-Offset and Work-Function Tool: "paw_1davpot" 143

12 The Trajectory Analysis Tool: "paw_tra" 144
12.1 Command . . . 144
12.2 Example for the control input file . . . 144
12.3 Argument description for the control input file "tcntl" . . . 145
12.3.1 !TCNTL . . . 145
12.3.2 !TCNTL!FILES . . . 145
12.3.3 !TCNTL!FILES!FILE . . . 146
12.3.4 !TCNTL!SEQUENCE . . . 147
12.3.5 !TCNTL!MOVIE . . . 148
12.3.6 !TCNTL!MOVIE!VIEWBOX . . . 148
12.3.7 !TCNTL!MOVIE!FILE . . . 149
12.3.8 !TCNTL!MOVIE!SELECT . . . 150
12.3.9 !TCNTL!CORRELATION . . . 150
12.3.10 !TCNTL!CORRELATION!FILE . . . 151
12.3.11 !TCNTL!CORRELATION!CENTER . . . 151
12.3.12 !TCNTL!CORRELATION!CENTER!SELECT . . . 151
12.3.13 !TCNTL!CORRELATION!PARTNER . . . 152
12.3.14 !TCNTL!CORRELATION!PARTNER!SELECT . . . 152
12.3.15 !TCNTL!SNAPSHOT . . . 152
12.3.16 !TCNTL!SNAPSHOT!FILE . . . 153
12.3.17 !TCNTL!SPAGHETTI!SELECT . . . 154
12.3.18 !TCNTL!SPAGHETTI . . . 154
12.3.19 !TCNTL!SPAGHETTI!FILE . . . 154

12.3.20 !TCNTL!SPAGHETTI!SELECT . . . 155
12.3.21 !TCNTL!TEMPERATURE . . . 155
12.3.22 !TCNTL!TEMPERATURE!SELECT . . . 156
12.3.23 !TCNTL!TEMPERATURE!FILE . . . 156
12.3.24 !TCNTL!SOFT . . . 157
12.3.25 !TCNTL!SOFT!SELECT . . . 157
12.3.26 !TCNTL!SOFT!FILE . . . 158
12.3.27 !TCNTL!NEIGHBORS . . . 158
12.3.28 !TCNTL!MODE . . . 159
12.3.29 !TCNTL!MODE!BOND . . . 159
12.3.30 !TCNTL!MODE!ANGLE . . . 160
12.3.31 !TCNTL!MODE!TORSION . . . 161
12.3.32 !TCNTL!MODE!MODE . . . 162
12.3.33 !TCNTL!OUTPUT . . . 163
12.4 View data using OPENDX[13] . . . 164

13 The Density of States Analysis Tool: paw_dos 165
13.1 Command . . . 165
13.2 Theoretical basis for the paw_dos tool . . . 165
13.2.1 Orbitals . . . 168
13.2.2 Matrix elements . . . 169
13.2.3 Operations . . . 170
13.2.4 Thermal broadening of the Density-of-States . . . 170
13.2.5 Format of the Density-of-States and Number-of-States files 171
13.3 Example for the control input file . . . 172
13.4 Argument description for the control input file . . . 173
13.4.1 !DCNTL . . . 173
13.4.2 !DCNTL!GENERIC . . . 173
13.4.3 !DCNTL!GRID . . . 174
13.4.4 !DCNTL!FILES . . . 175
13.4.5 !DCNTL!FILES!FILE . . . 175
13.4.6 !DCNTL!ORBITAL . . . 176
13.4.7 !DCNTL!ORBITAL!ORB . . . 177
13.4.8 !DCNTL!WEIGHT . . . 179
13.4.9 !DCNTL!WEIGHT!ATOM . . . 181
13.4.10 !DCNTL!WEIGHT!ORB . . . 182
13.4.11 !DCNTL!COOP . . . 183
13.4.12 !DCNTL!COOP!ORB1 . . . 184

13.4.13 !DCNTL!COOP!ORB2 . . . 185

14 The Density of States Plotting Tool: paw_dosplot.x 186
14.1 Command . . . 186
14.2 Example for the control input file . . . 186
14.3 Argument description for the control input file . . . 188
14.3.1 !DPCNTL . . . 188
14.3.2 !DPCNTL!GRAPH . . . 189
14.3.3 !DPCNTL!GRAPH!SET . . . 190

15 The Band Structure Analysis Tool: paw_bands 193
15.1 Command . . . 193
15.2 Example for the control input file . . . 193
15.3 Description . . . 194
15.3.1 BANDSTRUCTURE . . . 195
15.3.2 PDOS . . . 196
15.3.3 Input for a fcc-lattice . . . 196
15.4 Argument description for the control input file . . . 198
15.4.1 !BCNTL . . . 198
15.4.2 !BCNTL!INPUTFILE . . . 198
15.4.3 !BCNTL!BANDSTRUCTURE . . . 198
15.4.4 !BCNTL!BANDSTRUCTURE!LINE . . . 199
15.4.5 !BCNTL!BANDSTRCUTURE!LINE!FATBAND . . . 202
15.4.6 !BCNTL!PDOS . . . 203
15.5 Theoretical background . . . 205
15.6 Diagonalisation . . . 205
15.7 Symmetry . . . 207

16 The Setup-Analysis program: "paw_stpa.x" 208
16.1 Command . . . 208

17 The Structure Pre-Optimization Tool: paw_preopt 210
17.1 Command . . . 210
17.2 Example for the control input file . . . 210
17.3 Argument description for the control input file . . . 211
17.3.1 !PCNTL . . . 211
17.3.2 !PCNTL!GENERIC . . . 211
17.3.3 !PCNTL!FILES . . . 212

17.3.4 !PCNTL!FILES!FILE . . . 212
17.3.5 !PCNTL!OUTPUT . . . 214
17.3.6 !PCNTL!FREEZEONLY . . . 215
17.3.7 !PCNTL!MOVEONLY . . . 215
17.4 Argument description for the structure input file . . . 216
17.4.1 !STRC!ATOM . . . 216
17.4.2 !STRC!BOND . . . 217

18 Scripts **218**
18.1 Options and *getopts* . . . 218
18.2 The Frontend Tool: "paw_do" . . . 218
18.2.1 Command . . . 218
18.3 The Energy Analysis Tool: "paw_show" . . . 219
18.3.1 Command . . . 220
18.4 Copy a project: "paw_copy" . . . 220
18.4.1 Command . . . 220
18.5 change the extension of a et of files: "paw_ext_copy" . . . 221
18.5.1 Command . . . 221
18.6 Collect energies "paw_collect" . . . 221
18.6.1 Command . . . 221
18.7 Collect energies "paw_get" . . . 221
18.7.1 Command . . . 221
18.8 Collect energies "paw_resolve" . . . 222
18.8.1 Command . . . 223
18.9 Scan parameters "paw_scan" . . . 223
18.9.1 Command . . . 223
18.10 Scan lattice constants "paw_scanlat" . . . 225
18.10.1 Command . . . 225
18.11 Murnaghan fit "paw_murnagan.x" . . . 225
18.12 "paw_waittilempty" . . . 226
18.12.1 Command . . . 226
18.13 PAW-snapshot "paw_snapshot" . . . 226

19 Specifications of formatted and unformatted output data files **228**
19.1 Protocoll . . . 228
19.2 waveplot . . . 228
19.3 Projected density of states files . . . 229
19.4 Trajectory files . . . 230

19.4.1 Position Trajectory 230

20 Units and constants **231**
20.1 Vibrations 234
20.2 Magnetic hyperfine parameters 236

21 Periodic Table **238**

22 The Universal Force Field (UFF) **241**

23 Methods **243**
23.1 Friction dynamics and annealing schedules 243
23.2 Optimum friction scheme 244
23.3 Wave function dynamics 245
23.4 Nuclear dynamics 246
23.5 Thermostats 246
23.6 Occupations 247
23.7 Sawtooth behavior of the total energy. 247
23.8 Overcompleteness problem when using too many projector functions 248
23.9 Constraints acting on atoms 248
23.10 Electrostatic decoupling and coupling of point charges 248
23.11 Non-collinear densities 248

24 Troubleshooting **250**
24.1 Errors in the input data 250
24.2 Runtime errors 251
24.3 Overcompleteness of projector functions 252

25 Code documentation **254**
25.1 Strategy 254
25.2 Doxygen 254

A Installation **254**

B Symmetry tables **256**
B.1 Bravais lattices 256
B.2 Spacegroups 257

C Bands **258**
C.1 Face-centered cubic lattice 259
C.2 Body-centered cubic lattice 259

1 Initial remarks

I am frequently changing the CP-PAW program. Therefore, it is unavoidable that this description is incomplete in some places, that it may lists options that are no longer supported, or that it contains errors. Please let me know if you find something unclear or even incorrect. Other users will be grateful.

I have attempted to make the program test for inconsistencies of the input data, such as the selection of conflicting options. You will find that the program stops in such cases and tries to advise the user concerning what has gone wrong. New users are particularly creative in the combinations of options they choose. If you run into problems that the program does not detect, for example if it crashes without giving a useful message, please let me know *before* you get used to working around the problem. Your input is particularly valuable in making the the code more secure to use.

Any other suggestions on how to improve the clarity of this description or the code are most welcome.

Programs become outdated nearly as quickly as newspapers. Note, that the executable of this code may also have an expiration date. The code will print a warning about a month before it expires. Please make sure that you know the expiration date when you receive an executable. If the license expires, discuss with your contact person about obtaining an extension of the license and a new executable program.

Note that the CP-PAW code including all related material is copyrighted. You require a valid license to execute it.

2 What is the projector augmented wave method?

The projector augmented wave method [2] is an all-electron electronic structure method, which allows accurate electronic structure calculations and ab-initio molecular dynamics simulations on the basis of density functional theory.

What is all that?

Density functional theory (DFT) [3, 4]. Density functional theory describes the ground state of a many-electron system by electrons that do not interact other than through an effective potential that depends on the electron density. It is based on an exact theorem, which specifies that such a description, based on the electron density rather than on the electronic many-particle wave function, be rigorously possible for ground states. In practice the density functional, which also defines

the effective potential as a functional of the density, is not exactly known. However, highly successful approximations have been found. In contrast to Hartree-Fock calculations, density functional theory explicitly treats electron correlation. The accuracy is typically comparable to that of MP2 calculations, i.e. only a few kcal/mol [5, 6].

Ab-initio molecular dynamics (AIMD) [7, 8, 9] is an extension of traditional electronic structure methods which has been invented in 1985 by Roberto Car and Michele Parrinello [7]. The best way to think of it is as a series of electronic structure calculations, one for each time slice, for always different atomic positions. From one time slice to the next, the atomic positions are changed according to Newton's equations of motion $M_i \ddot{R}_i = F_i$. Here M_i is the mass of a nucleus, R_i the position, F_i the force acting on the nucleus as calculated from the electronic structure, and the double-dots stand for the second time derivative. Self-consistent iterations at each time step are avoided by a dynamical evolution of the wave functions, and thus simulations of several picoseconds are possible, which is sufficient to simulate directly chemical reactions and diffusion with low barriers or at high temperatures.

Whereas the basic idea of ab-initio molecular dynamics is to perform real-time and finite temperature simulations, it can be used like a traditional electronic structure method – using a friction to "cool" the temperature to zero – and it has been combined with statistical approaches to study processes with large barriers.

The **projector augmented wave method (PAW)** [2] has been developed by the author in response to the invention of the ab-initio molecular dynamics approach. Whereas the latter was based on the plane wave pseudopotential approach, a new method was needed to enhance the accuracy and computational efficiency of the approach and to provide the correct wave functions, rather than the fictitious wave functions provided by the pseudopotential approach. The PAW method describes the wave function by a superposition of different terms: There is a plane wave part, the so-called pseudo wave function, and expansions into atomic and pseudo atomic orbitals at each atom. On one hand, the plane wave part has the flexibility to describe the bonding and tail region of the wave functions, but used alone it would require prohibitive large basis sets to describe correctly all the oscillations of the wave function near the nuclei. On the other hand, the expansions into atomic orbitals are well suited to describe correctly the nodal structure of the wave function near the nucleus, but lack the variational degrees of freedom for the bonding and tail regions. The PAW method combines the virtues of both numerical representations in one well-defined basis set.

Of course, one does not want to make two electronic structure calculations –

one using plane waves and one with atomic orbitals –, and thus double the computational effort. Therefore, the PAW method does not determine the coefficients of the "atomic orbitals" variationally. Instead, they are unique functions of the plane wave coefficients. It is possible to break up the total energy, and most other observable quantities, into three almost independent contributions: one from the plane wave part and a pair of expansions into atomic orbitals on each atom. The contributions from the atomic orbitals can be broken down furthermore into contributions from each atom, so that strictly no overlap between atomic orbitals on different sites need to be computed.

The PAW method is in principle able to recover rigorously the density functional total energy, if plane wave and atomic orbital expansions are complete. This provides us with a systematic way to improve the basis set errors. The present implementation uses the frozen core approximation, even though the general formalism allows extensions in this respect. It provides the correct densities and wave functions, and thus allows us to calculate hyperfine parameters etc. Limitations of plane wave basis sets to periodic systems (crystals) can easily be overcome by making the unit cell sufficiently large and decoupling the long-range interactions [10]. Thus the present method can be used to study molecules, surfaces, and solids within the same approach.

3 How the code is built

This section is about programming philosophy. I write this up because I myself often wonder when I use other software why something is done in a particular fashion. Therefore, I shall present my reasoning here. Feel free to skip this section if you wish.

The program is written in an **OO (object-oriented)** manner. This means that it consists of agents (objects) that perform certain operations or provide the selected information. Each agent holds the data needed to its job, or it can request the data it needs from other agents. This is a major software strategy, widely used today, which allows the programmer to hide certain details, he need not really worry about at the present level of programming. It also allows him to assemble the code from little "boxes", which are easy to maintain and enhance.

As part of the OO design, the program uses its own low level object library, which customizes a number of common operations, such as interproceessor communication, error handling, file handling, tree-linked-list structures for intuitive IO, periodic table, constants, DFT functionals, tracing, timing, string handling,

and a few more. These low-level objects are rather unspecific to the PAW code, and can be used in combination with self-developed analysis tools. (However, they are still subject to the license agreement for the PAW code.)

The language used is **FORTRAN90**. FORTRAN90 is itself not an OO language such as C++ and Java, and thus limits the possibilities in this respect. However, it has a number of advantages for number crunching, such as good compilers and, for my taste, a natural and easily comprehensible way to write mathematics. Compared to FORTRAN77 it is a significant advance towards the OO features of C++ or Java because it incorporates features such as dynamic memory allocation, derived data types (structures), operator overloding, modules, etc. The option of using templates has been implemented using a self-made preprocessor. FORTRAN90 is largely compatible with FORTRAN77, so you need to pick up only a few new things if you want to work with the existing tools.

The program allows **parallel processing** using MPI (Message Passing Interface) [11]. It is (almost) scalable in central memory and CPU time. The scalar version program is identical to the parallel version with the exception that a dummy interface is used instead of MPI.

The program relies heavily on linear algebra packages such as LAPACK, BLAS and FFTW. These libraries takes care of basic computations such as matrix multiplications and FFTs (Fast Fourier transforms). A large fraction of the total CPU time is spent in these routines. In this way the code development concentrates on algorithmic developments and not, for example, on how to optimize a FFT. The latter will be done by experts who continually adapt this library to modern computer architectures.

One programming principle, I try to follow, is the engineering motto **"Fewer Parts!"**. For the user this implies that he will find few instances where two options provide the same functionality. I hope that the limitations will be offset by clarity.

The program uses **Hartree atomic units**, that is $\hbar = e = m_e = 4\pi\epsilon_0 = 1$, and Cartesian coordiantes. Angles are handled in radian ($2\pi \text{ rad} = 360 \text{ deg}$). The conversion to other units is often done during printing. The conversion factors are provided by a particular agent (see section 20), which is based on the values of fundamental physical constants recommended in 1986 by CODATA (Committee on Data for Science and Technology of the International Council of Scientific Unions) [12].

The program divides work into **three steps: simulation – analysis – visualization**. The program consists of the simulation code, which is the core of CP-PAW, and a number of tools used to analyze the results. The simulation code calculates energies, densities, wave functions, trajectories, etc., and writes the re-

sulting data into files. These files are then read by tools, which collect the desired information, and bring it into the desired form, typically as another file that can be read by your graphics utilities.

Why this three-layer strategy?

Analysis is both an iterative process and an art: when you find something interesting and want to take a closer look. Sooner or later, you will probably want to make your own analysis tools, because you have found a way to understand your data that nobody has thought of before. Or you want examine a property, which nobody has tried to calculate. Or you have a unique visualizer and need an interface for it. And because you want to discuss this particular result with your collegues at the upcoming coffee break, you choose the quick-and-sloppy approach. You do not want to do this inside the simulation code, because you dare not jeopardize its operation.

In contrast to analysis, simulation is computationally expensive. Therefore, it is desirable to archive as much data from the simulation as possible, and be able to go back later and look at it again. Hence the simulation writes most data rather unspecifically in machine-readable form (which saves a considerable amount of disk space). The data are written in a simple format so that it is easy to read them into the analysis tools.

Visualization is also separated from analysis, because many tools exist and the choice is a matter of taste and wealth. The analysis tools that come with the CP-PAW code will write data formats for the visulatization tools that we currently use. (Those are currently the IBM Dataexplorer [13] for 3D representations of the wave functions or trajectories, xmgrace [14] for x-y plots, and CryMolCAD[15] for the analysis of the atomic structure.) However, you can easily change the format to adapt them to your preferred graphics programs.

Of course you can see the most important information, such as total energies and one-particle energies directly while the simulation is proceeding. You may also contact other users about further analysis tools and exchange them. If you wish to write a graphical interface that hides all three steps from the user, feel free to do so.

4 Input data structure

The input data of the simulation code and the tools uses a format that attempts to be both general and intuitive. Logically, the data are arranged in a tree structure similar to pull-down menues or directory trees in unix. It allows one to hide

options of the program that the user may not be interested in, which then can be handled by default values, and it avoids the unnecessary restrictions of formatted input. The PAW library has objects that can handle such structures easily, so that it is widely used for data input for both the simlation code and the analysis tools. The following section shall make you familiar with the general layout of input data.

4.1 Syntax rules for the input data

Input data are structured as nested data blocks, each identified by a key word. Each data block may contain other data blocks and data. Data are again specified by keywords.

The following simple expample shall illustrate the structure

```
!FIRSTBLOCK
  DATA=5
  !SECONDBLOCK OTHERDATA=T !END
  !SECONDBLOCK OTHERDATA=F !END
        # this is a comment. The entire line is ignored.
  !THIRDBLOCK_OFF TEXTDATA='THIS IS A TEXT' !END
!END
!EOB
```

The indentation and the arrangement of the data is arbitrary. The only requirements are that data and keywords be separated by blanks or line breaks, and that their sequence observes the logical tree structure described below.

Lines which begin with hash (#) are ignored while parsing the input file. This is one mechanism to include **comments**. Another one is to add text beyond the `!EOB`.

Every block starts with a block identifier and ends with the string "!END". The identifier starts with an exclamation mark such as !FIRSTBLOCK. (Of course one can use any other name instead of "FIRSTBLOCK". The recognized block key words are described in the later sections of this manual.) The last block must be followed by "!EOB" (End-Of-Buffer) to indicate the end of all data blocks. Each block, together with its data and any contained subblocks, can be made invisible by appending "_OFF" to the block name as done for !THIRDBLOCK in the example. Data blocks may occur multiply such as !SECONDBLOCK, if specified in this manual. The order in which the data blocks are given is irrelevant as long as

their hierarchy is observed. An exception are multiple data or datablocks, where the order in which they occur may or may not be significant. Each data refers only to its block. For example, the two occurrences of OTHERDATA= are different even though their key words are identical, because they are within different subblocks.

The general format of input data is a key word, as specified in this manual separated by an "=" sign from the input data. The input data can be simple data or arrays. Higher dimensional arrays are treated like in Fortran with the first dimension incremented first, such as a(1,1) a(2,1) a(1,2) a(2,2). The data are read in free format.

Note that a mistyped key word makes the data or entire branches of the tree structure invisible. There is no way to warn the user if some key words have not been recognized. If the same key word is given several times in a given data block, without being specified as a multiple, only the first occurrence is recognized. The same is true for data blocks. It is therefore recommended that the protocol be checked to determine, whether all data have been used as intended.

The type of data is determined as follows:

- if a data contains a single or double apostrophe, it is assumed to be of character type,
- if a data contains an open parenthesis, it is assumed to be of complex type,
- if a data is either T, F, .true. or .false. it is assumed to be of type logical,
- if a data contains a period it is assumed to be of type real,
- otherwise, the data is assumed to be of type integer.

These rules are checked in the order given here. It is allowed to precede a data item by an integer and a multiplication sign such as $3 * 0.$, which is shorthand for 0. 0. 0. (In some cases, one is allowed to provide both integer and real. Therefore, if you provided a number of the wrong type and, against your expectation, the program does not complain, a conversion has been done by rounding the number to the nearest integer or a simple type conversion from integer to real has occured.)

The description uses the following notation. Data blocks are indicated by a frame. The enclosed name contains the entire hierarchy of data blocks. Only the last one must be specified on the date file. However, this data block must be enclosed by the higher level data blocks. The key words relate to the individual data. If we cross reference data, we use the entire hierarchy of parent blocks, followed by a colon and the data keyword.

4.2 Extended notation for atoms

The PAW code refers to atoms by name. No two atoms must be given the same name. Arbitrary names, given a length limit, are allowed. However, the recommended notation choses the first two characters as the element symbol, with blanks replaced by underscores, and the remainder is a number. Starting the name with the element symbol allows to exploit some default settings. The names are defined in the structure input file and can be referred to in other specifications and by the PAW tools.

In solids, it will be neccesary to distinguish between periodic images of the same atom. An example is a bondlength constraint between two atoms, of which one is a periodic image of a original atom. In that case we use in some cases an extended naming convention.

In the **extended notation the atom name** has the form "*name*:*ijk*", where *name* is the name of the atom in the original atom cell. and *i,j,k* are the three translations along the lattice vectors. The latter can be single digit integer numbers with or without preceeding sign. To give a few examples "H_2:100", "H_2:+1+0+0", "H_2:10-1".

Note, that the translations do *not* specify a unit cell that contains that atom. For example, an atom that moves about will never change its lattice translations, despite traversing several unit cells. Rather the atoms are considered first as a initial cluster, which forms a lattice by repetition and lattice translations. There is no restriction that the atoms of the initial cluster are confined in one unit cell.

The extended notation will be used increasingly. However, please refer in all cases to the manual before using the extended notation. It is also mandatory that original atom names are chosen such that no confusion can occur. A simple rule that avoids confusion, is not to use colons in atom names.

5 Before starting...

In this manual, I frequently refer to $PAWDIR and $ROOT. They are explained in the following:

5.1 The PAW directory

$PAWDIR is the name of the directory where the PAW code is installed.

In the directory $PAWDIR you will find the following directories

- `src` contains the source codes. The sources for the analysis tools are located in a subdirectory `Tools`.
- `doc` contains the manual and installation notes
- `bin` will be created if it does not exist. It contains the executable codes
- `dx` contains files required by the dataexplorer to visualize wave functions, densities and atomic structures

If the program is set up properly, you will have an environment variable `$PAWDIR`, which refers to the directory where the PAW code is installed. You can try it with "`echo $PAWDIR`", which will print the PAW directory. Then you can go to the PAW directory using "`cd $PAWDIR`", and see its contents using "`ls -l`".

5.2 The project data structure

All files for a given system should be located in one directory, and it is recommended that this directory contain only that project. All filenames belonging to that system will have a common root, which we denote in a Unix-like fashion by $ROOT (the value of the variable "`ROOT`"). The individual files are distinguished by their extensions. For example, you can look into the directory "`$PAWDIR/Sample`". Here you will find a number of files that begin with "`h2co`". In this case, the root for the filenames ($ROOT) is "`$PAWDIR/Sample/h2co`". Note that te root is always specified by its absolute path. The filenames have extensions of the kind "`.cntl`", "`.strc`", "`.rstrt`", etc. Each extension defines a particular role of the file within the project. For example, in the file "`$ROOT.cntl`" one defines which operations the simulation code shall do, in the file "`$ROOT.strc`" one defines the molecule or crystal to be studied, and the file "`$ROOT.rstrt`" holds, among other data, the instantaneous atomic coordinates and electron wave functions that the simulation code needs to know in order to continue the simulation.

You can make exceptions to this rule and define the filenames explicitly. This may be useful, for example, if you wish to store the bulkiest file elsewhere, because you have run out of space on the disk, where you store the projects.

Note that, unlike $PAWDIR, your environment does not know what $ROOT is. Therefore, you should use $ROOT in this manual as a placeholder for the full root name, unless the variable "`ROOT`" is explicitly defined.

6 The simulation code

6.1 How to perform a simulation

6.1.1 Input files

In order to run the simulation code, two input files have to be prepared:

1. The **structure input file** (or often simply called STRC), describes the system to be studied. Here you provide information about which atoms you wish to simulate, what their specifics are, whether you wish to simulate a molecule or a crystal and so on.

2. The **control input file** (sometimes abbreviated as CNTL) describes what the program shall do, such as optimizing the wave functions, relaxing the atoms or simlating at finite temperature.

After the simlation has finished, the program writes a restart file, which holds the actual wave functions and the positions of the atoms. This file holds all the necessary information to continue a simulation from the point where you finished. It is important to keep the restart file and the structure file if you wish to return later to what you have done.

6.1.2 Execute the simulation code

The simulation code is executed using the command

```
paw_fast.x  controlfile 1>err 2>&1 &
```

where *controlfile* is the name of the control input file described in the next section. With `1>err` one redirects the print statements to the file out. If the file existed before it is overwritten. This information is normally irrelevent and is used for development purposes. However, it is crucial for tracing errors. With `2>&1` one redirects also error messages into the same file, namely `err`. Thus also error messages be it from PAW or from the system are written in the same directory. The last `&` simply sends the job into the background. That is you can continue working in the same window while the job is running.

`paw_fast.x` is simply the name of the PAW executable. There will be several executables. If you are using a parallel computer using MPI you will need an executable that starts with `ppaw` instead of `paw`. The executable with `fast` is

the one compiled with all reasonable optimization flags for the compiler. For all normal purposes this one should be used. In addition there may be an executable with `dbg` instead of `fast` for debugging purposes and one with nothing, with no special flags, which can be used to find errors induced by optimization.

You can also obtain information about the executable

`--help`	print help information
`-h`	print help information
`?`	print help information
`--version`	print version information
`--parmfile`	print parameter file used for compilation

The version information shall allow to correlation the executable to a well-defined version of the source code. The parameter file in addition provides information about what libraries are linked and which compilation flags have been used etc. This information is important to track bugs in the code.

Sometimes it is convenient to write a little wrapper shell-script for the execution command.

```
#!/bin/sh
#=====================================================
# sample doit file to execute the simulation code ==
#=====================================================
#                                    define the rootname
ROOT=/home/user/Tree/Testrun/H2CO/h2co
#                                  execute paw simulation
paw_fast.x ${ROOT}.cntl 1>${ROOT}.err 2>&1
```

This file would be called `$ROOT.doit`, which is also the command to execute the code. Such a wrapper avoids some unnecessary typing if you run the job repeatedly. It can easily be modified to execute the simulation code several times with different control files.

What does the wrapper do? Let us discuss it line by line. (Comment lines, i.e. those starting with a hatch sign, are not discussed.)

1. The first line `#!/bin/sh` simply specifies that the wrapper is a shell script

2. The second line defines a variable `ROOT`. In the following commands $ROOT is shorthand for `/home/user/Tree/Testrun/H2CO/h2co`. Thus one can use the same wrapper for different simulations by simply changing the value of `ROOT`.

The last line executes the the simulation code using the control file `$ROOT.cntl`.

In order to track the simulation one can issue the command

```
tail -f $ROOT.prot
```

which continually prints the lines written to the protocoll file also to the screen. Another way to trace the simulation is the command `paw_show` described in section 18.3.

6.1.3 Terminate the simulation

The execution can be stopped before regular completion by creating a so-called exit file

`touch` *exitfile*

where *exitfile* is the name of the exit file. The standard name is $ROOT`.exit`, but this name can be changed in the control input file.

As an alternative to setting the exit file, the CP-PAW code also contains a signal trap. The command

`kill -30` *PID*

where *PID* is the process id number, causes the program to terminate the execution after the next iteration. Note, however, that this command can sometimes crash the code, namely if the code is writing to a file at the moment you issue the command. Its use is therefore not recommended, and it may even be removed from future versions. The process id number can be obtained either from the command `top` or from the command `ps -elf |grep paw`.

6.2 The control input file "CNTL"

The control file is responsible for "what to do" in the simulations. These data are generally unspecific to the system.

6.2.1 Examples for the control input file "CNTL"

The following is a particularly simple example for a control input file. This file can be used for getting started.

```
!CONTROL
  !GENERIC START=T NSTEP=100 !END
  !FOURIER EPWPSI=30 CDUAL=2 !END
  !PSIDYN
    !AUTO FRIC(+)=0.3 FACT(+)=1.
          FRIC(-)=0.3 FACT(-)=0.97 !END
  !END
!END
!EOB
```

The following example illustrates a few more options that give a quick impression of the choices available. The detailed description of all options is given in the next section.

```
!CONTROL
 !GENERIC START=T DT=10 NSTEP=100 NWRITE=100 !END
 !DFT     TYPE=2 !END
 !FOURIER EPWPSI=30 CDUAL=2 !END
 !PSIDYN  STOP=T FRIC=0.005 RANDOM=0.1
          MPSI=1000 MPSICG2=0.5
   !AUTO  FRIC(-)=0.3 FACT(-)=0.97
          FRIC(+)=0.5 FACT(+)=0.97 !END
   !THERMOSTAT_OFF STOP=T FRIC=0
          T[K]=100 FREQ[THZ]=10 !END
 !END
 !RDYN    STOP=T FRIC=0 RANDOM[K]=0
   !AUTO  FRIC(+)=0.5 FACT(+)=0.97
          FRIC(-)=0.3 FACT(-)=0.97 !END
   !THERMOSTAT_OFF STOP=T FRIC=0
          <EKIN>=0.02 FREQ[THZ]=50 !END
 !END
 !FILES
   !FILE  ID='EXIT' EXT=F NAME='~/EXIT' !END
 !END
```

```
 !ANALYSE
   !HYPERFINE ATOM='H_1' EFG=T ISOMERSHIFT=T
              FERMICONTACT=T ANISOTROPIC=T   !END
   !DENSITY FILE='./density.ev' TYPE='TOTAL'
         DR=0.4 OCC=T DIAG=T CORE=F !END
   !WAVE FILE='./wavefunction.wv'
         DR=0.4 B=10 K=1 S=1 IMAG=F !END
   !POTENTIAL FILE='./potential.wv'
         dr=0.4 !END
 !END
!END
!EOB
```

6.2.2 Argument keywords for the control input file "CNTL"

6.2.3 !CONTROL

Rules: optional
Description: defines the operations performed on the system; largely independent of the system

6.2.4 !CONTROL!FILES

Rules: optional
Description: specifies the file names that deviate from the standard values

ROOT

rootname. Files defined as extension will have this name combined with the extension. All files connected as default are defined as extensions.

Type: character
Rules: optional
Default: string preceding the '.cntl' ending of the control input file

6.2.5 !CONTROL!FILES!FILE

Rules: optional, multiple
Description: Specifies one file

ID

identifier for the file; options are:

'PROT'	protocol; extension: `.prot` monitors the simulation.
'STRC'	structure input file; extension: `.strc` defines atoms, structure, electron occupations etc.
'CNTL'	control input file; extension: `.cntl` used to control the simulation.
'PARMS_STP'	setup parameter file; default: `$PAWDIR/parameters/stp.cntl` defines the parameter sets for the augmentation.
'RESTART_IN'	input restart file; extension `.rstrt` instantaneous coordinates
'RESTART_OUT'	output restart file; extension `.rstrt` see 'RESTART_IN'
'EXIT'	exit file; extension: `.exit` simulation terminates if exit file exists
'PDOS'	projected density of states; extension: `.pdos` used by the paw_dos tool generated by the main paw program.
'PDOSOUT'	projected density of states; extension: `.pdosout` used by the paw_dos tool. generated by the paw_bands tool.
'BANDDATA'	paw hamiltonian for band structure; extension: `.banddata` used by the paw_bands tool.
'CONSTRAINTS'	constraint report; extension: `_constr.report`
'POSITION_TRAJECTORY'	atomic positions; extension `_r.tra` trajectory used by the paw_tra tool.
'BANDS_TRAJECTORY'	one-particle energies; extension `_r.tra` (not yet used)
'AUGPARMS'	parameter sets for the augmentation; extension: no default extension

Type: character
Rules: mandatory
Default: none

NAME

filename. Can be the relative file name or an extension to the PAW "root". Standard output can be specified by NAME='stdout' and EXT=.false.
Type: character
Rules: mandatory
Default: none

EXT

.true.: NAME specifies the extension only/ .false.: full name
Type: logical
Rules: optional
Default: .false.

6.2.6 !CONTROL!GENERIC

Rules: optional
Description: general data that do not fit into other blocks

START

T: start with random wave functions, and atomic positions from file "STRC"
F: wave functions and atomic positions are taken from restart file, unless specified otherwise
Type: logical
Rules: optional
Default: F

NSTEP

number of time steps
Type: integer
Rules: optional
Default: 100

DT

time step Δ in a.u. (1 a.u.$\approx$ 0.024 fs)
Type: real
Rules: optional
Default: 10.0

NWRITE

Every "NWRITE" time steps, the program writes detailed information into the protocoll file and it updates the restart file. Note that writing the restart file is time consuming.
Type: integer
Rules: optional
Default: 100

FORCES

T: evaluate the forces on the atoms also when the atoms are fixed (no !CONTROL!RDYN), and print them at full precision at the end of the run (lines "FORCE name F_x F_y F_z" in H/a_0 and eV/Å, on standard output and in the protocol file).
Type: logical
Rules: optional
Default: F

STRESS

T: evaluate the stress tensor also for a fixed cell (no !CONTROL!CELL) and print it at the end of the run: $\sigma = -\frac{1}{V}\frac{dE}{d\epsilon}$ in GPa (positive when the cell wants to expand), $dE/d\epsilon$ in Hartree, and the pressure $\mathrm{tr}\,\sigma/3$. The analytic stress is taken at a fixed set of plane waves; it differs from a finite difference at fixed cutoff by the Pulay stress, which vanishes with increasing EPWPSI.
Type: logical
Rules: optional
Default: F

TRACE

provides trace information when entering or leaving subroutines under trace control. Used for debugging purposes.
Type: logical
Rules: optional
Default: .false.

ENDIAN

defines whether unformatted files are written in little endian as typical on intel computers or in big endian as on ibm computers. The value can be 'little','intel','big','ibm'. The first two values have identical meaning and the last two have identical meaning. It is only functional using certain compilers (absoft).
Type: character
Rules: optional
Default: little

RUNTIME

A soft stop is initialized after the given time has elapsed since start of the code. Runtime is specified as a three element vector containing (hours,minutes,seconds). The three elements of the vector are internally converted into seconds and added up.
Type: integer
Rules: optional
Default: ∞

AUTOCONV

The autopilot defines a strategy to vary the friction for various dynamical variables, to test the convergence and to terminate the optimization loop. The decision to terminate is taken if the energy remains within an energy window of 10^{-5} H for autoconv time steps.

Type: integer
Rules: optional
Default: 20

RSTRTTYPE

allows to cut down the information on the restart file. Option RSTRTTYPE='STATIC' only stores one set of wave functions. This reduces the amount of diskspace by nearly a factor of two. This option should not be used in a dynamical simulation, because, the velocity of the wave functions will be set to zero.

Type: character
Rules: optional
Default: NONE

6.2.7 !CONTROL!DFT

Rules: optional
Description: selects density functional parameterization; default is Perdew-Zunger parameterization of the Ceperley Alder quantum Monte Carlo calculation

TYPE

exchange-correlation functional, given as a number or (where listed) as a name; names are case insensitive and also accept the VASP `GGA` tags. Functionals marked ✓ agree with libxc (and the VASP implementation) to machine precision, see Table 2.1 of the theory book.

Local density approximation

(1) `'PZ'`: Perdew-Zunger [16] parameterization of Ceperley-Alder [17] ✓;
(2) `'PW92'`: Perdew-Wang 92 [18] parameterization of Ceperley-Alder ✓;
(3) local (Slater) exchange only, X_α with $\alpha = 2/3$ ✓;
(4) X_α with $\alpha = 0.7$.

Generalized gradient approximations

(10) `'PBE'` or `'PE'`: Perdew-Burke-Ernzerhof [19] (`GGA=PE`) ✓;
(11) `'RPBE'` or `'RP'`: RPBE of Hammer, Hansen and Nørskov [20] (`GGA=RP`) ✓;
(12) `'PW91'` or `'91'`: PW91 exchange and correlation [21] (`GGA=91`) ✓;
(13) `'REVPBE'` or `'RE'`: revPBE, PBE with $\kappa = 1.245$ [22] (`GGA=RE`) ✓;
(14) `'PBESOL'` or `'PS'`: PBEsol, $\mu = 10/81$ in exchange and $\beta = 0.046$ in correlation [23] (`GGA=PS`) ✓;
(5001) `'BLYP'`: Becke-88 exchange [24] and Lee-Yang-Parr correlation [25] ✓;
(81) BP86: Perdew-Zunger + Becke-88 exchange + Perdew-86 correlation [26] ✓;
(8) like (81), but on Perdew-Wang 92 local correlation (not the standard BP86);
(7) Becke-88 exchange + Perdew-Wang 92 correlation ✓;
(71) Becke-88 exchange + Perdew-Zunger correlation;
(9) Becke-88 exchange + PBE correlation ✓;
(6) Becke-88 exchange only, no correlation ✓.

Meta-generalized gradient approximations

(21) `'SCAN'`: SCAN [27] with the orbital kinetic-energy density; requires `TSCANDELOC=T` (below) for the plane-wave and one-center parts. Exchange is spin scaled, correlation fully spin polarized (`METAGGA=SCAN`) ✓;
(22) `'R2SCAN'`: r^{2}SCAN [28], the regularized and restored SCAN, with the orbital kinetic-energy density; requires `TSCANDELOC=T` like SCAN; spin treatment as for SCAN (`METAGGA=R2SCAN`) ✓. Checked against libxc to 10^{-14}. r^{2}SCAN is numerically smoother than SCAN: its band gaps change little with the cutoff, while SCAN gaps scatter by a few 0.01 eV. Gaps compared with VASP 6.4.2 (PAW_PBE): Si

Type: integer or character
Rules: optional
Default: 1

TSCANDELOC

evaluate the orbital-tau meta-GGA, SCAN (TYPE=21) or r^{2}SCAN (TYPE=22), for the plane-wave density and the one-center terms, i.e. use it everywhere. Needed for TYPE='SCAN' and TYPE='R2SCAN', and for SCAN0r and r^{2}SCAN0r.
Type: logical
Rules: optional
Default: F

VDW

Adds van der Waals Interactions[32]. The van der Waals interaction is described by an interatomic pair potential that has a long-ranged r^{-6} behavior and which is cut of smoothly at short distances.
Type: logical
Rules: optional (Untested)
Default: F

VDW-3BODY

adds three-body interactions[32] to the pair-wise van der Waals interactions of Grimme et al. (As of March 2012, not yet recommended by Grimme)
Type: logical
Rules: optional, only used when VDW=T, (Untested)
Default: F

ANGULARXC

treatment of the non-spherical one-center density in the LDA/GGA exchange-correlation energy. 'TAYLOR': second-order expansion about the spherical density (the CP-PAW scheme); 'EXACT': the density and its full gradient are evaluated on an angular quadrature grid (14x24 points) and the potential is the exact derivative of that energy (as VASP's LASPH). 'EXACT' is about twice as expensive for transition-metal oxides.

Type: character
Rules: optional; collinear spin only
Default: 'TAYLOR'

SPINORBIT

adds spin-orbit coupling $\xi(r)\vec{L}\cdot\vec{S}$ inside the augmentation spheres, $\xi = \frac{1}{2M^2c^2}\frac{1}{r}\frac{dV}{dr}$ with the scalar-relativistic mass $M = 1 - V/(2c^2)$ and the spherical all-electron potential of the isolated atom, evaluated between all-electron partial waves inside the covalent radius. Requires noncollinear spin (`!STRUCTURE!OCCUPATIONS NSPIN=3`). Si: $\Delta_{so}(\Gamma) = 48$ meV.

Type: logical
Rules: optional
Default: F

DIPOLECORR

dipole correction for slabs (Bengtsson, PRB 59, 12301 (1999)). The slab normal is the third lattice vector; a sawtooth potential with its step in the middle of the largest vacuum gap cancels the artificial field of the periodic images. Adds $2\pi m^2/\Omega$ to the energy. The stress is not corrected.

Type: logical
Rules: optional
Default: F

6.2.8 !CONTROL!DFT!NTBO

Rules: optional
Description: Experimental option. Do not use! Constructs wave function representation in natural tight-binding orbitals. Enables to add the additional terms for the hybrid functionals or the interface for a solver for the dynamical mean-field theory.

MODUS

switch betwee different energy contributions. May be 'HYBRID' or 'DMFT'.
Type: character
Rules: optional
Default: 'HYBRID'

OFFSITE

Calculates off-site U-tensor. OFFSITE=T is only compatible with MODUS='HYBRID'.
Type: logical
Rules: optional
Default: false

HFWEIGHT

Mixing Factor for replacement of the Fock term.
Type: real
Rules: optional
Default: 0.25

K2

K2=2 Ekin[H] of the envelope function. The bare Hankel function decay faster with decreasing k2.
Type: real
Rules: optional
Default: -0.25

SCALERCUT

Atom pairs are included in the neighborlist for structure constants, if their distance is less than the sum of covalent radii multiplied with SCALERCUT. (The covalent radius of an empty sphere is set to 1 a_0.

Type: real
Rules: optional
Default: 2.

6.2.9 !CONTROL!FOURIER

Rules: optional

Description: Plane wave cutoffs $E_{PW} = \frac{1}{2}G^2_{max}$ for wave functions and charge density. A cutoff of 30 Ry=15 H for the wave function and CDUAL=2 is sufficient for most applications. In order to account for all plane wave components in the density, the plane wave cutoff for the density should in prinicple be four times the cutoff for the wave function. (For low cutoffs, even CDUAL>4 may be required since the exchange and correlation functional is nonlinear, and therefore the energy may not necessarily decrease monotonically as the basis set size is increased with a fixed CDUAL. In practice this is rarely a problem.)

EPWPSI

plane wave cutoff for the wave functions in Rydberg (1 Ry=0.5 a.u.). All plane waves up to a maximum kinetic energy equal to the plane wave cutoff are considered. Note that the plane wave cutoff refers only to the plane wave part of the basis functions.

Type: real
Rules: optional
Default: 30.

EPWRHO

plane wave cutoff for the charge density in Rydberg (1 Ry=0.5 a.u.)
Type: real
Rules: optional
Default: 4*EPWPSI

CDUAL

EPWRHO=CDUAL*EPWPSI; The default cdual=4 produces the correct result; cdual=2 is often a good choice.
Type: real
Rules: optional; this value is overwritten by any occurance of "EPWRHO"
Default: 4

EPWBUCKET

Used mostly for cell dynamics. See section 23.7 for an explanation of the sawtooth behavior. Parameter E_B in Ry specified as follows: If specified an additial potential energy term $\sum_{G,n} f_n \langle \tilde{\Psi}_n | G \rangle B(G) \langle G | \tilde{\Psi}_n \rangle$, is added top the total energy where $B(G) = c_B \theta(\frac{1}{2}G^2 - E_B)(G - \sqrt{2E_B})^2$. When this term is set to a fixed value, the plane wave convergence is artificially accelerated, which is important to evaluate stresses. The energy converged this way corresponds however not to equal to the energy converged without this term, but corresponds to the total energy obtained with a plane wave cutoff slightly higher than EPWBUCKET. Example: $E_{PW} = 50$ Ry, $E_B = 30$ Ry, $c_B = 1$
Type: real
Rules: optional;mandatory of BUCKETPAR is specified
Default: none

BUCKETPAR

Parameter c_B see also description of EPWBUCKET above.
Type: real

Rules: optional; mandatory, if EPWBUCKET is specified
Default: 0.

6.2.10 !CONTROL!PSIDYN

Rules: optional; default uses default values for the electron dynamics.
Description: Parameters used to control the dynamics of the wave functions. The wave function dynamics is governed by the equation

$$m_\Psi |\ddot{\tilde{\Psi}}_n\rangle = -\frac{\partial E}{\partial \langle \tilde{\Psi}_n|} - m_\Psi |\dot{\tilde{\Psi}}_n\rangle \alpha - \sum_m |\tilde{\Psi}_m\rangle \Lambda_{mn},$$

where α can be tuned by a Nosé thermostat or the parameters below. The equation of motions are integrated using the Verlet algorithm [33]. The friction parameter α is converted into a parameter $c_\alpha = \alpha\Delta/2$, which can range from 0 to 1. A value of $c_\alpha = 0$ indicates frictionless dynamics, whereas a value of $c_\alpha = 1$ indicates steepest descent. Δ is the time step. A discussion of the virtues of friction dynamics can be found in Ref. [34]

STOP

if STOP=true start with zero velocity of the wave functions
Type: logical
Rules: optional
Default: F

MPSI

mass m^0_Ψ for the wave function dynamics. The wave function mass is an operator of the form

$$\hat{m}_\psi = \sum_{\vec{G}} |\vec{G}\rangle m^0_\psi (1 + c\vec{G}^2)\langle\vec{G}|$$

The coefficient c is set in MPSICG2.
The G-dependent wave function mass is used to reduce the otherwise very large frequency of the wave function components with large $|\vec{G}|$. For the large G-components of the auxiliary wave functions we can approximate the effective potential by a constant, which makes the Hamiltonian diagonal in $|\vec{G}\rangle$. This allows to estimate the frequency from

$$\hat{m}^0_\psi (1 + c\vec{G}^2)\ddot{\Psi}_G = \left[\frac{\vec{G}^2}{2} + v_{eff}\right]\Psi_G$$

Type: real
Rules: optional
Default: 10 Δ^2 (for Δ see "!CONTROL!GENERIC:DT")

MPSICG2

high-G enhancement c for the wavefunction mass. The fictitious electron mass for the wave function dynamics is G-dependent $m_\Psi(G) = m_\Psi(1 + cG^2)$. A recommended value is $c = 0.5$. A reasonable value is $\frac{1}{2}(\frac{x}{2\pi})^2\frac{\Delta^2}{m_\Psi}$, where x is the fraction of shortest oscillation period for the electron dynamics and the time step. The stability criterion of the Verlet algorithm requires $x > 3$. The default uses this expression with $x = 10$. For proper mass renormalization (discounting the nuclear mass by the effective mass of the wave function cloud) the values of !STRUCTURE!SPECIES:PS<G2> and !STRUCTURE!SPECIES:PS<G4> need to be specified.
Type: real
Rules: optional
Default: $\frac{1}{2}(\frac{10}{2\pi})^2\frac{\Delta^2}{m_\Psi}$

FRIC

constant friction c_α for the wave function dynamics
Type: real
Rules: optional, not used if "!CONTROL!PSIDYN!AUTO" is selected.
Default: 0.0

SAFEORTHO

chooses the way the orthogonality constraints for the wave functions is enforced. If SAFEORTHO=T is selected the traditional way is used, which results in a strictly energy-conserving dynamics. If SAFEORTHO=F a new method is used which converges at eigenstates of the Hamiltonian, but is energy conserving only if the states are sufficiently close to eigenstates. This option allows to calculate excited states or to treat metals.
The option "SAFEORTHO=F" can artificially stabilize the calculation with states is in the "wrong" order, i.e. with occupations that are not monotonically decreasing. Since the wrong order corresponds to a saddle point of the total energy, the wave functions accelerate to reach the correct order. This behavior can also occur, if the two states that are mis-ordered are both occupied or both unoccupied. In that case the energy increases temporarily as the wavefunctions acquire kinetic energy, while the DFT total energy remains approximately constant. The "zentrifugal" forces of the wave function dynamics also perturb the DFT total energy, so that also the latter increases temporarily. Rarely, this behavior can also lead to a instability, namely when the energy-levels shift in such a way that the final state is again in the incorrect order.
Type: logical
Rules: optional
Default: T

STRAIGHTEN

Transform the wave functions onto energy eigenstates. The transform is performed during the first big writeup of the protocoll file. A unitary transform among the wave function is performed. That is, the Hilbert space remains unchanged. Note, that the occupations may be inconsistent with the new states. It may be adviseable to simultaneously restart the occupations.
This option can be used to accelerate convergence with !CONTROL!PSIDYN:SAFEORTHO=F, because it shortcuts the lengthy dynamics towared energy eigenstates. It may also be useful in the early stages of the wave function optimization.

Type: logical
Rules: optional
Default: F

SWAPSTATES

Experimental option! You are on your own! Only used with SAFEORTHO=F. For SWAPSTATES=F, the approximate eigenstates are ordered in increasing energy expectation values. This is used for optimizing electronic and atomic structures. Thus the given occupations refer to increasing one-particle energies in the final state.
If, in a photochemical reaction with a state crossing with fixed occupations, the system shall remain on the upper branch of the energy surface, choose SWAPSTATES=F. If the system shall follow the crossing to the lower branch of the energy surface, choose SWAPSTATES=T.
In combination with the Mermin functional the following happens at a band crossing at the Fermi level.

- SWAPSTATES=F: the wave functions change their character, which results in wave function heating. The time scale of the conversion of the state depends on matrix elements and on the fictitious mass of the wave functions. This process is therefore rather uncontrolled. The occupations remain approximately constant. The crossing is treated like an avoided crossing.

- SWAPSTATES=T: the wave function maintains its character, but the occupations dynamically change from occupied to unoccupied and vice versa. If the croosing is an avoided crossing, the wave functions are reordered only if the if the conversion of the character of the wave functions is sufficiently retarded, that a band crossing of the one-particle energies takes place.

Type:	logical
Rules:	optional, not tested
Default:	F

6.2.11 !CONTROL!PSIDYN!AUTO

Rules: optional

Description: Automatic annealing procedure for wave function dynamics. (overwrites setting of !CONTROL!PSIDYN:FRIC.) For optimizing the electronic structure the default set has been proven very useful. The friction switches between a lower and an upper value depending on whether the energy decreases or increases. The upper and lower friction values are each scaled in each time step. With this option, the program will stop if the total energy changes during 20 iterations by less than 10^{-5} Hartree. This does not necessarily imply convergence of the annealing step. It can happen, in particular near convergence, that the friction jumps from step to step between the lower and the upper value. In this case the system is not converged, even if the total energy is virtually constant, and a constant friction should be used.

FRIC(-)

start value for the friction factor c_α used for decreasing energy

Type: real
Rules: optional
Default: 0.3

FACT(-)

factor multiplied with the friction factor FRIC(-) in each step that reduces the energy

Type: real
Rules: optional
Default: 0.97

FRIC(+)

start value for the friction factor c_α used for increasing energy

Type: real
Rules: optional
Default: 0.3

FACT(+)

factor multiplied with the friction factor FRIC(+) in each step that reduces the energy
Type: real
Rules: optional
Default: 1.0

MINFRIC

minimum friction used
Type: real
Rules: optional
Default: 0.

6.2.12 !CONTROL!PSIDYN!THERMOSTAT

Rules: optional
Description: Thermostat for the wave functions.[35] This thermostat shuffles energy from the wave functions to the atomic motion, when the wave-function kinetic energy former exceeds an estimated target value $\frac{1}{2}\sum_i K_i \dot{\vec{R}}_i^2$. A friction on the thermostat shall be used to avoid instabilities. The thermostat shall be used only if the atoms are moving.

$$
\begin{aligned}
M_i \ddot{\vec{R}}_i &= \vec{F}_i + K_i \dot{\vec{R}}_i \dot{x}_\psi \\
Q_\psi \ddot{x}_\psi &= 2\theta(\dot{x}_\psi) \left(\sum_n f_n \langle \dot{\tilde{\psi}}_n | m_\psi | \dot{\tilde{\psi}}_n \rangle - \frac{1}{2} \sum_i \vec{K}_i \dot{R}_i^2 \right)
\end{aligned}
$$

The values for K_i are derived from the atomic pseudo wave functions.

PERIOD

Oscillation period of the Nosé variable in a.u.

Type: real
Rules: optional
Default: see !CONTROL!PSIDYN!THERMOSTAT:FREQ[THz]

FREQ[THZ]

Frequency of the Nosé variable in THz.
Type: real
Rules: optional; not used if !CONTROL!PSIDYN!THERMOSTAT:PERIOD present
Default: 100.

FRIC

Friction acting on the Nosé variable. Can have values between zero and one. The largest value before overdamping is $\frac{4\pi}{period}$, where period is that of the thermostat variable defined in this section.
Type: real
Rules: optional
Default: 0.

STOP

set velocity for Nosé variable to zero in the first iteration
Type: logical
Rules: optional
Default: F

6.2.13 !CONTROL!RDYN

Rules: optional; default is fixed atomic positions
Description: parameters used to control the dynamics of the atomic positions. The atomic dynamics is governed by the equation

$$M_R \ddot{R} = -\frac{\partial E}{\partial R} - M_R \dot{R} \alpha,$$

where α can be tuned by a Nosé thermostat or the parameters below. The friction parameter α is converted into a parameter $c_\alpha = \alpha\Delta/2$, which can range from 0 to 1. A value of $c_\alpha = 0$ indicates frictionless dynamics, whereas a value of $c_\alpha = 1$ indicates the steepest descent. Δ is the time step.
Remark: Do not use this option before the wave functions are optimized, because unreasonable forces will result from an incorrect electron distribution.

STOP

start with zero velocity for the atomic positions
Type: logical
Rules: optional
Default: F

START

Take initial structure from structure control file instead of the restart file.
Type: logical
Rules: optional
Default: F

RANDOM[K]

adds random velocities at the start; the velocity distribution corresponds to a temperature in Kelvin. Can be used to bring the system quickly to a finite temperature. Note, however, that the electron wave functions cannot immediately follow this sudden kick. They will start to lag behind, and deviate from the Born Oppenheimer surface. A small friction acting on the wave functions will allow the wave functions to recover.

Type: real
Rules: optional
Default: 0

FRIC

constant friction c_α
Type: real
Rules: optional, not used if "AUTO" is selected.
Default: 0.0

OPTIMIZER

How the atoms are moved.

- 'MD': damped Newtonian dynamics as described above.
- 'LBFGS': geometry optimization with the limited-memory BFGS method. The atoms stay where they are until the SCF has converged at the current geometry (the stop signal of **MIXTOL** / **EDIFF** of **!PSIDYN**, which then no longer ends the run); then one L-BFGS step is taken and the SCF histories are restarted. The run stops when the largest force is below **FMAX[EV/AA]** (and, with a moving cell, the stress below **!CELL:SMAX[GPA]**). A moving cell (**!CELL MOVE=T**) is optimized together with the atoms. Safeguards: a step that raises the energy (enthalpy) is halved back towards the last accepted point; pairs without positive curvature and uphill directions reset the history; no atom moves more than a trust radius, which starts at **MAXSTEP[AA]**, is halved at every backtrack and recovers after successful steps. As the forces become small, **EDIFF** and **MIXTOL** of **!PSIDYN** are tightened automatically for each geometry to 1% of the energy gain expected from the step (but not below 10^{-9} H), because the energy test needs an SCF energy more accurate than that gain. For molecules and slabs in vacuum use **MIXER='PULAY'**: the variational mixer leaves an error of about 10^{-5} H there that stalls the optimization near 0.05 eV/Å. Restart files do not contain the L-BFGS history. Requires **!PSIDYN OPTIMIZER='DAV'**, 'RMM' or 'CG'. Cannot be combined with **!AUTO**, **!THERMOSTAT**

Type: character
Rules: optional
Default: 'MD'

FMAX[EV/AA]

L-BFGS only: convergence criterion on the largest force on an atom.
Type: real
Rules: optional
Default: 0.01

MAXSTEP[AA]

L-BFGS only: largest displacement of an atom in one step.
Type: real
Rules: optional
Default: 0.2

HESSIAN[EV/AA2]**

L-BFGS only: initial guess for the Hessian; its inverse sets the length of the first step.
Type: real
Rules: optional
Default: 70.

NMEM

L-BFGS only: number of steps kept in the history.
Type: integer
Rules: optional
Default: 10

NSCFMAX

L-BFGS only: if the SCF has not converged after this number of iterations at one geometry, the step is taken anyway (with a warning in the protocol).
Type: integer
Rules: optional
Default: 60

NONEGEFRIC

Experimental option! Avoids negative friction on the atoms. It is recommended for structure optimizations. It must not be used in connection with thermostats. The friction can become negative via the compensation for the drag by the wave function cloud. A friction on the wave functions implicitely causes an effective friction on the atoms. This effective friction is compensated by adding a negative friction on the atoms. In this case the total friction on some atoms can become negative so that the atoms cannot come to rest. [Future developents shall automatically exclude simultaneous use together with thermostats.]
Type: logical
Rules: optional
Default: false

USEOPTFRIC

Experimental option! Estimates the optimum friction for optimization of the atomic structure. as

$$a_{opt} = \Delta \sqrt{\frac{\dot{\vec{R}}\dot{\vec{F}}}{\dot{\vec{R}}\mathbf{M}\dot{\vec{R}}}}$$

In addition a floating average of this friction value is used. The formula is derived from a projection os the Lagrangian onto a one-dimensional motion and the assumption of a harmonic oscillator. The force constant can then be estimated from the change of the forces between two time steps. See Sec. 23.2 for further information.
Type: logical

Rules: optional
Default: false

6.2.14 !CONTROL!RDYN!AUTO

Rules: optional
Description: Automatic annealing procedure for atomic motion (overwrites setting of !CONTROL!RDYN:FRIC). See the description for !CONTROL!PSIDYN!AUTO. Note that frequent switching between the lower and upper friction indicates a problem that can even result in a heating up of the electrons. In this case, switch to constant friction.

FRIC(-)

start value for the friction factor c_α used for decreasing energy
Type: real
Rules: optional
Default: 0.0

FACT(-)

factor multiplied with the friction factor FRIC(-) in each step that reduces the energy
Type: real
Rules: optional
Default: 1.0

FRIC(+)

start value for the friction factor c_α used for increasing energy
Type: real
Rules: optional
Default: 0.01

FACT(+)

factor multiplied with the friction factor FRIC(+) in each step that reduces the energy
Type: real
Rules: optional
Default: 1.0

MINFRIC

minimum friction used
Type: real
Rules: optional
Default: 0.

6.2.15 !CONTROL!RDYN!WARMUP

Rules: optional, not tested
Description: Heating procedure for QM nuclei. Applies a series of sinusoidal heat pulses in orthogonal directions to provide optimal excitation of the atomic system without dislodging the wavefunctions from the BO surface. This is an approximate procedure.

TARGET_TEMP

Target temperature that should be achieved in Kelvin. Due to approximate nature of the algorithm, it will not be reached exactly in most cases.
Type: real
Rules: mandatory
Default: none

NPULSES

The required temperature increase specified in TARGET_TEMP will be applied in NPULSES steps.
Type: integer
Rules: mandatory
Default: none

PULSE_LENGTH

Each of the NPULSES heat pulses will stretch over PULSE_LENGTH time steps.
Type: integer
Rules: mandatory
Default: none

6.2.16 !CONTROL!RDYN!THERMOSTAT

Rules: optional
Description: Nosé thermostat [36, 37, 38] for creating a constant temperature ensemble for the ions. When using this thermostat, the friction acting indirectly on the atoms, and that results from a friction on the wavefunctions, is corrected for by an opposing force $C_i \dot{R}_i f_\Psi$ acting on the atoms. f_Ψ is the friction acting on the wave functions, which is either fixed or dynamically tuned by a wave function thermostat
This thermostat has the added option of a friction for faster equilibration. The friction must be zero to obtain a canonical ensemble.

T[K]

temperature of the ions in Kelvin
Type: real
Rules: optional
Default: 293.15 (=room temperature=20°C)

<**EKIN**>

average kinetic energy of the atoms
Type: real
Rules: optional
Default: see T[K]

PERIOD

Oscillation period of the Nosé variable in a.u.
Type: real
Rules: optional
Default: see !CONTROL!RDYN!THERMOSTAT:FREQ[THz]

FREQ[THZ]

frequency of the Nosé variable in THz.
Type: real
Rules: optional; not used if !CONTROL!RDYN!THERMOSTAT:PERIOD is present
Default: 10.

FRIC

friction on the Nosé variable. Sensible values lie between 0 and 1; FRIC=0 will give the original Nosé thermostat.
Without friction, the thermostat tends to induce large oscillations when the temperature is changed. This oscillations may break your system in the high-temperature peaks, and they decrease only slowly to their physical amplitude. In this case it is recommended to use a friction of about 0.01 until the system is about stationary at the new temperature. Once the stationary state has been reached, the friction must be removed, so that the system can approach the canonical ensemple
Type: real
Rules: optional
Default: 0

STOP

velocity for Nosé variable is set to zero in the first iteration
Type: logical
Rules: optional
Default: F

6.2.17 !CONTROL!MERMIN

Rules: optional

Description: Describes the treatment of variable occupations of the one-electron energy levels. It allows to describe the occupations as dynamical variables in the framework of the Mermin functional[39], which includes finite temperature effects. The total energy in this case is the free energy and includes an entropic term $-TS$ for the partially occupied orbitals at finite temperatures. The occupations will converge to the Fermi distribution function: $f_i = (1 + e^{-(\epsilon_i - \mu)/kT})^{-1}$. The eigenvalues are the $\epsilon_n = \langle \Psi_n | H | \Psi_n \rangle$.
In the adiabatic mode of operation also the improved tetrahedron method[2] [42] can be used, which is the recommended option for static ground state calculations of metallic systems.
Use this option (variable occupations) only with eigenstates of the Hamitlonian, i.e. with !CONTROL!PSIDYN:SAFEORTHO=.false. Otherwise the results are unpredictable.
It is important to initialize the occupations once using START=.TRUE.. (If occupations are close to zero or one, they will take a very long time to move away from these values.) if Start=.false., total charge and total spin is obtained from the restart file and not from the structure input file
Remark the dynocc object has been largely rewrittem 23-28.12.06. Please report any suspicious observations.

T[K]

Temperature of the electrons in Kelvin (enters the Fermi distribution function)
Type: real
Rules: optional
Default: 1000

M

mass for the occupation dynamics. Should be larger than 200.
Type: real
Rules: optional

Default: 300

ADIABATIC

chooses quasi-adiabatic mode for occupations. New energy levels are introduced that approach the energy levels $\epsilon = \frac{\partial E}{\partial f_n}$.in a retarded fashion. The occupations are determined for these retarded energies. Note that in the previous formula the energy also includes the fictitious kinetic energy. This mode of operation does not produce an energy conserving dynamics. It is recommended for static calculations. The current implementation of the tetrahedron method for Brillouin zone integration only works with adiabatic=T.

Type: logical
Rules: optional
Default: .false.

RETARD

The energy levels used for the Brillouin-zone integration in the adiabatic mode approach the actual enegy levels as $\sim \exp(-\frac{1}{r}\frac{t}{\Delta})$.

Type: real
Rules: optional, used only if !ADIABATIC=T
Default: 0.

TETRA+

if true, the Brillouin zone integration is performed with the improved version of the tetrahedron method[42]. If false, the Brillouin zone integration is done by sampling and the Mermin functional. The tetrahedron method is the recommended method for metallic systems. However it is not fully variational and restricted to the quasi-adiabatic mode. The tetrahedron method usually causes problems when only 1 k-point is used, because this results in delta-function peaks of the density of states. In that case it is better to use TETRA=false and a finite temperature.

Type: logical
Rules: optional, requires ADIABATIC=T
Default: .false.

STOP

set velocity for occupation dynamics to zero in the first iteration

Type: logical
Rules: optional
Default: F

STARTTYPE

defines how the initial occupations are determined.

'X': Occupations are read from restart file.

'E': energies are read from restart file. Occupations are constructed using the fermi distribution function.

'N': States are filled with $T = 0$ assuming absolutely flat bands ordered according to increasing energy.

Type: character
Rules: optional; 'X' incompatible with ADIABATIC=T
Default: 'N'

START

Restart occupations, total spin and total charge from Lagrange multipliers for wave function orthogonalization, which approximate the one-particle energies. If start=.false., the information is read from the restart file.

Type: logical
Rules: optional, not used if !CONTROL!MERMIN:STARTTYPE is set.
Default: F

MOVE

Propagate occupations. Otherwise the occupations are kept frozen.

Type: logical
Rules: optional
Default: T

FRIC

friction for the occupation dynamics
Type: real
Rules: optional
Default: 0.0

FIXQ

conserve total charge
Type: logical
Rules: optional
Default: .true.

FIXS

conserve total spin
Type: logical
Rules: optional
Default: .false.

EFERMI[EV]

chemical potential of the elctrons in eV; only used if fixq=.false.
Type: real
Rules: optional
Default: 0.0

MAGFIELD[EV]

external magnetic field in eV; only used if fixs=.false.
Type: real
Rules: optional
Default: 0.0

6.2.18 !CONTROL!MERMIN!DIAL

Rules: optional
Description: allows to linearly vary the thermodynamic variables of the Mermin functional such as temperature, (not implemented: total charge,total spin, chemical potential, magnetic field). The initial Value is taken from the actual setting of the object

NAME

Specifies the variable to be modified. Can be TEMPERATURE[K]; CHARGE[E]; SPIN[HBAR].
Type: character
Rules: mandatory
Default: none

RATE

Specifies the rate of change of the variabble. The unit of the variable is that indicated by the name. The time scale unit is a.u.
Type: real
Rules: mandatory
Default: none

FINAL

Specifies the final value of the modified quantity. After this value is reached, the quantity remains constant.
Type: real
Rules: mandatory
Default: none

6.2.19 !CONTROL!CELL

Rules: optional; default is fixed unit cell, and no stress calculation
Description: This option is experimental!. Invokes pressure calculation, and Parrinello-Rahman dynamics of the unit cell, if requested.

MOVE

Dynamical evolution of the unit cell.
Type: logical
Rules: optional
Default: T

STOP

reset velocities to zero
Type: logical
Rules: optional
Default: F

FRIC

friction parameter
Type: real
Rules: optional
Default: 0.0

M

mass for unit cell dynamics
Type: real
Rules: optional
Default: mass corresponding to a period of 50*dt for diamond

P

external pressure in Hartree/bohr^3. Alternatively
P[GPA]
in GPa.
Type: real
Rules: optional
Default: 0.0

STRESS

external stress tensor (row by row, symmetric) in Hartree/bohr^3, alternatively
STRESS[GPA]
in GPa. Same sign convention as
P
and as the printed stress $\sigma = -\frac{1}{V}\frac{\partial E}{\partial \epsilon}$ (positive is compressive): the cell relaxes until the printed stress equals
P
$\cdot\mathbf{1}+$
STRESS
. The load acts on the actual cell (true, Cauchy stress), so the result does not depend on the starting cell. Works with Parrinello–Rahman dynamics and with L-BFGS.
Type: real(9)
Rules: optional
Default: 9×0.0

OPTIMIZER
'MD' (Parrinello–Rahman dynamics) or 'LBFGS'. Not needed for a joint optimization: with
!RDYN OPTIMIZER='LBFGS'
a moving cell is optimized with the atoms.
OPTIMIZER='LBFGS'
without
!RDYN
optimizes the cell at fixed fractional coordinates.
Type: character
Rules: optional
Default: follows !RDYN:OPTIMIZER

SMAX[GPA]
L-BFGS only: convergence criterion on the largest element of the difference between the printed stress and the target
P
$\cdot\mathbf{1}+$
STRESS

Type: real

Rules: optional
Default: 0.1

B0[GPA]

L-BFGS only: bulk modulus that sets the length of the first cell step (exact for a hydrostatic strain of a material with this bulk modulus).
Type: real
Rules: optional
Default: 100.

CONSTRAINTTYPE

Constraints on the cell dynamics. Allowed values are the following:

- 'ISOTROPIC' only allows isotropic expansions and contractions.
- 'NOSHEAR' allows anisotropic expansions and contractions but no shearing.
- 'FREE' does not restrict the dynamics other than to remove the rotation from the stress tensor.
- 'NOSTRESS' sets the stresses to zero. (Only used for tests).

Type: character
Rules: optional
Default: 'FREE'

6.2.20 !CONTROL!QM-MM

Rules: optional
Description: controls parameters for the dynamics of the molecular mechanics environment specified in the structure file.
Remark: Switch this option on only after the wave functions are optimized, because unreasonable electrostatic forces will act on the environment atoms if the electron density is incorrect

STOP

sets initial velocities of the environment to zero.
Type: logical
Rules: optional
Default: false

FREEZE

keeps environment atoms except link atoms frozen
Type: logical
Rules: optional
Default: false

ADIABATIC

Relaxes the environment in every time step to zero (in contrast to damped or undamped dynamics)
Type: logical
Rules: optional
Default: true

FRIC

Friction (FRIC=1 corresponds to steepest decent; FRIC=0 to zero friction)
Type: real
Rules: optional
Default: 0.0

RANDOM[K]

randomizes the atomic positions (not yet implemented)
Type: real
Rules: optional
Default: 0.0

MULTIPLE

makes environment multiple times faster (multiple time steps). For very large values the QM reaction center experiences free energy forces from the environment.
Type: integer
Rules: optional
Default: 1

6.2.21 !CONTROL!QM-MM!AUTO

Rules: optional
Description: Automatic annealing procedure for the dynamics of the MM-environment. See also remarks for !CONTROL!PSIDYN!AUTO

FRIC(+)

start value for the friction factor c_α used for increasing energy
Type: real
Rules: optional
Default: 0.3

FACT(+)

factor multiplied with the friction factor FRIC(+) in each step that reduces the energy
Type: real
Rules: optional
Default: 1.0

FRIC(-)

start value for the friction factor c_α used for decreasing energy
Type: real
Rules: optional
Default: 0.3

FACT(-)

factor multiplied with the friction factor FRIC(-) in each step that reduces the energy
Type: real
Rules: optional
Default: 0.97

MINFRIC

minimum friction used
Type: real
Rules: optional
Default: 0.

6.2.22 !CONTROL!QM-MM!THERMOSTAT

Rules: optional
Description: Nosé thermostat [36, 37, 38] to for constant temperature ensemble for the QM-MM environment atoms. It has the added option of a friction for faster equilibration and faster acceleration from small velocities. To regain the original thermostat leave the default FRIC=0.

T[K]

temperature of the ions in Kelvin
Type: real
Rules: optional

Default: 293.15 (=room temperature=20°C)

<EKIN>

average kinetic energy
Type: real
Rules: optional
Default: see T[K]

FREQ[THZ]

frequency of the Nosé variable in THz. This is an alternative to !CONTROL!RNOSE:PERIOD
Type: real
Rules: optional; is overwritten by PERIOD
Default: see PERIOD

FRIC

friction on the Nosé variable. Sensible values lie between 0 and 1; FRIC=0 will give the original Nosé thermostat, while FRIC=1 is used for equilibration and does not produce a canonical ensemble.
Type: real
Rules: optional
Default: 0

STOP

velocity for Nosé variable is set to zero in the first iteration
Type: logical
Rules: optional
Default: F

6.2.23 !CONTROL!COSMO

Rules: optional
Description: controls parameters for the dynamics of the continuum description of solvent. Essentially identical to the COSMO method by Klamt and Schuurmann, with modifications that allow consistent forces[43].

STOP

sets initial velocities of the environment to zero
Type: logical
Rules: optional
Default: false

START

restarts screening charges with value zero
Type: logical
Rules: optional
Default: false

M

Mass for the surface charge dynamics
Type: real
Rules: optional
Default: 1000.

MULTIPLE

Number of time steps of the surface charge dynamics per PAW timestep
Type: integer
Rules: optional
Default: 1

ADIABATIC

chooses optimizations of screening charges in each time step

Type:	logical
Rules:	optional
Default:	false

ETOL

energy convergence criterion for ADIABATIC=T. Inner loop terminates, if estimated energy deviation from minimum is smaller than etol.

Type:	logical
Rules:	if ADIABATIC=T either ETOL or QTOL or both must be specified. Not used for ADIABATIC=F.
Default:	10^{-7} if qtol is not specified

QTOL

charge convergence criterion for ADIABATIC=T. Inner loop terminates, if estimated charge deviation from minimum , i.e. $|\vec{q} - \vec{q}_{min}|$ is smaller than etol.

Type:	logical
Rules:	if ADIABATIC=T either ETOL or QTOL or both must be specified. Not used for ADIABATIC=F.
Default:	charge criterion is not used

FRIC

friction for the dynamics of the screening charges

Type:	real
Rules:	optional
Default:	0.

OPTFRIC

If true, the optimum friction scheme used for the relaxation of the screening charges. See Sec. 23.2 for further information. Is only used in combination with option ADIABATIC.

Type: logical
Rules: optional, only used in combination with ADIABATIC=T
Default: .false.

RETARD

Only used in combination with OPTFRIC=T. The running average of the optimum friction scheme adjusts within RETARD time steps to one-half of the original deviation.

Type: real
Rules: optional, only used in combination with OPTFRIC=T
Default: 0.d0

6.2.24 !CONTROL!ANALYSE

Rules: optional
Description: Prepares special data for analysis

OPTIC

This option is currently disconnected. Writes data required for the optic package of M. Alouani. Attention! Many files are written starting with extension ".optics...". Does not work on parallel computers, and not for non-collinear spin density.

Type: logical
Rules: optional, currently disconnected
Default: false

6.2.25 !CONTROL!ANALYSE!TRA

Rules: optional
Description: select trajectories to be written.

R

positions and point charges. (extension "_r.tra")
Type: logical
Rules: optional
Default: .true.

FORCE

forces acting on the atoms. (extension "_f.tra")
Type: logical
Rules: optional
Default: .false.

E

total energy contributions. (extension "_e.tra")
Type: logical
Rules: optional
Default: .false.

BANDS

one-particle energies.(not yet implemented, extension "_b.tra")
Type: logical
Rules: optional
Default: .false.

QMMM

atomic trajectory including the environment studied with QM-MM coupling.(not yet implemented, extension "_rqmmm.tra")

Type: logical
Rules: optional
Default: .false.

NSKIP

number of time slices to be skipped between two written ones.
Type: integer
Rules: optional
Default: 0

6.2.26 !CONTROL!ANALYSE!HYPERFINE

Rules: optional,multiple
Description: calculates hyperfine parameters for a selected atom. Note that hyperfine parameters are extremely sensitive quantities, and therefore plane wave convergence should be checked carefully. In some cases it is also necessary to use more partial waves in the augmentation than normally used for total energy calculations.
The magnetic hyperfine field B_i^N describes the magnetic field produced at the nuclear site by the electron spin distribution. The total hyperfine field depends on the direction of the electron spin, which is imposed by the external magnetic field. Hence the anisotropic hyperfine field is provided as a matrix that needs to be multiplied by a normalized vector pointing in the direction of the external magnetic field to give the contribution of the anisotropic part to the total hyperfine field. The total hyperfine field is obtained by adding the Fermi-contact field and the anisotropic hyperfine field. The hyperfine splitting ΔE is obtained by multiplication of the total hyperfine field B_i^N with $2\hbar\frac{m^N}{S_N}$, where m^N is the nuclear magnetic moment and S^N is the nuclear spin.

ATOM

atom name consistent with structure input file
Type: character

Rules: mandatory
Default: none

EFG

select electric field gradient calculation. Result will be documented in the protocol. The electric field gradient is related to the second derivatives of the potential at the nuclear site
Type: logical
Rules: optional
Default: .false.

ISOMERSHIFT

select isomer shift calculation. Result will be documented in the protocol. The isomer shift is related to the charge density at the nuclear site.
Type: logical
Rules: optional
Default: .false.

FERMICONTACT

calculates the Fermi contact field. It is the magnetic field acting on the nucleusResult will be documented in the protocol. The Fermi contact term is related to the spin density at the nuclear site.
Type: logical
Rules: optional
Default: .false.

ANISOTROPIC

selects anisotropic hyperfine parameter calculation. Result will be documented in the protocol. The anisotropic hyperfine parameter is related to the $\ell = 2$ component of the spin density near the nucleus
Type: logical
Rules: optional
¡ Default: .false.

6.2.27 !CONTROL!ANALYSE!CORELEVELS

Rules: optional, not tested

Description: calculates the positions of the core levels (absolute and relative to the isolated atom (atomic calculation)). Note that core relaxation effects, which contribute typically to 15 %, are not included. The absolute position of the core levels is only meaningfull for isolated molecules. For extended systems, the zero of the electrostatic potential is not the vacuum level, so that only relative shifts within a single calculation are meaningful. For Tests see Pasquarello et al., Phys. Rev. B 53, 10942 (1996).
Note does not work yet for parallel execution. Minor changes are required

DEFAULT

Selects if core-levels shall be calculated for atoms, which are not in the list of selected atoms.(see below).

Type: logical
Rules: optional
Default: .false.

ATOMS

List of atoms, for which the core levels shall be calculated or avoided. If default=F the core levels of specified are calculated. If default=T, the core levels of all atoms not specified are calculated.

Type: character,array of arbitrary length
Rules: optional
Default: none

6.2.28 !CONTROL!ANALYSE!WAVE

Rules: optional,multiple
Description: Writes a wave function. The file created can then be processed by the paw_wave tool to produce an input file for OPENDX[13]

TITLE

title of the image. Currently it is not used other than in the printout.
Type: character
Rules: optional
Default: none

FILE

full file name of the file to be produced, which can be converted into a input file for OPENDX[13] using the paw_wave tool.
Type: character
Rules: mandatory
Default: none

DR

grid spacing for the output file. rounded to get an integer factor relative to the real-space grid used in the calculation.
Type: real
Rules: optional
Default: 0.4

B

band index
Type: integer
Rules: mandatory
Default: none

K

k-point index
Type: integer
Rules: mandatory
Default: 1

S

spin index
Type: integer
Rules: mandatory
Default: 1

IMAG

use imaginary part
Type: logical
Rules: optional
Default: F

6.2.29 !CONTROL!ANALYSE!DENSITY

Rules: optional,multiple
Description: Writes a density. The file created can then be processed by the paw_wave tool to produce an input file for OPENDX[13].

TITLE

title of the image. Currently it is not used other than in the printout.
Type: character
Rules: optional
Default: none

FILE

full file name of the file to be produced, which can be converted into a input file for OPENDX[13] using the paw_wave tool.
Type: character
Rules: mandatory
Default: none

DR

grid spacing for the output file. rounded to get an integer factor relative to the real-space grid used in the calculation.
Type: real
Rules: optional
Default: 0.4

TYPE

can be 'TOTAL', 'SPIN', 'UP' or 'DOWN'. Determines the weights of the states in the density plots. 'TOTAL' takes the actual occupations and k-point or uniform weights (depending on 'OCC'). 'SPIN' is like 'TOTAL', but counts states for spin=2 negative. 'UP' and 'DOWN' give the spin up and down densities respectively.
Type: character
Rules: optional
Default: 'TOTAL'

OCC

use acutal occupations
Type: logical
Rules: optional
Default: T

DIAG

use eigenstates in the subspace of the dynamic wave functions.
Type: logical
Rules: optional

Default: T

CORE

include core density.
Type: logical
Rules: optional
Default: F

EMIN[EV]

lowest eigenenergy to be included.
Type: real
Rules: optional
Default: -1^{-10}

EMAX[EV]

highest eigenenergy to be included.
Type: real
Rules: optional
Default: 1^{-10}

BMIN

lowest band to be included.
Type: integer
Rules: optional
Default: 1

BMAX

highest band to be included.
Type: integer
Rules: optional
Default: 10000000

6.2.30 !CONTROL!ANALYSE!POTENTIAL

Rules: optional
Description: Writes the hartree potential. The file created can then be processed by the paw_wave tool to produce an input file for OPENDX[13].

TITLE

title of the image. Currently it is not used other than in the printout.
Type: character
Rules: optional
Default: none

FILE

full file name of the file to be produced, which can be converted into a input file for OPENDX[13] using the paw_wave tool.
Type: character
Rules: mandatory
Default: none

DR

grid spacing for the output file. It is rounded to get an integer factor relative to the real-space grid used in the calculation.
Type: real
Rules: optional
Default: 0.4

6.2.31 !CONTROL!ANALYSE!EELS

Rules: optional,multiple
Description: Construct EELS (Electron-Energy Loss Spectra).

In an EELS experiment, the material is exposed to a highly-energetic electron

beam. Due to inelastic interactions with the probe the transmitted electrons have a lower energy. The resulting energy loss spectrum is this a measure of the inelastic collisions in the probe. One important effect is the excitation of a core electron into an empty orbital of the probe. Because the core electrons are very localized, the loss spectrum measures the density of states of the conduction band projected onto the orbitals on the same atom with angular momentum raised or lowered by $\Delta\ell = \pm 1$.

Following Wooten[44], we calculate the loss function $L(\hbar\omega)$.

$$L(\hbar\omega) = -\Im[\epsilon_r^{-1}(\hbar\omega)] \quad (1)$$

$$\epsilon_r(\hbar\omega) = 1 + \frac{4\pi\hbar^2 q^2 N/V}{4\pi\epsilon_0 m_e} \sum_c \int \frac{d^3k}{V_g} \frac{\left(1 - f_n(\vec{k})\right) Q_{n,0}(\vec{k})}{(\epsilon_n - \epsilon_c)^2 - (\hbar\omega)^2 - i\Gamma\hbar\omega} \quad (2)$$

$$Q_{n,c}(\vec{k}) = \frac{2m_e}{\hbar^2} \sum_c (\epsilon_n - \epsilon_c) \cdot \frac{1}{3} \sum_{j=1}^{3} |\langle\psi_c|x_j|\psi_n\rangle|^2 \quad (3)$$

Eq. 2, specifying the dielectric constant $\epsilon(\hbar\omega)$, corresponds to Wooten's Eq. 3.68. Eq. 3, specifying the oscillator stength $Q_{n,0}(\vec{k})$, corresponds to Wooten's Eq. 3.67 and Eq. 1, giving the loss function, corresponds to Wooten's Eq. 5.62. With $\Im$, I denote the imaginary part. N/V is the number density of the specified atom (i.e. 1/unit-cell-volume). $\epsilon_r(\omega)$ is the relative dielectric constant. Here only the core-excitations are considered. $|\psi_c\rangle$ are the core levels and ϵ_c are their energies. $\sum_\alpha |\phi_\alpha\rangle\langle\tilde{p}_\alpha|\tilde{\psi}_n\rangle$ is the n-th band state and ϵ_n is its energy. Γ is the lifetime broadening (as FWHM, full-width half-maximum) given as sum of core-hole lifetime and the life time of the valence state. Core-hole life time is taken from Eqs. 12 and 14 of Egerton[45]. The life time of band states is obtained from Eq. 15 of Egerton [45] with the inelastic mean free path of Seah[46], Eq. 5 and table 1 (entry for inorganic compounds). The loss function is furthermore smeared by a Gaussian with the specified experimental broadenig. The spectrum is calculated from 5 eV below the estimated Fermi level to he minimum of the top-most band. (i.e. the minimum for both spin directions.)

ATOM

Name of the atom Atom for which the EELS spectra are determined.

Type:	character
Rules:	mandatory
Default:	none

IC

Core level shell. Currently formulated in without spin orbit splitting. (Caution when used with spin-orbit splitting. IC refers the shell index in the atomic calculation.)

IC	shell	X-ray notation
1	$1s_{(1/2)}$	K_1
2	$2s_{(3/2)}$	L_1
3	$2p_{(1/2)}$	L_2
3	$2p_{(3/2)}$	L_3
4	$3s_{(3/2)}$	M_1
5	$3p_{(1/2)}$	M_2
5	$3p_{(3/2)}$	M_3
6	$3d_{(3/2)}$	M_4
6	$3d_{(5/2)}$	M_5

Type: integer
Rules: optional
Default: 1

BROADENING[EV]

Instrumental broeadening FWHM (Full width at half maximum) in eV.
Type: real
Rules: optional. Must not be set to zero.
Default: 1.0

FILE

File name for the spectrum. The data are written in column form to the specified file as energy in electron volt (1), the oscillator-strength-weighted density of states (2), the loss function (3), and the loss function with the specified experimental broadenig (4).
Type: character
Rules: optional

Default: *ATOM*_IC*IC*.eels, where the text in italics is replaced by the corresponding values as specified above.

6.3 The structure input file "STRC"

The structure input file defines the molecule or crystal to be studied.

6.3.1 Example for the structure input file "STRC"

This is an example of a structure input file for formaldehyde in a fcc supercell with lattice constant of 10 Å.

```
!STRUCTURE
  !GENERIC LUNIT=1.8897259926 !END
  !OCCUPATIONS NBAND=8 NSPIN=1 !END
  !LATTICE T= 0.00000  5.00000  5.00000
              5.00000  0.00000  5.00000
              5.00000  5.00000  0.00000 !END
  !SPECIES NAME= 'H_' ZV=1. M=1.9380
           FILE='/u/blo.2/PAW/Setups/BP86/001H/h1.out'  !END
  !SPECIES NAME= 'C_' ZV=4. M=18.9984
           FILE='/u/blo.2/PAW/Setups/BP86/006C/c1.out'  !END
  !SPECIES NAME= 'O_' ZV=6. M=15.99946
           FILE='/u/blo.2/PAW/Setups/BP86/008O/o1.out'  !END
  !ATOM    NAME= 'H_1'  R= -0.50000  0.00000 -0.86600 !END
  !ATOM    NAME= 'H_2'  R= -0.50000  0.00000  0.86600 !END
  !ATOM    NAME= 'C_3'  R=  0.00000  0.00000  0.00000 !END
  !ATOM    NAME= 'O_4'  R=  1.50000  0.00000  0.00000 !END
  !ISOLATE NF=3 RC=0.5 RCFAC=1.5 GMAX2=3.0 DECOUPLE=T !END
!END
!EOB
```

The following is not a working example. It is aimed at illustrating the syntax of the input file. It is more complex than a typical case because it incorporates (almost) all options at the same time.

```
!STRUCTURE
  !GENERIC LUNIT=1.0 !END
```

```
  !LATTICE T=0. 10. 10. 10. 0. 10. 10. 10. 0. !END
  !KPOINTS K= 0 0 0 !END
  !SPECIES  NAME='H_' ZV=1.  M=1.008 PSEKIN=0.0
           FILE=~/PAW/Setups/006H/h1.out !END
  !SPECIES NAME='C_' ZV=4. M=12.011
           FILE=~/PAW/Setups/006C/c1.out !END
  !ATOM NAME='alpha-hydrogen'
                 R= 1.0 1.0 1.0 SP='H_' !END
  !ATOM NAME='H_2' R=-1.0000 -1.0000  1.0000 !END
  !ATOM NAME='H_3' R=-1.0000  1.0000 -1.0000 !END
  !ATOM NAME='H_4' R= 1.0000 -1.0000 -1.0000 !END
  !ATOM NAME='C_1' R=0.0 0.0 0.0 M=5.0 !END
  !OCCUPATIONS
    NBAND=10 NSPIN=2 CHARGE[E]=0. SPIN[HBAR]=1.
    !STATE K=1 S=1 B=5 F=1.000 !END
    !STATE K=1 S=1 B=6 F=1.000 !END
  !END
  !ISOLATE NF=3 RC=0.5 RCFAC=1.5 GMAX2=3.
           DECOUPLE=T !END
  !GROUP NAME='BOND1'  ATOMS='C_1' 'H_1' !END
  !GROUP NAME='BOND2'  ATOMS='C_1' 'H_2' !END
  !CONSTRAINTS
    !RIGID GROUP='BOND1' !END
    !FREEZE GROUP='BOND2' !END
    !FREEZE ATOM='H_3'  !END
    !TRANSLATION GROUP='ALL' !END
    !TRANSLATION GROUP='BOND1' DIR= 1. 0. 0. !END
    !ORIENTATION  GROUP='BOND1' AXIS= 1. 0. 0. !END
    !LINEAR
      !LINE ATOM='H_1' VEC= 0.5 0.0  0.0 !END
      !LINE ATOM='H_2' VEC= 0.5 0.0 0.0 !END
    !END
  !END
!END
!EOB
```

6.3.2 Argument description for the structure input file "STRC"

6.3.3 !STRUCTURE

Rules: mandatory
Description: Defines the system to be studied. Specifies geometry, atom types, electronic occupations, atomic masses. etc...

6.3.4 !STRUCTURE!GENERIC

Rules: optional
Description: data that do not fit into other blocks.

LUNIT

specifies the length unit (in atomic units) for structural input in this file. All data are converted into atomic units (1 a.u.=0.529167$\times 10^{-10}$ m) by multiplication with LUNIT
Type: real
Rules: optional, LUNIT and LUNIT[AA] are mutually exclusive
Default: 1.0

LUNIT[AA]

specifies the length unit in angstrom for structural input in this file. All data are converted into atomic units (1 a.u.=0.529167$\times 10^{-10}$ m) by multiplication with LUNIT=LUNIT[AA]*angstrom
Type: real
Rules: optional, LUNIT and LUNIT[AA] are mutually exclusive
Default: none

STPVERSION

overwrites the required version of the setup parameters file (parameters/stp.cntl). Background: Since version 2.0.0 of the paw code there is an internal check in the release versions of the program that makes sure that the version of the paw code (see --version command line argument) is the same as the version of the setup parameters file that is loaded at runtime (see PAWDIR environment variable or PARMS_STP-option).
Type: Character
Rules: optional
Default: none

6.3.5 !STRUCTURE!LATTICE

Rules: mandatory
Description: lattice translation vectors

T

Type: real(3,3)
Rules: mandatory
Lattice vectors in the order; (x,y,z of first vector; x,y,z of second vector...)
Default: none

6.3.6 !STRUCTURE!KPOINTS

Rules: optional
Description: specifies the k-point grid. Default is Γ-point sampling.

R

defines the density of k-points for the automatic generation of k-points. $\Delta k = 2\pi/R$. The number n_i of k-points along the reciprocal lattice vector $\vec{g}_i$ is obtained from R as the minimum number obeying $n_i \geq \frac{1}{2\pi}|\vec{g}_i|R$.
Type: real

Rules: optional
Default: 12.

DIV

fractions of reciprocal lattice vectors defining the reciprocal sublattice for the k-points.If `K` is not specified, the k-points are equally spaced an set compatible with the inversion symmetry. Caution with non-orthogonal lattice vectors: $\vec{G}_1 = \frac{1}{V}\vec{T}_2 \times \vec{T}_3$, $\vec{G}_2 = \frac{1}{V}\vec{T}_3 \times \vec{T}_1$, $\vec{G}_3 = \frac{1}{V}\vec{T}_1 \times \vec{T}_2$, where $\vec{G}_i$ are the reciprocal space lattice vectors and $\vec{T}_i$ are the real space lattice vectors and $V = \vec{T}_1(\vec{T}_2 \times \vec{T}_3)$ is the volume of the real space unit cell.
Type: integer(3)
Rules: optional; mutually exclusive with `R`
Default: 2 2 2

SHIFT

shift is displaces the K-point grid such that the Γ-point is avoided. The three components of SHIFT specify the shift directions into the three lattice vectors. Each component may have the values 0 or 1. For zero, the grid is not shifted along the specified direction. For a value of 1, the grid is shifted by one-half of the grid spacing along the corresponding reciprocal lattice vector. While the density of the k-point grid is not affected by shifting the k-point grid, the k-point convergence is apparently faster. Shifting the grid avoids the high-symmetry points. Thus, one does not see the band edges, which usually lie on high symmetry points. The computational effort is not much affected by shifting the grid.
Type: integer(3)
Rules: optional
Default: 0 0 0

K

k-point coordinates (i_1, i_2, i_3) in units of fractions of reciprocal lattice vectors: $k_i = 1/2 \sum_j G_{ij} * i_j/n_j$, where G_{ij} is defined by the real space lattice vectors T_{ij} by $\sum_k G_{ki} T_{kj} = 2\pi\delta_{ij}$. The fractions are provided by DIV. The k-point integration weights are equally distributed. This option is not recommended for general use. The safe approach is to set either R or DIV. If K is not specified, the k-points are set equally spaced according to R or div

Type: integer(3)
Rules: multiple, optional
Default: 0 0 0

SYMMETRY

uses the space group of the structure. The operations are found automatically from the lattice, the atomic positions and the species; atoms of one species that carry different orbital-potential seeds (!STRUCTURE!ORBPOT or !ORBPOT_OFF, e.g. the two Ni sublattices of antiferromagnetic NiO) are treated as inequivalent, so the magnetic order is preserved. The k-point mesh (DIV, SHIFT) is reduced to its irreducible wedge, and the pseudo density, the one-center density matrices, the forces and the stress are symmetrized. The density is symmetrized exactly on the real-space grid when every operation maps the grid onto itself, otherwise in reciprocal space. Works with MPI (the density grid is assembled on every task for the symmetrization). Not available with noncollinear spin, onsite hybrids (!NTBO) or delocalized TSCAN.

Type: logical
Rules: optional
Default: F

6.3.7 !STRUCTURE!SPECIES

Rules: Mandatory, multiple
Description: Specifies an atom type (Element).

NAME

Name of the atom type. It is recommended to begin the name with the two-character element symbol, where a whitespace as second character can be replaced by an underscore. This allows the program to select certain default values form the periodic table.

Type:	character
Rules:	mandatory
Default:	none

ID

Identifier for the setup type. Allows to select the parameter set defining the augmentation (e.g. partial waves and projector functions). The ID must be one of the internally recognized strings, or it must identify a parameter set on a parameter file specified by !CNTL!FILES!FILE with NAME=AUGPARMS. The code stops if the ID is ambiguous.

Type:	character
Rules:	optional. not used when !STRUCTURE!SPECIES!AUGMENT is present.
Default:	none

M

Atomic mass in mass units (u=1822.89 a.u.=$\frac{1}{12}m(^{12}\mathrm{C}) \approx \mathrm{m}_p$).

Type:	real
Rules:	Optional, if either !STRUCTURE!SPECIES:Z is specified, or if !STRUCTURE!SPECIES:NAME begins with the element symbol
Default:	Atomic mass, as obtained from !STRUCTURE!SPECIES:Z or, if not present, from !STRUCTURE!SPECIES:NAME

NPRO

Array which determines the maximum number of projector functions per angular momentum – together with its all-electron and pseudo partial waves – used in the calculation. The maximum possible value is determined by the setup file. Each element refers to the main angular momentum ℓ, in increasing order. The length of the array determines the maximum angular momentum for the augmentation.

Type: integer array
Rules: Mandatory
Default: None

LRHOX

One center expansion for the density is limited to a maximum angular momentum of lrhox. If the value of LRHOX is larger $2\ell_{max}$, where ℓ_{max} is the maximum angular momentum in the set of partial waves, LRHOX is reset internally to $2\ell_{max}$.

Type: integer
Rules: Optional
Default: 2

RAD

Atomic radius in a_0 used to evaluate the integration volume for the projected density of states etc.

Type: real
Rules: optional, obsolete: is overwritten by RAD/RCOV.
Default: R(ASA) from periodictable object. Equals 1.10533 times the covalent radius.

RAD/RCOV

Atomic radius in units of the covalent radius used to evaluate the integration volume for the projected density of states etc.

Type: real
Rules: optional

Default: R(ASA) from periodictable object. Equals 1.10533 times the covalent radius.

6.3.8 !STRUCTURE!SPECIES!DFTU

Rules: optional

Description: DFT+U in the rotationally invariant form of Dudarev et al. (PRB 57, 1505 (1998)), $E_U = \frac{U-J}{2} \sum_\sigma \mathrm{Tr}[n^\sigma - n^\sigma n^\sigma]$, with the occupation matrix n of one angular-momentum shell built from the one-center density matrix and the overlaps of the all-electron partial waves inside a sphere (as in VASP, LDAUTYPE=2). Works for non-spin, collinear and noncollinear runs. NiO with $U - J = 6.3$ eV on Ni d: gap 3.12 eV.

L

angular momentum of the correlated shell

Type: integer
Rules: mandatory
Default: –

U[EV]

Hubbard U in eV

Type: real
Rules: optional
Default: 0.

J[EV]

Hund's J in eV; only $U - J$ enters

Type: real
Rules: optional
Default: 0.

RCUT/RCOV

radius of the sphere for the occupation matrix, in units of the covalent radius

Type: real
Rules: optional
Default: 1.

6.3.9 !STRUCTURE!SPECIES!AUGMENT

Rules: optional, mandatory, if !STRUCTURE!SPECIES:ID is not present

Description: Defines the augmentation, (e.g. partial waves and projector functions).

ID

Setup identifier. Will be reported in the protocoll file.

Type: character
Rules: optional
Default: 'NONAME'

EL

Element symbol.

Type: character
Rules: optional, mandatory if keyword Z is not specified, incompatible with keyword Z.
Default: none

Z

Atomic number

Type: real
Rules: optional, mandatory if keyword EL is not specified, incompatible with keyword EL.
Default: none

ZV

Number of valence electrons.
Type: real
Rules: mandatory
Default: none

RCSM/RCOV

Radius for the compensation charge in units of the covalent radius.
Type: real
Rules: optional
Default: 0.25

ZORA

selects the zeroth-order regular approximation for the relativistic treatment of the partial waves. The default is the energy-dependent scalar-relativistic treatment (as in VASP); ZORA contracts the fcc Cu lattice constant by about 0.4 %.
Type: logical
Rules: optional
Default: F

DEREF[H]

reference energies of the partial waves. 0: the CP-PAW scheme, where every partial wave beyond the first of a shell has the same logarithmic derivative at RBOX as the first one (”box states”). > 0: each further partial wave of a shell lies DEREF above the previous one and is obtained by outward integration at that energy. Only the HBS construction uses box states; NDLSS setups place all partial waves of a shell at the valence energy (energy-derivative recursion) and ignore DEREF. The best spacing is element dependent (bcc Mo, HBS_SC, PBE: lattice constant +0.79 % with box states, -0.05 % with DEREF=0.25; fcc Cu, HBS: -0.67 % with box states, -0.87 % with 0.5 and a broken setup with 0.25, where the partial waves become nearly linearly dependent). Validate a setup before changing it.

Type: real
Rules: optional
Default: 0.

TYPE

Augmentation method. Allowed values are ’NDLSS’, ’HBS’ and ’KERKER’.

Type: character
Rules: mandatory
Default: none

RCL/RCOV

Radius parameter array for the construction of pseudo partial waves in units of the covalent radius.

Type: real array
Rules: mandatory
Default: none

LAMBDA

Second parameter array for the construction of pseudo partial waves. Currently used only for TYPE=’HBS’.

Type: real array

Rules: mandatory for TYPE='HBS'. Not used otherwise.
Default: none

RBOX/RCOV

.....to be completed.....
. Type: real

Rules: mandatory
Default: none

6.3.10 !STRUCTURE!SPECIES!AUGMENT!POT

Rules: mandatory in !STRUCTURE!SPECIES!AUGMENT
Description: Defines auxiliary potential. The radial component of the potential $v(|\vec{r}|)Y_s(\vec{r})$ is given by

$$\tilde{v}(r) = \begin{cases} a + br^{\alpha} + cr^{\alpha+2} & \text{for } r < r_c \\ v(r) & \text{for } r > r_c \end{cases} \quad (4)$$

where $v(r)$ is tha radial component of the true atomic potential.

POW

Power α in the Taylor expansion. A higher value leads to a flatter potential at the origin.
Type: real
Rules: mandatory
Default: none

RC/RCOV

Matching radius r_c in units of the covalent radius.
Type: real
Rules: mandatory
Default: none

VAL0

Value of the auxiliary potential at the origin VAL0=aY_s. If not specified, the parameter c is set to zero and the value a is obtained from the differentiability at r_c.
Type: real
Rules: optional
Default: none

LLOC

Experimental. If specified, the auxiliary potential is not obtained from the Taylor expansion above, but, similar to the ICORE choice of VASP, as the norm-conserving (Troullier-Martins type) pseudopotential of angular momentum channel LLOC at energy ELOC with matching radius r_c. POW and VAL0 are then ignored. This potential is deep for channels without core states (e.g. -9.9 H at the origin for the d channel of Si). The HBS and KERKER partial waves are solutions in this potential and become hard; for Si the resulting setup produces ghost states. Do not use in production.
Type: integer
Rules: optional
Default: none

ELOC

Reference energy in Hartree for the channel LLOC. The default is the highest occupied valence level of the atom.
Type: real
Rules: optional, only with LLOC
Default: highest occupied valence level

6.3.11 !STRUCTURE!SPECIES!AUGMENT!CORE

Rules: mandatory in !STRUCTURE!SPECIES!AUGMENT
Description: Defines auxiliary core density. The radial component of the density $n^c(|\vec{r}|)Y_s(\vec{r})$ is given by

$$\tilde{n}^c(r) = \begin{cases} a + br^\alpha + cr^{\alpha+2} & \text{for } r < r_c \\ n^c(r) & \text{for } r > r_c \end{cases} \quad (5)$$

where $n^c(r)$ is the radial component of the true core density.

POW

Power α in the Taylor expansion. A higher value leads to a flatter density at the origin.
Type: real
Rules: mandatory
Default: none

RC/RCOV

Matching radius r_c in units of the covalent radius.
Type: real
Rules: mandatory
Default: none

VAL0

Value of the auxiliary core density at the origin VAL0=aY_s. If not specified, the parameter c is set to zero and the value a is obtained from the differentiability at r_c.
Type: real
Rules: optional
Default: none

6.3.12 !STRUCTURE!SPECIES!AUGMENT!GRID

Rules: mandatory in !STRUCTURE!SPECIES!AUGMENT
Description: Defines the radial grid for the one center expansions. The radial grid is a shifted logarithmic grid

$$r(x) = r_1\Big(\mathrm{e}^{\alpha(x-1)} - 1\Big) \tag{6}$$

with $x = 1, 2, 3, \ldots, N$.

DMIN

Smallest spacing (at the origin).
Type: real
Rules: mandatory
Default: none

DMAX

Largest spacing (at the end of the grid).
Type: real
Rules: mandatory
Default: none

RMAX

Outermost grid point.
Type: real
Rules: mandatory
Default: none

6.3.13 !STRUCTURE!SPECIES!NTBO

Rules: optional, atom cointribution is scaled to zero if not specified.

Description: Defines a set of local orbitals, called natural tight-binding orbitals (NTBOs). The NTBOs exhibit only scattering character in the context of nodeless wave functions[47] except for the head. NTBOs are represented as screened LMTO's. The screened envelope function is constructed atom-by-atom as a multicenter expansion of atom-centered Hankel functions on a cluster of atoms. Interaction matrix elements are calculated from a tailed representation of the NTBOs, which is a one-center expansion. The region beyond a specified tail-matching radius is replaced differentiably by superposition of two exponential functions.
The interaction is projected on the natural tight-binding orbitals. The U-tensor is limited matrix elements where all orbitals reside on a bond, respectively a pair on a neighborlist, and can be further restricted by setting the keywords below.

NOFL

Number of tight-binding orbitals per angular momentum. The number of tight-binding orbitals per angular momentum must not exceed the number of projector functions. The latter is the default for the number of tight-binding orbitals.

Type: integer array
Rules: optional.
Default: values from !STRUCTURE!SPECIES:NPRO

RAUG/RCOV

Augmentation radius in units of the covalent radius of the atom. The augmentation radius defines where the partial waves are matched to the envelope function.

Type: real
Rules: optional
Default: 1.

RTAIL/RCOV

Tail-matching radius in units of the covalent radius of the atom. At the tail-matching radius the exponential tails are matched differentiably to the envelope function. The tail-matching radius may be larger or smaller than the augmentation radius.
Type: real
Rules: optional
Default: 1.

RTAILCUT/RCOV

Tail-truncation radius in units of the covalent radius of the atom. Beyond the tail-truncation radius, the one-center expansion of the orbitals is set to zero. Reasonable values range from 1.3 to 3.
Type: real
Rules: optional
Default: 99.

TAILLAMBDA

Decay constants of the two exponentials, that replace the tails of the orbitals.
Type: 2×real
Rules: optional
Default: (/4.,2./)

LHFWEIGHT

Overwrites the generic mixing factor for exchange of the onsite terms of this atom type.
Type: real
Rules: Optional
Default: none

FOCKSETUP

Partial waves and projector functions are obtained from an atom calculated by a hybrid functional
Type: logical

Rules: Optional
Default: false

CV

Switch for core-valence exchange
Type: logical
Rules: optional
Default: true

NDDO

Exchange term where the density of two orbitals of one atom interacts with the density of two orbitals of a bond partner
Type: logical
Rules: optional
Default: true

31

Exchange term with three orbitals on one site and one on the opposite side of a bond. That is, the density of two orbitals of one atom interacts with the overlap density of two orbitals, one from each of the bond partners.
Type: logical
Rules: optional
Default: true

BONDX

Exchange term of between two overlap densities from the same bond. Each overlap density is made from two orbitals centered at differnt bond partners.
Type: logical
Rules: optional
Default: true

6.3.14 !STRUCTURE!ATOM

Rules: mandatory, multiple
Description: Specifies one atom.

NAME

Atom name. If the Keyword "SP" is not specified, the first two characters have to be identical with the "NAME" of the corresponding species. It is recommended that the name start with the element symbol, with a underscore instead of a blank for one-letter symbols. All atoms must have different names.

Type: character
Rules: Mandatory
Default: None

R

Atomic coordinates in Cartesian coordinates and units of "!STRUCTURE!GENERIC:LUNIT". Note that the coordinates need not be updated, because the instantaneous coordinates are held in the restart file.

Type: real(3)
Rules: Mandatory; must follow atom name
Default: None

M

Mass of this atom. Overwrites value specified in block "SPECIES".

Type: real
Rules: optional
Default: Value specified for the corresponding species

SP

Name of the atom type (species) to which this atom belongs.

Type: character
Rules: optional
Default: First two letters of the atom name

6.3.15 !STRUCTURE!OCCUPATIONS

Rules: optional
Description: defines number of bands and the initial occupations of the one-particle states. The default occupations are chosen according to the assumption of zero temperature and that energies of the states increase with band index and that spin directions and k-points are degenerate. These default occupations can be changed, first by specifying the occupation of each state in !STRUCTURE!OCCUPATIONS!STATE and second when using the Mermin functional in !CONTROL!MERMIN.

NSPIN

Number of spin directions. (1 for spin-restricted calculations and 2 for spin-polarized calculations, 3 for noncollinear spins)
Type: integer
Rules: optional
Default: 1

NBAND

Number of bands. A band is the number of states per k-point, where a state can hold two electrons in a spin-restricted calculation and one electron otherwise.
Type: integer
Rules: optional
Default: calculated from the number of valence electrons and the highest specified band in block "OCCUP"(ations).

EMPTY

Number of unoccupied bands. (see also keyword "NBAND")
Type: integer
Rules: optional

Default: calculated from the number of valence electrons and the highest specified band in block "OCCUP"(ations).

CHARGE[E]

Ionization state of the entire system. An electron has the charge CHARGE[E]=-1. May be overwritten by "!STRUCTURE!STATE"
Type: real
Rules: optional
Default: 0

SPIN[HBAR]

Total spin in $\hbar$. A single electron has SPIN[HBAR]=0.5 $\hbar$. May be overwritten by "!STRUCTURE!STATE"
Type: real
Rules: optional
Default: 0

6.3.16 !STRUCTURE!OCCUPATIONS!STATE

Rules: optional
Description: allows one to change the number of electrons for certain states from the standard occupation. Standard occupation: states are completely filled beginning with the first, until the system is neutral; the highest occupied state may be partially occupied; all k-points and spin directions are occupied equally. Note that the occupation must decrease monotonically with the band index, unless the option !CNTL!PSIDYN!SAFEORTHO=F is specified.

K

selects a k-point. Number refers to the order in which the k-points appeared.
Type: integer
Rules: optional
Default: all k-points are selected

S

selects a spin (1 or 2). S=2 is allowed only for spin polarized calculations (see !STRUCTURE!GENERIC)

Type: integer
Rules: optional
Default: all spin directions are selected

B

selects band

Type: integer
Rules: mandatory
Default: none

F

occupation

Type: real
Rules: mandatory
Default: none

6.3.17 !STRUCTURE!ISOLATE

Rules: optional
Description: Decouples the system electrostatically from its periodic images [10]. Recommended isolated molecules. Isolate is also required to calculate approximate point charges for the QM-MM coupling.

NF

Number of Gaussians used to fit the charge density. Should contain at least two.

Type: integer
Rules: optional
Default: 3

RC

Smallest decay constant for the Gaussians.
Type: real
Rules: optional
Default: 0.5

RCFAC

Scaling factors for the decay constants. $r_c(i) = r_c(1) * \mathrm{RCFAC}^{i-1}$.
Type: real
Rules: optional
Default: 1.5

GMAX2

Cutoff for the plane waves of the pseudo charge density contributing to the fit
Type: real
Rules: optional
Default: 3.

DECOUPLE

switches electrostatic decoupling of periodic images on.
Type: logical
Rules: optional
Default: .true. (.false. if the block !STRUCTURE!ISOLATE is not present)

6.3.18 !STRUCTURE!CONFINE

Rules: optional

Description: Places a confining potential around the molecule which mimics the Pauli repulsion of a solvent. The potential starts to raise at a radius $r_{in,R}$, chosen equal to the van der Waals radius of the respective atom and reaches the maximum value V_0 at a rsadius $r_{out,R}$ equal to twice the Van der Waals radius. The confining potential has the form

$$v(\vec{r}) = V_0 \prod_R f\left(\frac{|\vec{r} - \vec{R}| - r_{in,R}}{r_{out,R} - r_{in,R}}\right)$$

where $f(x)$ is a differentiable step-like function, that serves to switch off the potential inside the cavity

$$f(x) = \begin{cases} 0 & \text{for} \quad x < 0 \\ 3x^2 - 2x^2 & \text{for} \quad 0 < x < 1 \\ 1 & \text{for} \quad 1 < x \end{cases}$$

POT

Height V_0 of the confinement potential
Type: real
Rules: optional
Default: 20.

6.3.19 !STRUCTURE!QM-MM

Rules: optional, under development

Description: Describes the quantum-mechanical molecular-mechanical coupling. The coupling involves three different systems. (1) The quantum mechanical (QM) molecule, (2) The molecular mechanical (MM) shadow of the QM molecule and (3) the MM embedding environment, which also includes the molecule. The atoms of the QM molecule are repeated both in the MM shadow and in the MM environment, and their positions are constrained to be identical in all three parts. The total energy is $E_1 + E_3 - E_2$. The total energy $E_3 - E_2$ decribes the external forces and potentials acting from the environment on the QM molecule.

Bonds are linked in the following way: Suppose there is a bond from atom X to atom Y, where X is part of the QM molecule, but Y is only part of the environment. The bond of the QM molecule is saturated, for example by a hydrogen attached to X. An image of this hydrogen atom and its bond to X is introduced into the shadow. By specifying the option ONLY='SHADOW', this atom is (a) not constrained to be on the same position as its QM image, and (b) it is not introduced into the MM environment. (Its initial positions, however, are taken from the QM molecule.) Atom Y and its bond to X are introduced into the MM environment. As a result the two dummy hydrogen atoms move independently of each other. The bond to X is introduced twice, first linking X-H and then linking X-Y.

6.3.20 !STRUCTURE!QM-MM!ATOM

Rules: mandatory, multiple

Description: Describes the quantum-mechanical molecular-mechanical coupling.

NAME

Atom name. The atom names in the QM-MM block can be chosen independently from the choice in !STRUCTURE!ATOM.

Type: Character
Rules: Mandatory
Default: None

FFTYPE

Atom type name describing the setting of the parameters. The naming convention must be consistent with the force field chosen.
Type: Character
Rules: Mandatory
Default: None

QMATOM

Links the atomic position of this atom to the QM atom with the specified name (the name refers to that specified in !STRUCTURE!ATOMS. No atomic positions, masses or charges need to be specified for link atoms.
Type: Character
Rules: optional
Default: None

R

atomic position.
Type: real(3)
Rules: mandatory if QMM is not specified
Default: None

Q

charge in electron charges (i.e. a positive value indicates a negatively charged atom.
Type: real
Rules: optional
Default: 0.0

M

Mass in proton masses
Type: real
Rules: optional
Default: interprets the first two letters of FFTYPE (a underscore in place of a blank is removed) as element symbol for lookup in the periodic table

ONLY

specifies whether an atom is only in the MM shadow of the QM part but not in the MM part including the environment (ONLY='SHADOW')
Type: character
Rules: optional
Default: Atom is part of the MM environment

6.3.21 !STRUCTURE!QM-MM!BOND

Rules: optional,multiple
Description: specifies a bond for the classical force field

ATOM1

name of the first atom in the bond, corresponding to !STRUCTURE!QM-MM!ATOM
Type: character
Rules: mandatory
Default: none

ATOM2

name of the second atom in the bond, corresponding to !STRUCTURE!QM-MM!ATOM
Type: character
Rules: mandatory

Default: none

BO

Bond order of the bond. BO=1 for a single bond, BO=2 for a double bond. BO=3 for a triple bond.
Type: real
Rules: optional
Default: 1.

6.3.22 !STRUCTURE!QM-MM!LINK

Rules: optional,multiple
Description: specifies a link-bond between the MM and QM parts.

MMJOINT

Name of the atom common to MM, QM, and Shadow atoms. The name refers to the MM part. An atom referred as MMJOINT, must have the attribute !QMMM!ATOM:QMATOM
Type: character
Rules: mandatory
Default: none

MMATOM

Name of the MM atom bonded to MMJOINT. This atom must not have the attribute !QMMM!ATOM:QMATOM
Type: character
Rules: mandatory
Default: none

QMATOM

Name of the QM atom bonded to MMJOINT. This atom must not be used as attribute !QMMM!ATOM:QMATOM to any MM atom.

Type: character
Rules: mandatory
Default: none

SHFFTYPE

Atom type name describing the setting of the parameters for the shadow dummy atom. The naming convention must be consistent with the force field chosen.
Type: Character
Rules: Mandatory
Default: None

6.3.23 !STRUCTURE!COSMO

Rules: optional
Description: Here, data is provided that pertains to the representation of the solute surface. For each atom a radius that specifies the distance of closest approach between solute and solvent for this atom.

TESSFILE

name of the tesselation file with the full path.
Type: Character
Rules: optional
Default: Default tesselation with 60 points.

EPSILON

Dielectric constant of the solvent
Type: real
Rules: optional
Default: 10^12 (metallic screening)

VPAULI

Only for testing purposes!

Type: real
Rules: optional
Default: 0.d0

6.3.24 !STRUCTURE!COSMO!ATOM

Rules: mandatory, multiple
Description: ThereHere, data is provided that pertains to the representation of the solute surface. For each atom a radius that specifies the distance of closest approach between solute and solvent for this atom.

NAME

Atom name. Must be correspond to !STRUCTURE!ATOM.
Type: Character
Rules: Mandatory
Default: None

RAD

Distance of closest approach between solute and solvent. Usually close to the vdW radius. Specify the distance in atomic units.
Type: Real
Rules: Mandatory
Default: None

6.3.25 !STRUCTURE!GROUP

Rules: Optional, multiple
Description: Groups a number of atoms together and defines a name for the group. A few groups are predefined: The group “ALL” containing all atoms and one group for all atoms for a given atom type with the name equal to species name

NAME

Group name
Type: Character
Rules: Mandatory
Default: None

ATOMS

Names of atoms in the group
Type: array of type character
Rules: mandatory
Default: None

6.3.26 !STRUCTURE!VEXT

Rules: Optional
Description: Applies an external potential to the atoms. Note that the total energy is changed by the external potential.

6.3.27 !STRUCTURE!VEXT!UNBIND

Rules: Optional, multiple
Description: Applies a pair potential between the selected atoms, which vanishes beyonmd the sum of the van der Waals radii, and increases in a parabolic way inwardly. This option can be used in order to avoid the formation of covalent bond between certain atoms.

atom1

atom id according to !STRUCTURE!ATOM:NAME
Type: Character
Rules: ATOM1 or GROUP1 must be specified
Default: None

GROUP1

GROUP id according to !STRUCTURE!GROUP:NAME
Type: Character
Rules: ATOM1 or GROUP1 must be specified
Default: None

E

Energy of the potential at the sum of covalent radii
Type: real
Rules: mandatory
Default: None

6.3.28 !STRUCTURE!CONSTRAINTS

Rules: Optional
Description: Lists all constraints acting on atomic positions. The constraints are enforced using the scheme proposed by Ryckaert, Cicotti and Berendsen [48], which conserves the total energy in a MD simulation. Constraints are used to search for transition states, to force a system adiabatically over a reaction barrier and, to calculate free energies of reaction at finite temperatures, or to map out the total energy in a subspace of the total phase space [49]. If not specified otherwise the value and velocities are given in atomic units. `LUNIT` is NOT used.

6.3.29 !STRUCTURE!CONSTRAINTS!RIGID

Rules: optional, multiple
Description: constrains a group of atoms as a rigid body. Should not be used for linear molecules.

GROUP

Name of the group to be kept rigid

Type: Character
Rules: Mandatory
Default: None

6.3.30 !STRUCTURE!CONSTRAINTS!FREEZE

Rules: optional, multiple
Description: Keeps the specified atom or group fixed in space.

GROUP

Name of the atom-group to be fixed in space.
Type: Character
Rules: either "GROUP" or "ATOM" must be specified
Default: None

ATOM

Name of the atom to be fixed in space.
Type: Character
Rules: either "GROUP" or "ATOM" must be specified
Default: None

6.3.31 !STRUCTURE!CONSTRAINTS!TRANSLATION

Rules: optional, multiple
Description: suppresses a translational momentum of a group of atoms in a given direction.

GROUP

Name of group to be fixed in space.
Type: character

Rules: Optional
Default: 'ALL'

DIR

Name of group to be fixed in space.
Type: real(3)
Rules: Optional
Default: Translation into all three space direction is suppressed

6.3.32 !STRUCTURE!CONSTRAINTS!ROTATION

Rules: optional,multiple

Description: Supresses the angular momentum $\vec{L}$ of a group of atoms in a given direction $\vec{e}$ in the reference frame of the center of gravity $\vec{R}_C(t)$ of the specified group.Thus the constraint enforces

$$\vec{e}\vec{L} = \vec{e}\sum_R M_R \left(\vec{R}_R(t) - R_C(t)\right) \times \left(\dot{\vec{R}}_R - \dot{\vec{R}}_C\right) = 0$$

Note, that this constraint cannot be represented as a hypersurface in phase space. It should not be used for phase space sampling or transition state searches. For those purposes, use the ORIENTATION constraint.

GROUP

Name of group to be fixed in space.
Type: Character
Rules: Optional
Default: 'ALL'

AXIS

Axis for which the angular momentum shall be constrained.

Type: real(3)
Rules: Optional
Default: Angular momentum into all three space directions is suppressed

6.3.33 !STRUCTURE!CONSTRAINTS!ORIENTATION

Rules: Not tested, optional, multiple
Description: Fixes the orientation of a group of atoms in a given direction. It is to be used together with a saddle search for isolated molecules. Note that this constraint does not imply zero angular momentum.

GROUP

Name of group to be fixed in space.
Type: Character
Rules: Optional
Default: 'ALL'

AXIS

Axis for which the rotation shall be constrained.
Type: real(3)
Rules: Optional
Default: translation into all three space directions is suppressed

6.3.34 !STRUCTURE!CONSTRAINTS!BOND

Rules: optional, multiple
Description: Fixes the distance between two atoms.

ATOM1

Name of the first atom. (Extended notation, see section 4.2 on p. 8)
Type: Character

Rules: mandatory
Default: none

ATOM2

Name of the second atom. (Extended notation, see section 4.2 on p. 8)
Type: Character
Rules: mandatory
Default: none

MOVE

Defines whether constrained value shall be moved to a new location or whether it should be given a specified constant velocity. The method uses a lowest order polynomial adjusted to the initial value and the specified final value and/or velocity to specify the time evolution of the constraint values.
Type: logical
Rules: optional
Default: .false.

NSTEP

number of time steps after which the new constraint values shall be satisfied
Type: integer
Rules: mandatory if MOVE=.true.
Default: none

VALUE

specified new value for the constraint.
Type: REAL
Rules: optional
Default: current value of the constraint.

VELOC

specified new value for the velocity of the constraint.
Type: REAL
Rules: optional
Default: 0.0

SHOW

print value and force of the constraint in the "CONSTRAINTS" file (standard extension: _constr.report)
Type: LOGICAL
Rules: optional
Default: .FALSE.

FLOAT

defines this constraint as floating constraint.
Type: logical
Rules: optional
Default: .false.

M

Mass of the constraint in a.u. A negative value results in an energy maximizing evolution for saddle point determination.
Type: real
Rules: optional
Default: 1.0=m_e

FRIC

Friction coefficient for the constraint dynamics.
Type: real
Rules: optional
Default: 0.0

6.3.35 !STRUCTURE!CONSTRAINTS!ANGLE

Rules: optional, multiple
Description: Fixes the bond angle between three atoms.

ATOM1

Name of the first atom.
Type: Character
Rules: mandatory
Default: none

ATOM2

Name of the second, central atom.
Type: Character
Rules: mandatory
Default: none

ATOM3

Name of the third atom.
Type: Character
Rules: mandatory
Default: none

MOVE

Defines whether constrained value shall be moved to a new location or whether it should be given a specified constant velocity. The method uses a lowest order polynomial adjusted to the initial value and the specified final value and/or velocity to specify the time evolution of the constraint values.
Type: logical
Rules: optional
Default: .false.

NSTEP

number of time steps after which the new constraint values shall be satisfied

Type: integer
Rules: mandatory if MOVE=.true.
Default: none

VALUE[DEG]

specified new value for the constraint in degrees.

Type: REAL
Rules: optional
Default: current value of the constraint.

VELOC

specified new value for the velocity of the constraint.

Type: REAL
Rules: optional
Default: 0.0

SHOW

print value and force of the constraint in the "CONSTRAINTS" file (standard extension: _constr.report)

Type: LOGICAL
Rules: optional
Default: .FALSE.

FLOAT

defines this constraint as floating constraint.

Type: logical
Rules: optional
Default: .false.

M

Mass of the constraint in a.u. A negative value results in an energy maximizing evolution for sassle point determination.
Type: real
Rules: optional
Default: 1.0=m_e

FRIC

Friction coefficient for the constraint dynamics.
Type: real
Rules: optional
Default: 0.0

6.3.36 !STRUCTURE!CONSTRAINTS!TORSION

Rules: optional, multiple

Description: Fixes the torsion along a bond.

ATOM1

Name of the first and first terminal atom, connected to ATOM2.
Type: Character
Rules: mandatory
Default: none

ATOM2

Name of the second, the first central atom.
Type: Character
Rules: mandatory
Default: none

ATOM3

Name of the third, the second central atom.
Type: Character
Rules: mandatory
Default: none

ATOM4

Name of the fourth, the second terminal atom, connected to ATOM3.
Type: Character
Rules: mandatory
Default: none

MOVE

Defines whether constrained value shall be moved to a new location or whether it should be given a specified constant velocity. The method uses a lowest order polynomial adjusted to the initial value and the specified final value and/or velocity to specify the time evolution of the constraint values.
Type: logical
Rules: optional
Default: .false.

NSTEP

number of time steps after which the new constraint values shall be satisfied
Type: integer
Rules: mandatory if MOVE=.true.
Default: none

VALUE

specified new value for the constraint. (A torsion angle of zero corresponds to a planar trans configuration)
Type: REAL
Rules: optional

Default: current value of the constraint.

VELOC

specified new value for the velocity of the constraint.
Type: REAL
Rules: optional
Default: 0.0

SHOW

print value and force of the constraint in the "CONSTRAINTS" file (standard extension: _constr.report)
Type: LOGICAL
Rules: optional
Default: .FALSE.

FLOAT

defines this constraint as floating constraint.
Type: logical
Rules: optional
Default: .false.

M

Mass of the constraint in a.u. A negative value results in an energy maximizing evolution for sassle point determination.
Type: real
Rules: optional
Default: 1.0=m_e

FRIC

Friction coefficient for the constraint dynamics.
Type: real
Rules: optional
Default: 0.0

6.3.37 !STRUCTURE!CONSTRAINTS!COGSEP

Rules: optional, multiple
Description: Fixes the distance between the center of gravities of two atom groups.

GROUP1

First group of atoms
Type: Character
Rules: mandatory
Default: none

GROUP2

second group of atoms.
Type: Character
Rules: mandatory
Default: none

MOVE

Defines whether constrained value shall be moved to a new location or whether it should be given a specified constant velocity. The method uses a lowest order polynomial adjusted to the initial value and the specified final value and/or velocity to specify the time evolution of the constraint values.
Type: logical
Rules: optional
Default: .false.

NSTEP

number of time steps after which the new constraint values shall be satisfied
Type: integer

Rules: mandatory if MOVE=.true.
Default: none

VALUE

specified new value for the constraint.
Type: REAL
Rules: optional
Default: current value of the constraint.

VELOC

specified new value for the velocity of the constraint.
Type: REAL
Rules: optional
Default: 0.0

SHOW

print value and force of the constraint in the "CONSTRAINTS" file (standard extension: _constr.report)
Type: LOGICAL
Rules: optional
Default: .FALSE.

FLOAT

defines this constraint as floating constraint.
Type: logical
Rules: optional
Default: .false.

M

Mass of the constraint in a.u. A negative value results in an energy maximizing evolution for sassle point determination.
Type: real
Rules: optional

Default: 1.0=m_e

FRIC

Friction coefficient for the constraint dynamics.
Type: real
Rules: optional
Default: 0.0

6.3.38 !STRUCTURE!CONSTRAINTS!MIDPLANE

Rules: optional, multiple
Description: Constrains the ATOM2 into a plane perpendicular to the vector from ATOM1 to ATOM3. The plane is attached at a point $R_1 + x(R_3 - R_1)$, where x is the value of the constraint. In other words $x = (R_2 - R_1)(R_3 - R_1)/|R_3 - R_1|^2$.

ATOM1

Name of the first atom (Extended notation, see section 4.2 on p. 8)
Type: Character
Rules: mandatory
Default: none

ATOM2

Name of the second atom (Extended notation, see section 4.2 on p. 8)
Type: Character
Rules: mandatory
Default: none

ATOM3

Name of the third atom (Extended notation, see section 4.2 on p. 8)
Type: Character

Rules: mandatory
Default: none

MOVE

Defines whether constrained value shall be moved to a new location or whether it should be given a specified constant velocity. The method uses a lowest order polynomial adjusted to the initial value and the specified final value and/or velocity to specify the time evolution of the constraint values.
Type: logical
Rules: optional
Default: .false.

NSTEP

number of time steps after which the new constraint values shall be satisfied
Type: integer
Rules: mandatory if MOVE=.true.
Default: none

VALUE

specified new value for the constraint.
Type: REAL
Rules: optional
Default: current value of the constraint.

VELOC

specified new value for the velocity of the constraint.
Type: REAL
Rules: optional
Default: 0.0

SHOW

print value and force of the constraint in the "CONSTRAINTS" file (standard extension: _constr.report)
Type: LOGICAL
Rules: optional
Default: .FALSE.

FLOAT

defines this constraint as floating constraint.
Type: logical
Rules: optional
Default: .false.

M

Mass of the constraint in a.u. A negative value results in an energy maximizing evolution for sassle point determination.
Type: real
Rules: optional
Default: 1.0=m_e

FRIC

Friction coefficient for the constraint dynamics.
Type: real
Rules: optional
Default: 0.0

6.3.39 !STRUCTURE!CONSTRAINTS!LINEAR

Rules: optional, multiple
Description: Specifies a linear constraints of the form $\sum_R c_R R_R = a$. This very flexible type of constraint can be used to represent a complex constraint approximately in the neighborhood of a given structure. Hereby the nonlinear constraint is expanded into a Taylor expansion, and the first two terms are retained.

MOVE

Defines whether constrained value shall be moved to a new location or whether it should be given a specified constant velocity. The method uses a lowest order polynomial adjusted to the initial value and the specified final value and/or velocity to specify the time evolution of the constraint values.

Type: logical
Rules: optional
Default: .false.

NSTEP

number of time steps after which the new constraint values shall be satisfied

Type: integer
Rules: mandatory if MOVE=.true.
Default: none

VALUE

specified new value for the constraint.

Type: REAL
Rules: optional
Default: current value of the constraint.

VELOC

specified new value for the velocity of the constraint.

Type: REAL
Rules: optional
Default: 0.0

SHOW

print value and force of the constraint in the "CONSTRAINTS" file (standard extension: _constr.report)

Type: LOGICAL

Rules: optional
Default: .FALSE.

FLOAT

defines this constraint as floating constraint.
Type: logical
Rules: optional
Default: .false.

M

Mass of the constraint in a.u. A negative value results in an energy maximizing evolution for sassle point determination.
Type: real
Rules: optional
Default: 1.0=m_e

FRIC

Friction coefficient for the constraint dynamics.
Type: real
Rules: optional
Default: 0.0

6.3.40 !STRUCTURE!CONSTRAINTS!LINEAR!ATOM

Rules: mandatory, multiple
Description: Specifies the contribution of one atom to the constraint vector

NAME

name of the atom.
Type: character
Rules: mandatory
Default: none

R

vector defining c_R
Type: REAL(3)
Rules: mandatory
Default: none

6.3.41 !STRUCTURE!ORBPOT

Rules: optional
Description: applies external potentials acting on orbitals. The relation $|\Psi\rangle \approx \sum_i |\phi_i\rangle\langle\tilde{p}_i|\tilde{\Psi}\rangle$ between the true wave function and its one-center expansion is used to represent the energy of a potential u that is localized well withing the augmentation region as

$$\sum_n \langle\Psi_n|u|\Psi_n\rangle \approx \sum_{i,j} D_{i,j}U_{j,i} \ ,$$

where $D_{i,j} = \sum_n \langle\tilde{p}_i|\tilde{\Psi}_n\rangle\langle\tilde{\Psi}_n|\tilde{p}_j\rangle$ is the one-center density matrix and $U_{i,j} = \langle\phi_i|u|\phi_j\rangle$.
The special angular dependent form

$$u(r,r') = U_0 \sum_m \mathrm{e}^{-(r/r_c)^q}\delta(|r| - |r'|)Y_L(r)Y_L(r')$$

is used, where $L = (\ell, m)$ and U_0, r_c, q, ℓ and the atomic site R, which is used as origin in the above equation, are input parameters.

6.3.42 !STRUCTURE!ORBPOT!POT

Rules: optional, multiple
Description: describes external potentials acting on orbitals

ATOM

atom name or species name of the atom on which the potential should act, referring to name of !STRUCTURE!ATOMS, if a species name is given the potential will be applied to all atoms of that species.
Type: character
Rules: mandatory
Default: none

VALUE

value of the external potential in Hartree
Type: real
Rules: mandatory
Default: none

TYPE

orbital type, can be 'S', 'P', 'D', 'ALL'. One can also specify specific angular momenta such as 'PX', 'PY', 'PZ', 'DX2-Y2', 'DXZ', 'D3Z2-R2', 'DYZ', 'DXY'. For the f-orbitals we use the notation 'F1', ..., 'F7'.
Type: character
Rules: mandatory
Default: none

rc

cutoff radius r_c as defined above
Type: real
Rules: mandatory
Default: none

pwr

exponent q for the cutoff radius as defined above
Type: real
Rules: optional
Default: 2.0

S

restricts the spin on which the potential act. If this parameter is omitted the potential acts on all electrons irrespective of their spin direction. For a collinear spin-polarized potential only $S = 1$ is allowed and defines a potential acting on the z-component of the spin. For a non-collinear calculation (NSPIN=3 in !STRUCTURE!GENERIC) the values of $S = 1, S = 2, S = 3$ refer to the x,y,z components of the spin respectively.

Type: integer
Rules: optional
Default: 0

7 The Energy Grab Tool: paw_grab

The tool paw_grab is useful to collect energies from different systems and compile a set of reaction energies. It looks for the last (long) printout of energies in a given protocol. This total energy is then attributed to a given molecule name. Arbitrary reaction energies between the selected molecules can automatically be compiled. Particularly useful is that a number of reaction energies can be updated with the latest computed energies using a single command.

7.1 Command

The calling sequence is

`paw_grab` *controlfile*

where *controlfile* is the file name of the control input file for the paw_grab tool. I recommend the extension ".gcntl".

7.2 Example for the control input file

```
!GCNTL
  !GENERIC UNIT(E)='KJ/MOL' !END
  !SUBSTANCE ID='H2' FILE='~/PAW/Sample/h2.prot' !END
  !SUBSTANCE ID='CO' FILE='~/PAW/Sample/co.prot' !END
  !SUBSTANCE ID='H2CO' FILE='~/PAW/Sample/h2co.prot' !END
  !REACTION
    !FROM ID='H2'   FAC=1.  !END
    !FROM ID='CO'   FAC=1.  !END
    !TO   ID='H2CO' FAC=1.  !END
  !END
!END
!EOB
```

7.3 Argument description for the control input file

7.3.1 !GCNTL

Rules: mandatory
Description: defines location of the protocol files to be searched and the reaction energies to be calculated.

7.3.2 !GCNTL!GENERIC

Rules: optional
Description: general definitions

UNIT(E)

Identifier of the energy unit. Can be “HARTREE”, “KJ/MOL” “KCAL/MOL”, “RY”, “EV”, “JOULE”
Type: character
Rules: optional
Default: “HARTREE”

7.3.3 !GCNTL!SUBSTANCE

Rules: optional
Description: defines a reactand or reaction product through a protocol file of the simulation code

ID

Identifier of the substance
Type: character
Rules: mandatory
Default: none

FILE

File name of the protocol file
Type: character
Rules: mandatory
Default: none

7.3.4 !GCNTL!REACTION

Rules: optional, multiple
Description: defines the reaction

ID

Identifier of the reaction.
Type: character
Rules: optional
Default: A stochiometric formula of the reaction created from reactands and products and their stochiometries

7.3.5 !GCNTL!REACTION!FROM

Rules: optional, multiple
Description: defines the reactand

ID

Identifier of the reactand
Type: character
Rules: mandatory
Default: none

FAC

defines the stochiometry with which the the reactand takes part in the reaction

Type: real
Rules: mandatory
Default: none

7.3.6 !GCNTL!REACTION!TO

Rules: optional, multiple
Description: defines the reaction product

ID

Identifier of the product
Type: character
Rules: mandatory
Default: none

FAC

defines the stochiometry with which the product is produced by the reaction
Type: real
Rules: mandatory
Default: none

8 The Atomic Structure Analysis Tool: "paw_strc"

8.1 Command

The tool paw_strc.x allows a first analysis of the atomic structure and prepares some structure files in common formats for use with molecular or crystal viewers. This tool uses the *rootname*.strc_out file of the simulation code, and writes the file *rootname*.cssr, *rootname*.xyz and *rootname*.cml. The protocoll file *rootname*.sprot contains, among others, bond lengths and bond angles and a segment to be used in the input structure file.

The calling sequence is

`paw_strc` *options rootname*

where rootname is the name of the protocol file without the ".prot" ending and option is one of the following:

-c considers the system as a crystal. (Default is use as a molecule)

-c*ijk* like -c but multiplies the lattice in the xyz file by factors i,j,k in the three lattice directions. *ijk* is a three digit integer composed out of three single digit numbers i,j,k by sinple concatenation. (No spaces!)

-i real from input structue file .strc instead from the output structure file .strc_out.

The xyz file contains the molecule or a cluster made of the atoms in one or several unit cells.

9 The Wave Function and Density Analysis Tool: "paw_wave"

The paw_wave analysis tool converts a datafile for wave functions or densities into a input format to be used by graphical tools.

The datafile used as input by the paw_wave analysis tool is produced by the option "!CONTROL!ANALYSE!WAVE" in the control input file of the simulation code. The paw_wave analysis tool uses in addition the structure information from the 'strc_out' file of the simulation code.

The output formats are the "cub" format and a "dx" format. The cub-format is specified by the GAUSSIAN code and can be used by most molecular viewers such as AVOGADRO. The dx format can be used by Dataexplorer OPENDX [13] using the Dataexplorer program "wave.net". (See directory "dx")

9.1 Command

The calling sequence is

`paw_wave` *controlfile*

where *controlfile* is the file name of the control input file for the paw_wave tool described below. I recommend the extension ".wcntl" for the control file of the density of states analysis tool.

9.2 Example for the control input file

```
!WCNTL
  !FILES
    !FILE ID='WAVE'
          NAME='~/PAW/Sample/h2co-homo.wv' !END
    !FILE ID='STRC'
          NAME='~/PAW/Sample/h2co.strc_out' !END
    !FILE ID='WAVEDX'
          NAME='~/PAW/Sample/h2co-w.dx' !END
  !END
  !VIEWBOX O=-5. -5. -5.
           T=10. 0. 0. 0. 10. 0. 0. 0. 10. !END
  !PLANE   O=-5. -5. -5. T=10. 0. 0.   0. 10. 10. !END
```

```
!END
!EOB
```

9.3 Argument description for the control input file

9.3.1 !WCNTL

Rules: mandatory
Description: defines the operations done on the system; largely independent of the system

9.3.2 !WCNTL!FILES

Rules: optional
Description: defines the energy grid and energy broadening

9.3.3 !WCNTL!FILES!FILE

Rules: optional, multiple
Description: defines filenames.

ID

Identifier of the file. Can be

- 'STRC',
- 'WAVE',
- 'WAVEDX',
- 'CUBE' (default extension '.cub')
- 'WRL' (default extension '.wrl')
- 'GNUCONTOUR' (default extension '_c.gnu')
- 'GNURUBBERSHEET' (default extension '_r.gnu')

Type: character
Rules: mandatory
Default: none

EXT

rootname flag: if true, the filename is interpreted as an extension of the rootname. The rootname is the filename of the wavecontrol file with out the last dot and the following characters.
Type: logical
Rules: optional
Default: .false.

NAME

File name. The default extension is 'wv' for the WAVE' file, 'strc' for the 'STRC' file, 'dx' for the output file for the data explorer, 'cub' for the output file in the cube format.
Type: character
Rules: mandatory
Default: none

9.3.4 !WCNTL!GENERIC

Rules: optional
Description: Sets generic data

MIN

If specified, data below this value are truncated.
Type: real
Rules: optional
Default: none

MAX

If specified, data above this value are truncated.
Type: real
Rules: optional
Default: none

9.3.5 !WCNTL!VIEWBOX

Rules: optional
Description: Sets the view box. Only the data within the view box are shown. The view box is defined by an origin o and three vectors t_1, t_2, t_3. The eight corners of the view box are $o, o + t_1, o + t_2, o + t_3, o + t_1 + t_2, o + t_2 + t_3, o + t_1 + t_3, o + t_1 + t_2 + t_3$

O

Origin (lower left, forward point of the view box) in a.u.
Type: real(3)
Rules: optional
Default: 0. 0. 0.

T

vectors defining the edges of the view box in a.u.
Type: real(3,3)
Rules: optional
Default: lattice vectors of the calculation

9.3.6 !WCNTL!PLANE

Rules: optional
Description: defines a plane for a rubbersheet plot. The plane is defined by an origin o and two vectors t_1, t_2.

O

Origin (lower,forward point of the plane) in a.u.
Type: real(3)
Rules: optional
Default: 0. 0. 0.

T

vectors defining the edges of the plane in a.u.
Type: real(3,2)
Rules: optional
Default: first two lattice vectors of the calculation

9.4 View data using OPENDX[13]

Start the Dataexplorer using

```
dx -image &
```

1. Select "**Load Macro**" from the "**file**" pulldown menu. In the file selector, which pops up, check that you are in the DX subdirectory of the PAW directory and click on "**Load all macros**".

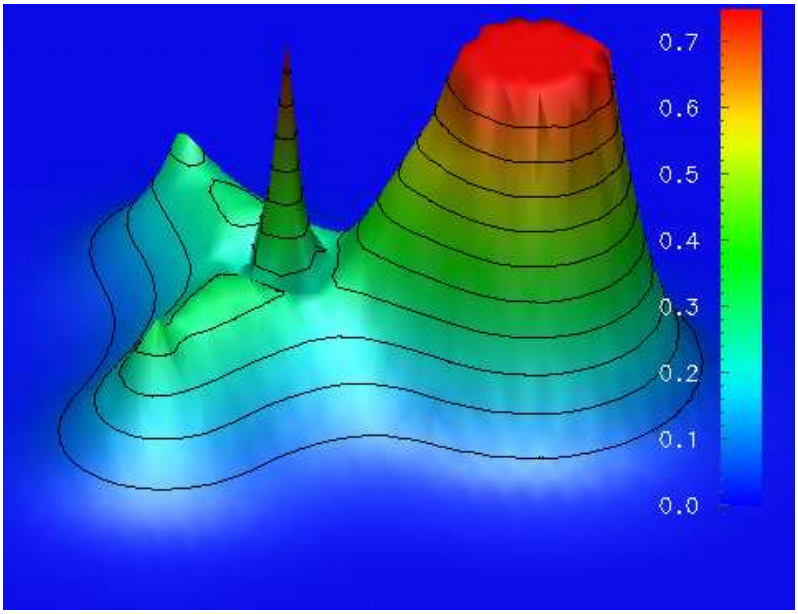
2. Select "**open**" from the "**file**" pulldown menu. In the file selector, which pops up, check that you are in the DX subdirectory of the PAW directory and select the dx program "**dx_wave**".

3. select "**open all control panels**" from the "**windows**" pulldown menu.

4. In the "**Main Control Panel**" that opens, select the input data file. You can do this by typing the file name into the text field of the file selector, or by pressing the button to the right of the text field and by maneuvering towards the right file in the file selector menu. After selecting the correct file, press OK.

5. Now select the the representation of your data. You can select

   - a ball-stick model of your atomic structure,
   - a surface of constant density or wave function amplitude (there is a golden surface for the value provided and a blue surface for the same value with opposite sign)
   - a rubber sheet representation of the density on a given plane. The plane is defined by a point through which the plane passes and a vector normal to the surface. You can, for example, select an atom position of interest as the point, and then adjust the orientation of the plane by modifying the vector. The plane will then rotate about the atom position selected.



If your viewing direction is straight onto the plane, you will obtain a (color) contour plot.

By adjusting the maximum value, you can cut off very large values, and thus obtain more detail for the lower density values.

9.5 View data using FreeWRL

The wave tool constructs a scene in the virtual Reality Modeling Language (VRML) format. The standard extension is ".wrl". A common viewer is FreeWRL.

VRML has been superceeded by the language X3D. The syntax is very similar and VRML files can be converted into X3D files.

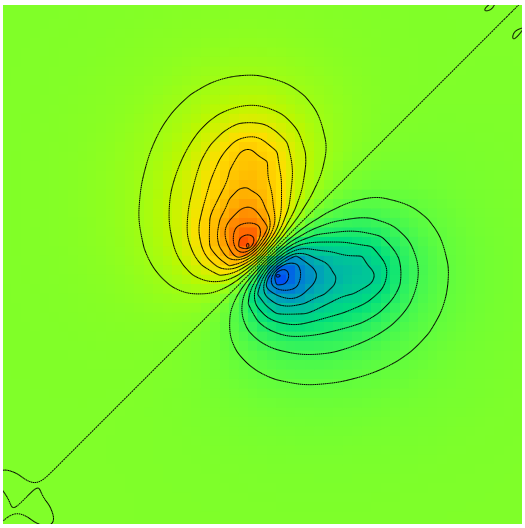
9.6 Contour plots using Gnuplot

If a plane has been sepcified in !WCNTL a input file for gnuplot to produce countour plots is generated. It has the File id. 'GNUCONTOUR' and the default file name *rootname*_c.gnu.

A plot is generated by the command

```
gnuplot rootname_c.gnu > tmp.eps
gv tmp.eps
```

The resulting file will look like



This plot may not be what one wants. You can edit the gnuplot file. The top part looks something like the following:

```
#
 #============================================================
 # DATA SECTION TO BE CHANGED BY THE USER                   ==
 #============================================================
 xmin=  -5.0000000000000000
 xmax=   5.0000000000000000
 ymin=  -5.0000000000000000
 ymax=   5.0000000000000000
 zmin= -0.52367716662429253
 zmax=  0.52367704490151912
 rot_x=0
 rot_z=0
 scale=   2.0000000000000000
 scale_z=1.
 #
 #============================================================
 # DEFINE LINE STYLES TO BE USED WITH LS IN SPLOT           ==
 #============================================================
 # DEFINE A LINESTYLE FOR SPLOT (USED WITH LS)
 set style line 1 lt 1 lc rgb "black" lw 1
 # MAP HIGHT VALUES TO COLORS
 set palette rgbformula 22,13,-31
 #
 #============================================================
```

```
# DATA RELATED STATEMENTS                                  ==
#============================================================
set xrange [xmin:xmax]
set yrange [ymin:ymax]
set zrange [zmin:zmax]
set dgrid3d  60,60,1     #SAMPLE DATA ONTO A 60X60 GRID
#
#============================================================
# SURFACE PLOT                                             ==
#============================================================
set surface
set hidden3d
set xyplane at 0.      # PLACE Z=0 INTO THE XY PLANE
unset border           # REMOVE AXES
unset xtics            # REMOVE TICS FROM THE AXES
unset ytics            # REMOVE TICS FROM THE AXES
unset ztics            # REMOVE TICS FROM THE AXES
set key off            # REMOVE TITLE
set pm3d hidden3d 0  # LINESTYLE FOR THE SURFACE GRID
#
#============================================================
# DEFINE CONTOUR                                           ==
#============================================================
set contour surface # DRAW CONTOUR ONTO THE SURFACE
set cntrparam levels incremental -0.5,0.05,0.5
set cntrparam bspline
set cntrparam order 6
unset clabel # NO AUTOCOLORING OF CONTOURS
#
#============================================================
# DEFINE ORIENTATION                                       ==
#============================================================
set view equal xy # avoids distortions of xy-plane
#  ANGLE, ANGLE, OVERALL SCALE, SCALE Z-AXIS
set view rot_x,rot_z,scale,scale_z
#
#============================================================
# SET Terminals (uncomment one)                            ==
#============================================================
#----USE POSTSCRIPT TERMINAL FOR EPS FILES-----------------
set terminal postscript eps size xmax-xmin,ymax-ymin  enhanced color clip font 'helvetica,20' linewidth 2
#----USE WXT TERMINAL FOR LINUX SCREEN---------------------
# set terminal wxt size 350,262 enhanced  font 'verdana,10' persist
#----USE PDF TERMINAL FOR PDF FILES------------------------
# set terminal pdf color enhanced
#----USE AQUA TERMINAL FOR OSX SCREEN----------------------
# set terminal aqua enhanced solid font 'helvetica,20'title 'contour'
#
#============================================================
# PERFORM PLOT                                             ==
#============================================================
# unset contour
set border
# unset surface
unset colorbox
splot '-' with pm3d linewidth 1 linecolor rgb '#000000'
#
#============================================================
# DATA SECTION                                             ==
#============================================================
 -5.0000000000000000       -5.0000000000000000       -6.3559036829332985E-009
 -5.0000000000000000       -4.8305084742605686        2.7132751008533330E-005
 -5.0000000000000000       -4.6610169485211372        8.1098647806454938E-005
 -5.0000000000000000       -4.4915254414081573        1.2624831976719569E-004
 -5.0000000000000000       -4.3220338970422745        1.1350877204762644E-004
 -5.0000000000000000       -4.1525423526763916        1.0682727783892541E-004
```

1. adjust the parameters `xmin`, `xmax`, `ymin`, `ymax` to select the viewing area.
2. adjust the values in `set cntrparam levels incremental -0.5,0.05,0.5` to select the values of the contours. The values are minimum value, increment, maximum value

3. if you do not like the coloring, uncomment the line 'unset surface'

4. select the terminal type to create a certain file type.

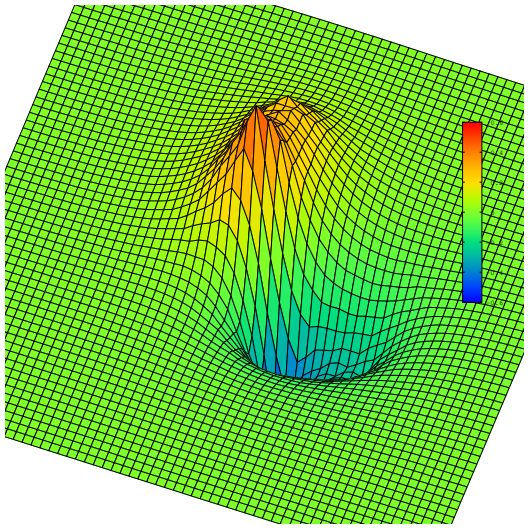
9.7 Rubbersheets using Gnuplot

If a plane has been sepcified in !WCNTL a input file for gnuplot to PRODUCE COUNTOUR PLOTS IS GENERATED. IT HAS THE FILE ID 'GNURUBBERSHEET' and the default file name *rootname*_r.gnu.

A plot is generated by the command

```
gnuplot rootname_r.gnu > tmp.eps
gv tmp.eps
```

The resulting file will look like



This plot may not be what one wants. You can edit the gnuplot file. The top part looks something like the following:

```
#
#============================================================
# DATA SECTION TO BE CHANGED BY THE USER                  ==
#============================================================
xmin=  -5.0000000000000000
xmax=   5.0000000000000000
ymin=  -5.0000000000000000
ymax=   5.0000000000000000
zmin= -0.52367716662429253
zmax=  0.52367704490151912
rot_x=30
rot_z=20
scale=   2.0000000000000000
scale_z=1.
#
#============================================================
# DEFINE LINE STYLES TO BE USED WITH LS IN SPLOT          ==
#============================================================
# DEFINE A LINESTYLE FOR SPLOT (USED WITH LS)
set style line 1 lt 1 lc rgb "black" lw 1
# MAP HIGHT VALUES TO COLORS
set palette rgbformula 22,13,-31
#
#============================================================
```

```
# DATA RELATED STATEMENTS                                  ==
#============================================================
set xrange [xmin:xmax]
set yrange [ymin:ymax]
set zrange [zmin:zmax]
set dgrid3d  60,60,1     #SAMPLE DATA ONTO A 60X60 GRID
#
#============================================================
# SURFACE PLOT                                             ==
#============================================================
set surface
set hidden3d
set xyplane at 0.      # PLACE Z=0 INTO THE XY PLANE
unset border           # REMOVE AXES
unset xtics            # REMOVE TICS FROM THE AXES
unset ytics            # REMOVE TICS FROM THE AXES
unset ztics            # REMOVE TICS FROM THE AXES
set key off            # REMOVE TITLE
set pm3d hidden3d 1  # LINESTYLE FOR THE SURFACE GRID
#
#============================================================
# DEFINE CONTOUR                                           ==
#============================================================
set contour surface # DRAW CONTOUR ONTO THE SURFACE
set cntrparam levels incremental -0.5,0.05,0.5
set cntrparam bspline
set cntrparam order 6
unset clabel # NO AUTOCOLORING OF CONTOURS
#
#============================================================
# DEFINE ORIENTATION                                       ==
#============================================================
set view equal xy # avoids distortions of xy-plane
#  ANGLE, ANGLE, OVERALL SCALE, SCALE Z-AXIS
set view rot_x,rot_z,scale,scale_z
#
#============================================================
# SET Terminals (uncomment one)                            ==
#============================================================
#----USE POSTSCRIPT TERMINAL FOR EPS FILES-----------------
set terminal postscript eps size xmax-xmin,ymax-ymin  enhanced color clip font 'helvetica,20' linewidth 2
#----USE WXT TERMINAL FOR LINUX SCREEN---------------------
# set terminal wxt size 350,262 enhanced  font 'verdana,10' persist
#----USE PDF TERMINAL FOR PDF FILES------------------------
# set terminal pdf color enhanced
#----USE AQUA TERMINAL FOR OSX SCREEN----------------------
# set terminal aqua enhanced solid font 'helvetica,20'title 'contour'
#
#============================================================
# PERFORM PLOT                                             ==
#============================================================
unset contour
# set border
# unset surface
# unset colorbox
splot '-' with pm3d linewidth 1 linecolor rgb '#000000'
#
#============================================================
# DATA SECTION                                             ==
#============================================================
 -5.0000000000000000       -5.0000000000000000       -6.3559036829332985E-009
 -5.0000000000000000       -4.8305084742605686        2.7132751008533330E-005
 -5.0000000000000000       -4.6610169485211372        8.1098647806454938E-005
 -5.0000000000000000       -4.4915254414081573        1.2624831976719569E-004
```

1. adjust orientation by changing the parameters `rot_x` and `rot_z`
2. adjust the viewing arey by changing the parameters `xmin`, `xmax`, `ymin`, `ymax`
3. if you like contours on the surface, comment the line 'unset surface'

4. select the terminal type to create a certain file type.

10 Wave function converter for CryMolCAD: “paw_cmcwave”

The tool converts the wave function file .wv produced by the simultation code into an input file for the CrymolCAD viewer. The wave function can be a general field such as wave function, density or potential.

paw_cmcwave.x case.wv

A new file is case.cmcv created, which can be read by CryMolCAD. The file contains only the information for the field. The structure information must be supplied separately.

11 Band-Offset and Work-Function Tool: "paw_1davpot"

The tool "paw_1davpot" produces the one dimensional potential obtained by averaging the potential over lattice planes. By comparing with the respective bulk calculations one can determine the band offsets or work functions.

The tool uses the electrostatic potential produces by the CP-PAW code using the option !control!analyse!potential. If the file of the resulting potential file is "case_pot.wv", the calling sequence is

paw_1davpot.x case_pot.wv

Three files are produced, namely AVPOT100.DAT, AVPOT010.DAT and AVPOT001.DAT. The first is an xy file of the averaged potential along the normal of the plane spanned by the lattice vectors $\vec{T}_2$ and $\vec{T}_3$. The other files contain the corresponding information along the two other plane normals.

In order to obtain a band offset from a supercell calculation determine first the one-dimensional potential $v_{sup}(z)$ averaged over planes parallel to the surface or interface. Next perform a calculation of the bulk material using exactly the same geometry (including any lateral strain), and determine the averaged potentials $v_{bulkA}(z)$ and $v_{bulk,B}$ for the same lattice planes. Determine furthermore the Fermi-level or the valence band top in the bulk materials $e_{vbt,A}$ and $e_{vbt,B}$.

By forming the corresponding differences, one obtains a spatially dependent valence band top $e'_{vbt,A}$ and $e'_{vbt,B}$ for the slab calculation.

$$e'_{vbt,A}(z) = e_{vbt,A} - v_{bulk,A}(z + \Delta z_A) + v_{sup}(z)$$
$$e'_{vbt,B}(z) = e_{vbt,B} - v_{bulk,A}(z + \Delta z_B) + v_{sup}(z)$$

The shift $\Delta z_{A/B}$ are needed to account for the difference in z-coordinates for the lattice planes in the bulk and in the supercell.

If the spatially dependent valence band tops converge to a constant value in the interior of the slab, this value can be taken as the position of the valence band top in the supercell.

12 The Trajectory Analysis Tool: “paw_tra”

This trajectory analysis tool is used to inspect atomic trajectories. It allows one to create a movie input file for OPENDX[13], or print the time evolution of selected bond lengths, angles, torsions or any combination thereof.

The time scale is measured in psec. (If you cannot view the ball stick model in OPENDX[13], please convert the dx file globally to lower case.)

12.1 Command

The calling sequence is

`paw_tra` *controlfile*

where *controlfile* is the file name of the control input file for the paw_tra tool. I recommend the extension “.tcntl”.

12.2 Example for the control input file

```
!TCNTL
  !FILES
    !FILE ID='STRC'
          NAME='~/PAW/Sample/h2co.strc_out' !END
    !FILE ID='TRA'
          NAME='~/PAW/Sample/h2co_r.tra' !END
  !END
  !SEQUENCE T1[PS]=0. T2[PS]=2. DT[FS]=1. !END
  !MOVIE skip=2
    !VIEWBOX O=3*-5.  T=10. 3*0. 10. 3*0. 10. !END
    !FILE  EXT=T NAME='_movie.dx' !END
  !END
  !temperature retard[ps]=0.1
    !select atoms= 'H_1' 'H_2' !END
    !FILE  EXT=T NAME='.h.temp.dx' !END
  !end
  !spaghetti
    !FILE  EXT=T NAME='.temp.pasta' !END
  !end
```

```
  !MODE ID='H-C-BOND'
    !BOND SCALE=1.0 ATOM1='H_1' ATOM2='C_3' !END
  !END
  !MODE ID='H-ANTI-STRETCH'
    !MODE SCALE=1.0 ID='H1-C-BOND' !END
    !BOND SCALE=-1.0 ATOM1='H_2' ATOM2='C_3' !END
  !END
  !MODE ID='H-C-H-ANGLE'
    !ANGLE SCALE=1.0 ATOM1='H_1' ATOM2='C_3'
           ATOM3='H_2' !END
  !END
  !OUTPUT ID='H-ANTI-STRETCH' TYPE='VELOCITY' !END
!END
!EOB
```

12.3 Argument description for the control input file "tcntl"

12.3.1 !TCNTL

Rules: mandatory
Description:

12.3.2 !TCNTL!FILES

Rules: optional
Description: specifies the file names that deviate from standard file names

ROOT

root name. Files defined as extension will have this name combined with the extension. All files connected as default are defined as extensions.

Type: character
Rules: optional
Default: string preceding the '.tcntl' ending of the control input file

12.3.3 !TCNTL!FILES!FILE

Rules: optional, multiple
Description: define the file name:

- **PROT** protocol. Standard extension: '.tprot'
- **CNTL** control input file for the paw_tra tool. The name of this file is mandatory input and cannot be reset.
- **STRC** structure output file produced by the simulation and used as input. standard extension: '.strc_out'
- **TRA** trajectory file produced by the simulation and used as input. standard extension: '_r.tra'

ID

Identifier of the file.
Type: character
Rules: mandatory
Default: none

EXT

rootname flag: if true, the filename is interpreted as an extension of the rootname. The rootname is the filename of the wavecontrol file without the last dot and the following characters.
Type: logical
Rules: optional
Default: .false.

NAME

File name
Type: character

Rules: mandatory
Default: none

12.3.4 !TCNTL!SEQUENCE

Rules: optional
Description: specifies the beginning and ending time and the sampling frequency of the sequence to be analyzed.

T1[PS]

Beginning time of the sequence in picoseconds
Type: real
Rules: optional
Default: time of the first frame on the trajectory file

T2[PS]

ending time of the sequence in picoseconds
Type: real
Rules: optional
Default: time of the last frame on the trajectory file

DT[FS]

sampling frequency in femtoseconds
Type: real
Rules: optional
Default: spacing on the trajectory file

12.3.5 !TCNTL!MOVIE

Rules: optional
Description: writes a input file for OPENDX[13] to show the moving atoms as a ball-stick model. Bonds are drawn for every pair of atoms that are closer than 1.2 times the sum of the covalent radii.

FORMAT

descriptor for the format of the movie file to be created. can be 'DX' (OPENDX[13]), 'XYZ' (xyz format)
Type: character
Rules: optional
Default: 'DX'

SKIP

number of frames on the input trajectory files that are skipped between movie frames
Type: integer
Rules: optional
Default: 0.

12.3.6 !TCNTL!MOVIE!VIEWBOX

Rules: optional
Description: Sets the view box. Only the data within the view box are shown. The view box is defined by an origin o and three vectors t_1, t_2, t_3. The eight corners of the view box are $o, o+t_1, o+t_2, o+t_3, o+t_1+t_2, o+t_2+t_3, o+t_1+t_3, o+t_1+t_2+t_3$

O

Origin (lower left, forward point of the view box) in a.u.
Type: real
Rules: optional; incompatible with "!TCNTL!MOVIE!VIEWBOX:C"
Default: 0. 0. 0.

C

Center of viewbox
Type: real
Rules: optional; incompatible with "!TCNTL!MOVIE!VIEWBOX:O"
Default: see "!TCNTL!MOVIE!VIEWBOX:O"

T

vectors defining the edges of the view box
Type: real(3,3)
Rules: optional
Default: lattice vectors of the calculation

12.3.7 !TCNTL!MOVIE!FILE

Rules: optional
Description: specify the file where the movie plot is written. The file is then used with an appropriate visualizer.

EXT

switch to the rootname appended by the extension provided, versus using the filename (including its path).
Type: logical
Rules: optional
Default: .false.

NAME

file name
Type: character
Rules: optional
Default: the rootname appended by ".movie.dx" for !TCNTL!MOVIE:FORMAT='DX' and by ".movie.xyz" for !TCNTL!MOVIE:FORMAT='XYZ'

12.3.8 !TCNTL!MOVIE!SELECT

Rules: optional
Description: specify the atoms to shall be used. Unless specified, all atoms are plotted.

ATOMS

atom names consistent with the “strc” file. An arbitrary number of atoms can be included
Type: character
Rules: mandatory
Default: none

12.3.9 !TCNTL!CORRELATION

Rules: optional
Description: evaluates pair correlation functions for distances and angles

Type

select distance or angle correlation. (Angle correlations not yet implemented!)
Type: character(1); allowed values are ’D’ for distance and ’A’ for angle
Rules: otional
Default: ’D’

DEP

selects correlation over time as spaghetti plot or the time averaged histogram as function of angle or distance.
Type: character; allowed values are ’SPAGHETTI’ and ’HISTOGRAM’
Rules: optional

Default: 'SPAGHETTI'

12.3.10 !TCNTL!CORRELATION!FILE

Rules: optional
Description: specify the file where cssr file is written to.

EXT

switch to the rootname appended by the extension provided, versus using the filename (including its path).
Type: logical
Rules: optional
Default: .false.

NAME

file name
Type: character
Rules: optional
Default: the rootname appended by ".tra.cssr"

12.3.11 !TCNTL!CORRELATION!CENTER

Rules: optional
Description: specify the atoms to be used. Unless specified, all atoms are plotted.

12.3.12 !TCNTL!CORRELATION!CENTER!SELECT

Rules: optional
Description: specify the atoms to be used, as reference atoms. Unless specified, all atoms are SELECTED.

ATOMS

atom names consistent with the “strc” file. An arbitrary number of atoms can be included

Type: character
Rules: mandatory
Default: none

12.3.13 !TCNTL!CORRELATION!PARTNER

Rules: optional
Description: specify the atoms to be used as partner atoms. Unless specified, all atoms are selectes.

12.3.14 !TCNTL!CORRELATION!PARTNER!SELECT

Rules: optional
Description: specify the atoms to be used. Unless specified, all atoms are selected.

ATOMS

atom names consistent with the “strc” file. An arbitrary number of atoms can be included

Type: character
Rules: mandatory
Default: none

12.3.15 !TCNTL!SNAPSHOT

Rules: optional
Description: picks the unit cell and the atomic position for a given snapshot in a trajectory. The data are written to the protocoll file in a pseudo format for the structure input file and as cssr file to the file specified.

T[PSEC]

time at which the snapshot shall be taken out of the trajectory. The code selects the last timestep before the given time.

Type: real
Rules: either T[PSEC] or STEP must be selected
Default: none

STEP

time step number at which the snapshot shall be taken out of the trajectory.

Type: integer
Rules: either T[PSEC] or STEP must be selected
Default: none

12.3.16 !TCNTL!SNAPSHOT!FILE

Rules: optional
Description: specify the file where cssr file is written to.

EXT

switch to the rootname appended by the extension provided, versus using the filename (including its path).

Type: logical
Rules: optional
Default: .false.

NAME

file name

Type: character
Rules: optional
Default: the rootname appended by ".tra.cssr"

12.3.17 !TCNTL!SPAGHETTI!SELECT

Rules: optional
Description: specify the atoms to be used. Unless specified, all atoms are plotted.

ATOMS

atom names consistent with the "strc" file. An arbitrary number of atoms can be included
Type: character
Rules: mandatory
Default: none

12.3.18 !TCNTL!SPAGHETTI

Rules: optional
Description: performs a spaghetti plot ($|R(t) - R(0)|$) for a number of atoms

12.3.19 !TCNTL!SPAGHETTI!FILE

Rules: mandatory
Description: specify the file where the spaghetti plot is written. The file is formatted and to be used with an x-y plotting tool. Each line containes the time in ps and the distance in Angstrom

EXT

switch to the rootname appended by the extension provided, versus using the filename (including its path).
Type: logical
Rules: optional
Default: .false.

NAME

file name
Type: character
Rules: optional
Default: the rootname appended by ".tra.pasta"

12.3.20 !TCNTL!SPAGHETTI!SELECT

Rules: optional
Description: specify the atoms to be used. Unless specified, all atoms are plotted.

ATOMS

atom names consistent with the "strc" file. An arbitrary number of atoms can be included
Type: character
Rules: mandatory
Default: none

12.3.21 !TCNTL!TEMPERATURE

Rules: optional, multiple
Description: prints the average temperatures of the atoms and optionally a plot for the temperature versus time for individual atoms. **Warning:** The temperatures are obtained from the kinetic energy using $g = 3N$ where N is the number of atoms in the selected group. If translations and/or rotations are excluded, the temperature given here is underestimated and must be rescaled by a factor $3N/(3N-5)$ for dimers, $3N/(3N-6)$ for other molecules, $3N/(3N-3)$ for solids.

RETARD[PS]

The values are averaged over previous values with exponential decay.

$$x(t) = \frac{1}{t_0} \int_{-\infty}^{t} dt' x(t') \exp(-\frac{t - t'}{t_0})$$

This value is the decay time t_0 in pico seconds.
Type: real
Rules: optional
Default: 0.

12.3.22 !TCNTL!TEMPERATURE!SELECT

Rules: optional
Description: specify the atoms to be used. Unless specified, all atoms are plotted.

ATOMS

atom names consistent with the "strc" file. An arbitrary number of atoms can be included
Type: character
Rules: mandatory
Default: none

12.3.23 !TCNTL!TEMPERATURE!FILE

Rules: mandatory
Description: specifies the file where the plot is written. The file is formatted and to be used with an x-y plotting tool. Each line containes the time in ps and the temperature in Kelvin

EXT

switch to the rootname appended by the extension provided, versus using the filename (including its path).
Type: logical
Rules: optional

Default: .false.

NAME

file name
Type: character
Rules: optional
Default: the rootname appended by ".tra.temp"

12.3.24 !TCNTL!SOFT

Rules: optional
Description: produces the mass weighted path-length versus time

$$s(t) = \int_0^t \sqrt{\sum_i M_i \dot{R}_i^2}$$

. This mapping can be used to transform a time scale to a length scale, using the definitions of the intrinsic reaction coordinate (IRC)[50, 51]. The unit of the mass weighted path length is $\text{Å}\sqrt{\text{u}}$, where u is the "atomic mass unit", i.e. one twelfth of the mass of ^{12}C.

12.3.25 !TCNTL!SOFT!SELECT

Rules: optional
Description: specify the atoms to be used. Unless specified, all atoms are used.

ATOMS

atom names consistent with the "strc" file. An arbitrary number of atoms can be included
Type: character
Rules: mandatory
Default: none

12.3.26 !TCNTL!SOFT!FILE

Rules: optional
Description: specify the file where the mass-weighted path length versus time is written. The file is formatted and to be used with an x-y plotting tool. Each line contains the time in ps and the distance in angstrom

EXT

switch to the rootname appended by the extension provided, versus using the filename (including its path).
Type: logical
Rules: optional
Default: .false.

NAME

file name
Type: character
Rules: optional
Default: the rootname appended by ".tra.soft"

12.3.27 !TCNTL!NEIGHBORS

Rules: optional
Description: print a neighborlist at the beginning and the end of a trajectory and report all bon-breaking and bond-forming events. The report will be written into a file with extension ".tra.neighbors"

12.3.28 !TCNTL!MODE

Rules: optional
Description: defines a vibrational mode of the system such as a bond length, an angle etc. Note that all bond vectors that define bonds, angles or torsions are mapped onto the minimum image.

ID

Identifier of this mode to be used in further operations can be any string, but different from all other mode-identifierss.
Type: character
Rules: mandatory
Default: none

12.3.29 !TCNTL!MODE!BOND

Rules: optional
Description: adds a bond length as a contribution defining a mode. The bond is defined as the distance between two atoms R_1, R_2.

SCALE

Scales the parameter before adding it to the mode. (Used for example to define symmetric and antisymmetric bond stretch modes.)
Type: real
Rules: mandatory
Default: none

ATOM1

Name of the first atom R_1 defining the bond. The name is chosen consistent with the structure input file of the simulation code.
Type: character
Rules: mandatory
Default: none

ATOM2

Name of the second atom R_2 defining the bond. The name is chosen consistent with the structure input file of the simulation code.

Type: character
Rules: mandatory
Default: none

12.3.30 !TCNTL!MODE!ANGLE

Rules: optional
Description: adds a bond angle as a contribution defining a mode. The bond angle is the angle between the two bonds (R_1, R_2) and (R_2, R_3) The unit for angles is such that 2π is a full turn.

SCALE

Scales the parameter before adding it to the mode.

Type: real
Rules: mandatory
Default: none

ATOM1

Name of the first terminal atom R_1 defining the bond angle. The name is chosen consistent with the structure input file of the simulation code.

Type: character
Rules: mandatory
Default: none

ATOM2

Name of the central atom R_2 defining the bond angle. The name is chosen consistent with the structure input file of the simulation code.

Type: character
Rules: mandatory
Default: none

ATOM3

Name of the second terminal atom R_3 defining the bond angle. The name is chosen consistent with the structure input file of the simulation code.
Type: character
Rules: mandatory
Default: none

12.3.31 !TCNTL!MODE!TORSION

Rules: optional
Description: Warning! this option does not seem to work! adds a bond torsion as a contribution defining a mode. The unit for angles is such that 2π is a full turn. A torsion is defined by four atoms (R_1, R_2, R_3, R_4). The torsion about the central bond (R_2, R_3) is the angle between the two planes defined by two triples (R_1, R_2, R_3) and (R_2, R_3, R_4).

SCALE

Scales the parameter before adding it to the mode.
Type: real
Rules: mandatory
Default: none

ATOM1

Name of the atom R_1 defining the bond torsion consistent with the structure input file of the simulation code.
Type: character
Rules: mandatory
Default: none

ATOM2

Name of the atom R_2 defining the bond torsion consistent with the structure input file of the simulation code.
Type: character
Rules: mandatory
Default: none

ATOM3

Name of the atom R_3 defining the bond torsion consistent with the structure input file of the simulation code.
Type: character
Rules: mandatory
Default: none

ATOM4

Name of the atom R_4 defining the bond torsion consistent with the structure input file of the simulation code.
Type: character
Rules: mandatory
Default: none

12.3.32 !TCNTL!MODE!MODE

Rules: optional
Description: adds a predefined mode as a contribution defining a mode.

SCALE

Scales the parameter before adding it to the mode.
Type: real
Rules: mandatory
Default: none

ID

ID of the mode to be included.
Type: character
Rules: mandatory
Default: none

12.3.33 !TCNTL!OUTPUT

Rules: optional, multiple
Description: writes the time dependence of a mode to file

ID

identifier of the mode to be printed
Type: character
Rules: mandatory
Default: none

FILENAME

file name of the file to which the result is written
Type: character
Rules: mandatory
Default: none

TYPE

Defines the property to be written,

- default: the mode itself
- 'VELOCITY' the time derivative of the mode

Type: character
Rules: optional

Default: position

12.4 View data using OPENDX[13]

Start the Dataexplorer using

```
dx -image &
```

1. Select "**Load Macro**" from the "**File**" pulldown menu. In the file selector, which pops up, check that you are in the DX subdirectory of the PAW directory and click on "**Load all macros**".

2. Select "**Open**" from the "**File**" pulldown menu. In the file selector, which pops up, select the dx program "**tra.net**".

3. select "**open all control panels**" from the "**windows**" pulldown menu.

4. In the "**Main Control Panel**" that opens, select the input data file. You can do this by typing the file name into the text field of the file selector, or by pressing the button to the right of the text field and by maneuvering towards the right file in the file selector menu. After selecting the correct file, press OK.

5. Select "**Sequencer**" from the "**Execute**" Pulldown menu.

6. Use the sequencer like a VCR. (In the existing version, an error frequently occurs while reading the data file. The message says that it cannot read beyond a certain number of time slices. Remember the number, click on the right upper field of the sequencer, and type the number into the field for max. Then click on the same field of the sequencer, and continue.)

13 The Density of States Analysis Tool: paw_dos

The paw_dos tool allows one to analyze binding, both locally and energy resolved. It can tell you which states contribute mostly to a particular σ, π, δ bond. For example, you can learn whether either bonding and antibonding states are close to the Fermi level (HOMO-LUMO gap), in which case a structural deformation or a charge transfer can affect such a bond strongly. This analysis is similar in spirit to the Crystal-Orbital-Overlap-Population (COOP) analysis of R. Hoffmann. It is of tremendous value to obtain a chemical (local) description of binding in complex systems, where states are delocalized and closely spaced in energy.

13.1 Command

The calling sequence is

`paw_dos` *controlfile*

where *controlfile* is the file name of the control input file for the paw_dos tool. I recommend the extension ".dcntl".

13.2 Theoretical basis for the paw_dos tool

The density of states matrix is defined as follows

$$D_{\chi,\chi'}(\epsilon) = \sum_n \langle\chi|\Psi_n\rangle\delta(\epsilon-\epsilon_n)\langle\Psi_n|\chi'\rangle \tag{7}$$

from the Kohn-Sham orbitals Ψ_n and their one-particle energies ϵ_n. The orbitals $|\chi\rangle$ and $|\chi'\rangle$ pick out certain matrix elements of the density-of-states operator. The possible choices for the orbitals χ, χ' are described in more detail below.

The density of states is most useful to obtain a qualitative understanding of chemical binding, both locally and energy-resolved.

The band structure energy $\sum f(\epsilon_n)\epsilon_n$, where $f(\epsilon)$ is the Fermi distribution function and ϵ_n are the one-particle energies, can be decomposed into bond contributions.

$$\sum_n f(\epsilon_n)\epsilon_n = \sum_{i,j,k,l}\int d\epsilon f(\epsilon) D_{i,j}(\epsilon) O^{-1}_{j,k} H_{k,l} O^{-1}_{k,i} \tag{8}$$

where

$$H_{i,j} = \langle\chi_i| -\frac{1}{2}\nabla^2 + v|\chi_j\rangle \tag{9}$$

is the Hamilton operator and

$$O_{i,j} = \langle \chi_i | \chi_j \rangle \tag{10}$$

is the overlap matrix for a complete set of functions χ_i.

Two sum rules connect the overlap matrix element with the zeroth energy moment of the projected density of states

$$\begin{aligned}
\langle \chi_i | \chi_j \rangle &= \sum_{n,m} \langle \chi_i | \Psi_n \rangle \langle \Psi_n | \Psi_m \rangle \langle \Psi_m | \chi_j \rangle \\
&= \sum_{n} \langle \chi_i | \Psi_n \rangle \langle \Psi_n | \chi_j \rangle \\
&= \sum_{n} \int_{-\infty}^{\infty} d\epsilon \, \langle \chi_i | \Psi_n \rangle \delta(\epsilon - \epsilon_n) \langle \Psi_n | \chi_j \rangle \\
&= \int_{-\infty}^{\infty} d\epsilon \, D_{i,j}(\epsilon)
\end{aligned} \tag{11}$$

and the Hamilton matrix element with the first moment:

$$\begin{aligned}
\langle \chi_i | H | \chi_j \rangle &= \sum_{n,m} \langle \chi_i | \Psi_n \rangle \langle \Psi_n | H | \Psi_m \rangle \langle \Psi_m | \chi_j \rangle \\
&= \sum_{n} \langle \chi_i | \Psi_n \rangle \epsilon_n \langle \Psi_n | \chi_j \rangle \\
&= \sum_{n} \int_{-\infty}^{\infty} d\epsilon \, \epsilon \, \langle \chi_i | \Psi_n \rangle \delta(\epsilon - \epsilon_n) \langle \Psi_n | \chi_j \rangle \\
&= \int_{-\infty}^{\infty} d\epsilon \, \epsilon \, D_{i,j}(\epsilon)
\end{aligned} \tag{12}$$

Let us now make a simplifying assumption: assume that the local functions $|\chi_i\rangle$ have only diagonal overlap matrix elements, so that we can approximate the overlap matrix elements by $O_{i,j} = O_{i,i}\delta_{i,j}$. With this approximation, the bond contributions of Eq. 8 can be expressed by the integrals of the density of states Eq. 11 and Eq. 12.

$$\sum_n f(\epsilon_n)\epsilon_n = \sum_{i,j} \frac{\left[\int d\epsilon f(\epsilon) D_{i,j}(\epsilon)\right]\left[\int d\epsilon \, \epsilon D_{j,i}(\epsilon)\right]}{\left[\int d\epsilon D_{i,i}(\epsilon)\right]\left[\int d\epsilon D_{j,j}(\epsilon)\right]} \tag{13}$$

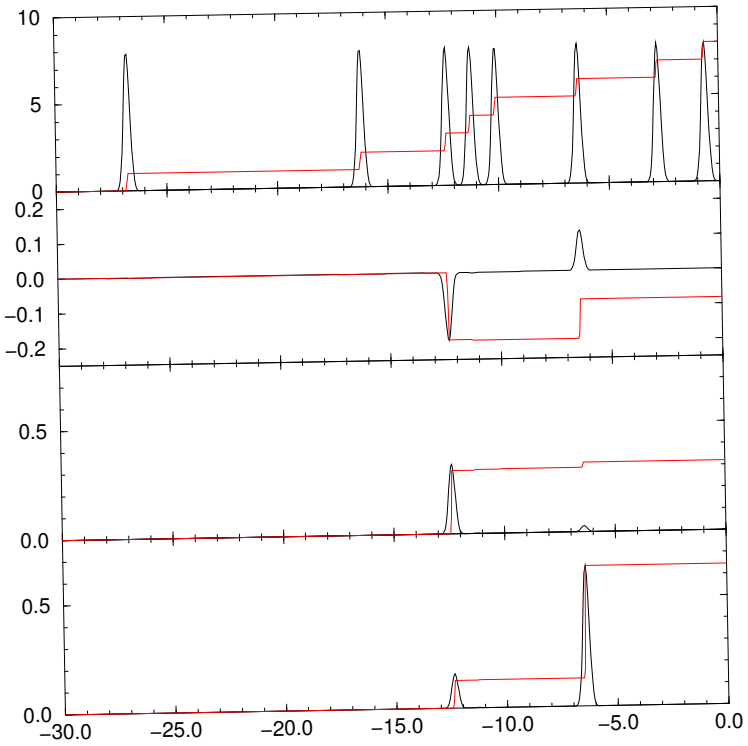


Figure 1: From top to bottom (1) total density of states of the formaldehyde molecule (H_2CO). (2) COOP between the p-orbitals lying in the plane of the molecule but perpendicular to the CO axis. (3) density of states of the p-orbital contributing to the COOP on the carbon atom, (4) density of states of the p-orbital contributing to the COOP on the oxygen atom

This expression divides the band-structure energy, the sum of occupied energy levels, into orbital contributions and contributions of orbital pairs. Roughly speaking, the orbital contribution describe the ionic character of the bond, while the orbital pairs reflect the covalent bonds.

Equation Eq. 13 allows one to estimate the covalent bond strength from the peak positions and peak strengths of the projected density of states. Note, that this analysis is only qualitative. As a qualitative tool, however, this analysis has been invaluable.

13.2.1 Orbitals

The paw_dos tool works with elementary orbitals, identified by the atom name, and an abbreviation of an angular momentum such as

s An s-orbital with angular dependence $Y_s = \sqrt{\frac{1}{4\pi}}$

px A p-orbital with angular dependence $Y_{p_x} = \sqrt{\frac{3}{4\pi}}\frac{x}{|r|}$

py A p-orbital with angular dependence $Y_{p_y} = \sqrt{\frac{3}{4\pi}}\frac{y}{|r|}$

pz A p-orbital with angular dependence $Y_{p_z} = \sqrt{\frac{3}{4\pi}}\frac{z}{|r|}$

dxy A d-orbital with angular dependence $Y_{d_{xy}} = \sqrt{\frac{60}{16\pi}}\frac{xy}{|r|^2}$

dxz A d-orbital with angular dependence $Y_{d_{xz}} = \sqrt{\frac{60}{16\pi}}\frac{xz}{|r|^2}$

dyz A d-orbital with angular dependence $Y_{d_{yz}} = \sqrt{\frac{60}{16\pi}}\frac{yz}{|r|^2}$

d3z2-r2 A d-orbital with angular dependence $Y_{d_{3z^2-r^2}} = \sqrt{\frac{5}{16\pi}}\frac{3z^2-r^2}{|r|^2}$

dx2-y2 A d-orbital with angular dependence $Y_{d_{x^2-y^2}} = \sqrt{\frac{15}{16\pi}}\frac{x^2-y^2}{|r|^2}$

sp An sp[1] orbital pointing in z-direction: $\sqrt{\frac{1}{2}}Y_s\phi_s(|r|) + \sqrt{\frac{1}{2}}Y_{p_z}\phi_p(|r|)$

sp2 An sp[2] orbital pointing in z-direction: $\sqrt{\frac{1}{3}}Y_s\phi_s(|r|) + \sqrt{\frac{2}{3}}Y_{p_z})\phi_p(|r|)$

sp3 An sp[3] orbital pointing in z-direction: $\sqrt{\frac{1}{4}}Y_s\phi_s(|r|) + \sqrt{\frac{3}{4}}Y_{p_z})\phi_p(|r|)$

where the spherical harmonics are expressed relative to the Cartesian coordinate system used in the calculation. Instead of projecting on atomic orbitals as in a Mulliken population analysis, we project here onto partial waves, that are truncated at the atomic sphere radius. The atomic sphere radius is approximately 10% larger than the covalent radius. (The factor corresponds to the ratio between the

radius of a volume-filling-sphere radius in an fcc crystal and the corresponding touching-sphere radius.)

More complex orbitals such as bond orbitals can be constructed from these elementary orbitals by using the block "!DCNTL!ORBITAL". The orbitals are named and can be referred to later by their name.

In practice we define the orbital as $\sum_i |\chi_i\rangle = \sum_j |\phi_j\rangle c_{j,i}$, and

$$\begin{aligned}\langle A\rangle &= \sum_{n,i,j}\langle\tilde{p}_i|\Psi_n\rangle f_n\langle\Psi_n|\tilde{p}_j\rangle\langle\phi_j|A|\phi_l\rangle \\ &= \sum_{n,i,j,k,l}\langle\tilde{p}_k|\Psi_n\rangle f_n\langle\Psi_n|\tilde{p}_l\rangle\frac{c_{l,A}c^*_{i,A}}{\sum_m c^*_{m,A}c_{m,A}}\langle\phi_i|A|\phi_j\rangle\frac{c_{j,B}c^*_{k,B}}{\sum_m c^*_{m,B}c_{m,B}}\end{aligned}$$

which is divided up into into a density matrix and and a matrix element according to

$$\begin{aligned}D_{B,A} &= \sum_{n,i,j,k,l}\frac{c^*_{k,B}}{\sum_m c^*_{m,B}c_{m,B}}\langle\tilde{p}_k|\Psi_n\rangle f_n\langle\Psi_n|\tilde{p}_l\rangle\frac{c_{l,A}}{\sum_m c^*_{m,A}c_{m,A}} \\ A_{A,B} &= \sum_{i,j} c^*_{i,A}\langle\phi_i|A|\phi_j\rangle c_{j,B}\end{aligned}$$

Thus if we choose a complete set of orthogonal vectors $\{c_{i,u}\}$, the total expectation value A is recovered by summing the individual contributions. For an particular atom and orbital, only the first partial wave is taken as default.

There is one important remark. The projection done here differs from the projection

$$\begin{aligned}P &= \frac{|\chi_u\rangle\langle\chi_u|}{\langle\chi_u|\chi_u\rangle} \\ &= |\phi_i\rangle\frac{\sum_{i,j} c^*_{i,u}c_{j,u}}{\sum_{k,l} c^*_{k,u}\langle\phi_k|\phi_l\rangle c_{l,u}}\langle\phi_j|\end{aligned}$$

that may be anticipated. In this case we would need a set of vectors normalized with the overlap between partial waves to obtain the complete result.

13.2.2 Matrix elements

Once the orbitals have been defined, we need to choose the matrix elements of the PDOS operator. Each matrix element is given a name, which allows us to use it in further operations. The matrix elements can be selected in two different ways:

- To select **diagonal elements** or sums of diagonal elements we use the block "!DCNTL!WEIGHT". This option allows us to plot the total density of states or the angular momentum weights of a particular orbital.
- We can also obtain **off-diagonal elements** using "!DCNTL!COOP" in order to analyze the bond order or overlap populations.

13.2.3 Operations

Projected charges, spin and eventually spin directions are printed to the protocol file. In order to access the data we have to define how they are to be represented.

- We can map the density of states on an energy grid and write the date to a file, so that they can be plotted.
- The values at a given energy can be printed to the protocol.
- The values of a given state can be written to a protocol.

13.2.4 Thermal broadening of the Density-of-States

The rationale for the thermal broadeing of the density of states is the following: An expectation value of an observable A can be written in the form

$$A(\mu) \;=\; \int_{-\infty}^{\infty} d\epsilon \; D_A(\epsilon) f_\beta(\epsilon - \mu) \tag{14}$$

where $D_A(\epsilon) = \sum_n \frac{1}{V_G} \int d^3k \;\; A_n(k) \delta(\epsilon - \epsilon_n(k))$ is a density of states weighted with the matrix element of A and $f_\beta(\epsilon - \mu)$ is the Fermi distribution function.

At zero temperature, the expectation value is simply the integral of the density of states up to the Fermi level. A similar expression can be obtained also for finite

temperatures, if the density of states is given a thermal broadening.

$$\begin{aligned} A(\mu) &= \int_{-\infty}^{\mu} d\mu' \left.\frac{dA}{d\mu}\right|_{\mu'} \\ &\overset{Eq.\,14}{=} \int_{-\infty}^{\mu} d\mu' \int_{-\infty}^{\infty} d\epsilon\; D_A(\epsilon)\partial_\mu f_\beta(\epsilon-\mu') \\ &= \int_{-\infty}^{\mu} d\epsilon \underbrace{\int_{-\infty}^{\infty} d\epsilon'\; D_A(\epsilon')\partial_\epsilon f_\beta(\epsilon'-\epsilon)}_{D_A^\beta(\epsilon)} \\ &= \int_{-\infty}^{\mu} d\epsilon\; D_A^\beta(\epsilon) \end{aligned} \tag{15}$$

The derivative of the Fermi distribution is

$$\partial_\mu f_\beta(\epsilon-\mu) = \beta\Big[\frac{1}{2}\cosh\Big(\frac{1}{2}\beta(\epsilon-\mu)\Big)\Big]^{-2} \tag{16}$$

The thermal broadeing as a second practical reason, naemly to smoothen delta-function like spikes in the density of states resulting from accidental degeneracies of energies within a tetrahedron, when the tetrahedron method is used.

If the energies are sampled (MODE='SAMPLE') the result without broadeing is a series of spikes with equal height. Here the broadening is important to produce a sensible density at the cost of loss of energy resolution. If sampling is used in a solid, the thermal broadeing shall mimick an interpolation. Then the broadeing should be comparible to the typical energy distance of neighboring k-points for in a typical band.

13.2.5 Format of the Density-of-States and Number-of-States files

The density of states file contains the following columns

1. energy in eV.
2. Density of states.
3. Density of states weighted with occupations

For non-collinear spin-polarized calculations, the contribution of the spin-$\uparrow$ electrons is drawn first, followed by that of the spin-$\downarrow$ electrons, drawn with opposite sign.

The format for the number of states files is analogous.

13.3 Example for the control input file

```
!DCNTL
  !FILES
    !FILE ID='PDOS' EXT=F NAME='~/xx.pdos' !END
    !FILE ID='PROT' EXT=F NAME='stdout'  !END
  !END
  !generic mode='tetra' DOS=T NOS=F prefix='Dos/' !END
  !GRID    DE[EV]=0.1 BROADENING[EV]=0.1 !END
  !ORBITAL NAME='SP3(C1-H2)'
    !ORB ATOM='C_1' TYPE='SP3' NNZ='H_2' FAC=1.0 !END
  !END
  !WEIGHT ID='TOTAL' TYPE='TOTAL' !END
  !WEIGHT ID='W(1)'
     !ORB NAME='SP3(C1-H2)' !END
  !END
  !WEIGHT ID='W(2)'
     !ATOM NAME='C_1' TYPE='ALL'!END
  !END
  !COOP   ID='W(3)'
    !ORB1  NAME='SP3(C1-H2)' !END
    !ORB2  ATOM='H_2' TYPE='S' !END
  !END
!END
```

13.4 Argument description for the control input file

13.4.1 !DCNTL

Rules: optional
Description: defines the operations done on the system; largely independent of the system

13.4.2 !DCNTL!GENERIC

Rules: optional
Description: defines generic settings

MODE

defines the mode for the calculation of the density of states: “SAMPLE” uses only the calculated energies on the diskrete k-point lattice. “TETRA” denotes the calculation using the tetrahedron method.[40, 41, 42].
Type: character
Rules: optional
Default: ’TETRA’

PREFIX

String will be prepended to the file names of density of states and number of states files
Type: character
Rules: optional
Default: ”

13.4.3 !DCNTL!GRID

Rules: optional

Description: defines the energy grid and thermal energy broadening. The grid extends from the minimum energy value to the minimum energy of the highest band. Read section 13.2.4 for the background of the thermal broadening. Large energy gaps are excluded from the grid.

DE[EV]

spacing of the grid points on the energy grid in eV. The value will be adjusted consistent with an integer number of grid points.

Type: real
Rules: optional
Default: 10^{-2}

BROADENING[EV]

Thermal broadening k_BT of the density of states expressed in eV.

Type: real
Rules: optional, mutually exclusive with BROADENING[K]
Default: 300 Kelvin/eV

BROADENING[K]

Thermal broadening k_BT of the density of states with T expressed in Kelvin.

Type: real
Rules: optional, mutually exclusive with BROADENING[EV]
Default: 300.

13.4.4 !DCNTL!FILES

Rules: optional
Description: Specifies the file names that deviate from the standard values

13.4.5 !DCNTL!FILES!FILE

Rules: optional, multiple
Description: Specifies one file

ID

identifier for the file; options are:

'PROT' protocol file
Standard extension:'.dprot'

'ERR' error file
Standard extension:'.derr'

'PDOS' data file produced by the simulation code. Input for paw_dos tool
Standard extension:'.pdos'

Type: character
Rules: mandatory
Default: none

NAME

filename. Can be the relative file name or an extension to the PAW "root". Standard output can be specified by NAME='stdout' and EXT=.false.
Type: character
Rules: mandatory
Default: none

EXT

.true.: NAME specifies the extension only/ .false.: full name
Type: logical
Rules: optional
Default: .false.

13.4.6 !DCNTL!ORBITAL

Rules: optional
Description: defines an orbital

NAME

names the newly defined orbital !DCNTL!ORBITAL!ORB
Type: character
Rules: mandatory
Default: none

13.4.7 !DCNTL!ORBITAL!ORB

Rules: optional, multiple
Description: select an orbital (fragment) to be added to the current orbital

- An elementary unit of a general orbital is a contribution from a single site in a specific unit cell. This contribution can be specified by an angular momentum character or as hybrid orbital. The orbital can be chosen as one of basic types TYPE in a local coordinate system.
- An existing orbital can be selected using NAME. The orbital needs to be defined by a preceeding !DCNTL!ORBITAL block. In this manner orbitals with arbitrary complexity can be composed.

Once defined, the orbital can be shifted by a lattice translation IT and multiplied with a complex factor FAC.

NAME

specifies an orbital previously specified by another block !DCNTL!ORBITAL!ORB
Type: character
Rules: optional; incompatible with atom and type
Default: none

FAC

Orbital coefficient, with which it contributes to the orbital to be built.
Type: complex (real and integer allowed)
Rules: optional
Default: 1.0

IT

translations along the three real space lattice vectors. The entire orbital is shifted by $\sum_{j=1}^{3} \vec{T}_j IT_j$.
Type: 3× integer
Rules: optional
Default: none

ATOM

atom name on which the orbital resides. (Extended notation, see section 4.2 on p. 8)
Type: character
Rules: optional; incompatible with name
Default: none

TYPE

orbital type. Can be one of the following. 'S', 'PX', 'PY', 'PZ', 'DXY', 'D3Z2-R2', 'DXZ', 'DYZ', 'DX2-Y2', 'SP', 'SP2', 'SP3'. The meaning of these orbitals is described in the preceding introduction to the paw_dos tool.
Type: character
Rules: optional; incompatible with name; mandatory if atom is specified.
Default: none

Z

vector defining the new z-direction to which TYPE refers
Type: real(3)
Rules: optional (overwritten by NNZ)
Default: 0.0,0.0,1.0

NNZ

atom name of the atom to which the vector defining the new z-direction points. (Extended notation, see section 4.2 on p. 8)
Type: character
Rules: optional; overwrites Z

Default: see Z

X

vector defining the new xz-plane together with z to which TYPE refers

Type: real(3)
Rules: optional (overwritten by NNX)
Default: 1.0,0.0,0.0

NNX

atom name of the atom to which the vector defining the new x-direction points. (Extended notation, see section 4.2 on p. 8)

Type: character
Rules: optional; overwrites X
Default: see X

13.4.8 !DCNTL!WEIGHT

Rules: optional,multiple
Description: defines orbital weights; three ways to specify the weights are possible

- specify "TYPE"; rather unspecific selections such as total density of states.
- specify one or several sub-blocks "!ATOM"; selects contributions from different atoms.
- specify one or several sub-blocks "!ORB". Most specific version, identifying individual orbitals

The selection type is not compatible with the other two possibilities.

ID

defines the name for this set of matrix elements. This ID defines the name of the output file as PREFIX//ID//'.dos', where PREFIX is defined in !DCNTL!GENERIC:PREFIX. The file format is described in section 13.2.5

Type: character
Rules: mandatory
Default: none

LEGEND

defines the legend used in the plot to annotate the dataset.

Type: character
Rules: optional
Default: none

TYPE

specifies how the weights are defined; can be

"ALL" sum over all projected density of states

"EMPTY" weight of the wave function not considered in any projection

"TOTAL" total density of states

Type: character
Rules: optional; if type is specified, no blocks !atom or !orbs must be present
Default: none

SPIN

specifies the axis $\vec{e}_S$ for the spin projection. The density of states is divided into contributions from eigenstates into spin eigenstates parallel and anti-parallel to this spin axis. (Even non-spin polarized calculations are interpreted in the data model for non-collinear calculations.) The parallel contribution is shown with positive values and the antiparallel contribution is shown with negative values.

'TOTAL' Both spin components are added up. (The second component is set to zero.)

'X' or '+X' spin axis is $\vec{e}_S = (1, 0, 0)$

'-X' spin axis is $\vec{e}_S = (-1, 0, 0)$

'Y' or '+Y' spin axis is $\vec{e}_S = (0, 1, 0)$

'-Y' spin axis is $\vec{e}_S = (0, -1, 0)$

'Z' or '+Z' spin axis is $\vec{e}_S = (0, 0, 1)$

'-Z' spin axis is $\vec{e}_S = (0, 0, -1)$

'MAIN' spin axis is the direction of the majority spin axis.

string holding three real numbers The three numbers are the three cartesian components of the spin axis $\vec{e}_S$

Type:	character
Rules:	optional
Default:	'+Z'

13.4.9 !DCNTL!WEIGHT!ATOM

Rules: optional
Description: select the angular momentum weights from a given atom.

NAME

atom name from which the contribution to this weight is selected
Type: character
Rules: mandatory
Default: none

TYPE

selects the weights from this atom. Can be

"ALL" sum over all projected density of states on this atom

"S" angular momentum weight for $\ell = 0$

"P" angular momentum weight for $\ell = 1$

"D" angular momentum weight for $\ell = 2$

"F" angular momentum weight for $\ell = 3$

or a combination of S,P,D,F, such as "SD".
Type: character
Rules: optional
Default: none

13.4.10 !DCNTL!WEIGHT!ORB

Rules: optional
Description: usage is same as !DCNTL!ORBITAL!ORB

13.4.11 !DCNTL!COOP

Rules: optional

Description: selects off-diagonal matrix elements of the density-of-states operator. The off-diagonal elements are defined by all pairs of orbitals specified in !DCNTL!COOP!ORB1 on the one hand and in !DCNTL!COOP!ORB2 on the other hand.
While it is possible, it is not recommended to use more than one pair of orbitals. The result of multiple orbitals is the sum of all coops that can be built from orbitals all pairs of orbitals, of which one is from ORB1 and the other from ORB2. The result is identical to the matrix element of an orbital obtained by summing all orbitals ORB1 and another one obtained by summing all orbitals ORB2.

ID

defines the name for this set of matrix elements. This ID defines the name of the output file as PREFIX//ID//'.dos', where PREFIX is defined in !DCNTL!GENERIC:PREFIX. The file format is described in section 13.2.5

Type: character
Rules: mandatory
Default: none

LEGEND

defines the legend used in the plot to annotate the dataset.

Type: character
Rules: optional
Default: none

SPIN

specifies the axis $\vec{e}_S$ for the spin projection. The density of states is projected onto contributions from spin eigenstates parallel (not antiparallel) to this spin axis. (Even non-spin polarized calculations are interpreted in the data model for non-collinear calculations.)

'TOTAL' Both spin components are added up. (The second component is set to zero.)

'X' or '+X' spin axis is $\vec{e}_S = (1, 0, 0)$

'-X' spin axis is $\vec{e}_S = (-1, 0, 0)$

'Y' or '+Y' spin axis is $\vec{e}_S = (0, 1, 0)$

'-Y' spin axis is $\vec{e}_S = (0, -1, 0)$

'Z' or '+Z' spin axis is $\vec{e}_S = (0, 0, 1)$

'-Z' spin axis is $\vec{e}_S = (0, 0, -1)$

'MAIN' spin axis is the direction of the majority spin axis.

string holding three real numbers The three numbers are the three cartesian components of the spin axis $\vec{e}_S$

Type: character
Rules: optional
Default: '+Z'

13.4.12 !DCNTL!COOP!ORB1

Rules: optional, multiple

Description: selects the 'left' orbital of the density of states matrix element Usage is same as !DCNTL!ORBITAL!ORB

13.4.13 !DCNTL!COOP!ORB2

Rules: optional, multiple

Description: selects the 'right' orbital of the density of states matrix element Usage is same as !DCNTL!ORBITAL!ORB

14 The Density of States Plotting Tool: paw_dosplot.x

The paw_dosplot tool prepares a batch file for the plotting software "Grace".[14]. It refers to the output files of paw_dos and arranges the data in one or more graphs stacked below each other. Within a graph several density of states data can be drawne color coded and stacked ontop of each other

Each **graph** is a rectangle containing its own x- and y-axis and it may contain several sets of Density-of-States. Graphs are stacked vertically from top to bottom. All graphs have the same range and ticmarks for the x-axis.

A **set** is the content of a **DoS-file**. The dos file has the extension '.dos'. Thus a set named 's1' is on the file 's1.dos'. The x-data are given in eV. A DoS-file contains typically four x-y datasets

1. up-spin filled-and-empty density-of-states,
2. down-spin filled-and-empty density-of-states
3. up-spin filled density-of-states,
4. down-spin filled density-of-states.

The down-spin density of states has the opposite sign, i.e. it is multiplied with -1 before it is written to file. The empty density of states is plotted in lighter color.

The sets are drawn into a graph in sequence, so that the later set covers the former. With the variable `STACK=T` the line is stacked ontop of an existing stack and it is itself added to a stack. The stack contains at least by the preceeding set.

14.1 Command

The calling sequence is

`paw_dosplot` *controlfile*

where *controlfile* is the file name of the control input file for the paw_dosplot tool. I recommend the extension ".dpcntl".

14.2 Example for the control input file

```
!DPCNTL
  EMIN[EV]=-7. EMAX[EV]=25.  YMIN=-0.0 YMAX=0.5  EZERO[EV]=14.
```

```
  !GRAPH PREFIX='DOS/'
    !SET ID='TOTAL'  COLOR='BLACK'           !END
    !SET ID='MN_EG'  COLOR='YELLOW'          !END
    !SET ID='MN_T2G' COLOR='GREEN'   STACK=T !END
    !SET ID='PR_F'   COLOR='MAGENTA' STACK=T !END
    !SET ID='PR_D'   COLOR='BLUE'    STACK=T !END
    !SET ID='O'      COLOR='RED'     STACK=T !END
  !END
  !GRAPH PREFIX='DOS/'
    !SET ID='TOTAL'  COLOR='BLACK'                  !END
    !SET ID='MN_EG'  COLOR='YELLOW'        SCALE=2. !END
    !SET ID='MN_T2G' COLOR='GREEN' STACK=T SCALE=2. !END
  !END
  !GRAPH PREFIX='DOS/'
    !SET ID='TOTAL'  COLOR='BLACK' !END
    !SET ID='MN_T2G' COLOR='GREEN' !END
  !END
!END
!EOB
```

14.3 Argument description for the control input file

14.3.1 !DPCNTL

Rules: mandatory
Description: containes direwctions on what the tool shall do

EZERO[EV]

shifts the energy axis of all graphs so the specified value (in eV) is the new energy zero. (Often this is chosen as the highest occupied state, respectively the Fermi level.) may be overwritten by EZERO in !graph and !set
Type: real
Rules: optional
Default: 0.

EMIN[EV]

lower bound (in eV) of the energy interval shown
Type: real
Rules: mandatory
Default: none

EMAX[EV]

upper bound (in eV) of the energy interval shown
Type: real
Rules: mandatory
Default: none

YMIN

lower bound in 1/eV of the density-of-states interval shown. A negative value allows to also show the opposite spin direction.
Type: real
Rules: mandatory
Default: none

YMAX

upper bound in 1/eV of the density-of-states interval shown.
Type: real
Rules: mandatory
Default: none

14.3.2 !DPCNTL!GRAPH

Rules: optional, multiple
Description: Defines a graph.

PREFIX

Default for the prefix used in this graph. The name of the density of states data produced by paw_dos are constructed by appending the prefix specified here to the set-id, and by appending the extension '.dos'.
Type: character
Rules: optional
Default: ”

SCALE

default value for the scale factor of the Density of States used in this graph
Type: real
Rules: optional
Default: 1.

EZERO[EV]

shifts the energy axis of sets in this graph so the specified value (in eV) is the new energy zero. May be overwritten by EZERO in !SET.
Type: real

Rules: optional
Default: set by !DPCNTL:XZERO

YMIN

lower bound in 1/eV of the density-of-states interval shown. A negative value allows to also show the opposite spin direction.
Type: real
Rules: optional
Default: set by !DPCNTL:YMIN

YMAX

upper bound in 1/eV of the density-of-states interval shown.
Type: real
Rules: optional
Default: set by !DPCNTL:YMAX

14.3.3 !DPCNTL!GRAPH!SET

Rules: optional, multiple
Description: Defines a set of density of states. A set contains the

1. density of states (irrespective of their occupation)
2. density of states weighted with occupation

For collinear spin polarized calculation, the spin-up density is written first followed by the spin down density.

ID

identifier for set. The name of the density of states data produced by paw_dos are constructed by appending the prefix to the set-id, and by appending the extension '.dos'.
Type: character
Rules: mandatory
Default: none

PREFIX

Prefix used for this set. The name of the density of states data produced by paw_dos are constructed by appending the prefix specified here to the set-id, and by appending the extension '.dos'.
Type: character
Rules: optional
Default: is set by !DCNTL!GRAPH:PREFIX

SCALE

scale factor of the Density of States used in this graph
Type: real
Rules: optional
Default: set by !DCNTL!GRAPH:SCALE

EZERO[EV]

shifts the energy axis for this set so the specified value (in eV) is the new energy zero.
Type: real
Rules: optional
Default: set by !DPCNTL!GRAPH:XZERO

STACK

If true, this set is added ontop of an existing stack. This redefines the new set and the stack. If STACK is non-existent or has the value .false. the stack is reset to the actual set.
Type: logical
Rules: optional
Default: F

COLOR

Fills the density of states with color. The colors to the left of the following table may be used. The corresponding colors on the right will be used for the empty density of states.

name	RGB	name	RGB
black	(000,000,000)	lblack	(150,150,150)
red	(255,000,000)	lred	(255,150,150)
green	(000,230,000)	lgreen	(150,255,150)
blue	(000,000,255)	lblue	(150,150,255)
yellow	(230,230,000)	lyellow	(255,255,150)
cyan	(000,230,230)	lcyan	(150,255,255)
magenta	(255,000,255)	lmagenta	(255,150,255)
orange	(255,165,000)	lorange	(255,210,120)
brown	(200,150,150)	lbrown	(255,180,180)
gold	(170,130,000)	lgold	(210,180,100)
darkgreen	(000,155,000)	ldarkgreen	(120,220,120)

Type: character
Rules: optional
Default: ”

15 The Band Structure Analysis Tool: paw_bands

The paw_bands tool allows one prepare the files for plotting bands structures including 2-d projected band structures.

15.1 Command

The calling sequence is

`paw_bands.x` *controlfile*

where *controlfile* is the file name of the control input file for the paw_dos tool. I recommend the extension ".bcntl".

15.2 Example for the control input file

An example for the input control file is given here

```
!BCNTL
  !INPUTFILE NAME="si.banddata" !END
  !BANDSTRUCTURE_OFF
    METHOD_DIAG=1
    EPWPSI_OFF=10.
    MODE_OFF='LINEARINTERPOLATION'
    MODE='DIAGONALISATION'

    !LINE FILE='BANDS_L_GAMMA_X.DAT'
      KVEC1=0.5 0.5 0.5
      KVEC2=0. 0. 0.
      KVECSCALE= 10.3059
      SPIN=1
      NB=20
      NK=10
      NKDIAG=10
      !FATBAND
        FILE='FATBANDS_L_GAMMA_X_SI1_IPRO_1.DAT'
        LINESPERBAND=100
        MAXWIDTH=1.0
        ATOM='SI1'
```

```
        IPRO=1
      !END
    !END
    !liNE FILE='BANDS_L_GAMMA_X.DAT'
      KVEC1=0. 0. 0.
      KVEC2=1.0 0.0 0.0
      KVECSCALE= 10.3059
      NK=11
      TAPPEND=T
    !END
    !LINE FILE='BANDS_xm.DAT' SPIN=1
      XK1= .5 0. 0. XK2= .5 .5 0. NK=101
      XKPROJECT= 0. 0. 1.  NPROJECT=11
    !END
  !END
  !PDOS
    METHOD_DIAG=1
    EPWPSI_OFF=10.
    SPACEGROUP=1
    TSHIFT=F
    NB=20
    NKDIV=7 7 7
  !END
!END
!EOB
```

15.3 Description

The two main tasks that can be accomplished with the paw_bands-tools are:

- calculation of bandstructures including fat bands and projected bandstructures from paw-calculations with either a simple linear interpolation scheme on the k-point grid or by a direct diagonalisation of the paw hamiltonian
- calculation of a new pdos-file with different k-points than the ones used in the paw calculation inclusing symmetries

For that purpose the control input file is split up into two main sections: BANDSTRUCTURE and PDOS. Which are connected by the fact that both use

the diagonalisation of the paw hamiltonian.
Non-collinear calculations are currently only partially implemented and are untested.

15.3.1 BANDSTRUCTURE

There are two modes available for the calculation of bandstructures in paw_bands:

1. linear interpolation (LINEARINTERPOLATION)

2. diagonalisation of the paw hamiltonian (DIAGONALISATION)

The first mode one uses a linear interpolation of the energy eigenvalues on the k-point grid by using the data in the pdos-file, which makes is very fast, but rather inaccurate.
The second one uses an explicit calculation of the energy eigenvalues of the paw hamiltonian at given k-points. That means that at the chosen k-points the bandstructure is represented exactly within the accuracy of the underlying paw calculation. This mode produces very nice bandstructures, but has a much higher computational demand than the interpolation method, because a large generalized eigenproblem has to be solved for every k-point. Because of the high computational demand of diagonalisation, there is also a mpi-parallelized variant, which is called ppaw_bands.x and can be compiled separately with

```
make bands_parallel
```

For computational details see the section about the theoretical background below.
Both variants write the bands are written in the format $|\vec{k}-\vec{k}_1|\ v_1(\vec{k})\ v_2(\vec{k})\ v_3(\vec{k})\ \ldots$.
$|\vec{k}-\vec{k}_1|$ is the norm of the difference k-vector between the k-point $\vec{k}$ and the first k-point $\vec{k}_1$ of this LINE-block (i.e. kvec1 resp. xk1) in absolute coordinates. $\vec{k}$ goes linearly from $\vec{k}_1$ to $\vec{k}_2$ (kvec2 resp. xk2). The values $v_i(\vec{k})$ are either the eigenvalues in eV for the i-th band or the coordinates for the fat bands if the FATBAND-option is used.
The plotting of a fat bands is done using a number (LINESPERBAND) of lines, so that the spread of the lines represents the value of the projection. That means that for every band there are LINESPERBAND columns in the file instead of one.
The calculation of fat bands is currently only implemented for the calculation of bandstructures via diagonalisation and will later be extended.
The result from the bandstructure calculation can be viewed using the program "xmgrace" with the command

`xmgrace -nxy` *bands file*

15.3.2 PDOS

This mode allows to recalculate the pdos-file which can the basis to calculate (projected) density of states and COOPs with the paw_dos-tool. The basic idea is to do a paw calculation as usual, extract the paw hamiltonian from that and diagonalize that for a different set of k-points. In order to reduce the number of k-points it can use the symmetry of the crystal, which has to be given in the control input file. Details about the symmetry can be found in the section about the theoretical background.
This mode can also be done in a mpi-parallelized fashion with ppaw_bands.x, which can be compiled separately with

```
make bands_parallel
```

In order to produce a new pdos file, the old pdos-file has to be read. The new pdos file is written by default to $ROOT.pdosout, to avoid a conflict with the existing pdos from the paw calculation.

15.3.3 Input for a fcc-lattice

Band structure file for the real-space fcc lattice. The real-space lattice constants are

$$\mathbf{T} = \begin{pmatrix} 0, \frac{1}{2}, \frac{1}{2} \\ \frac{1}{2}, 0, \frac{1}{2} \\ \frac{1}{2}, \frac{1}{2}, 0 \end{pmatrix} a \tag{17}$$

The corresponding reciprocal unit vectors are

$$\mathbf{g} = \begin{pmatrix} -1, 1, 1 \\ 1, -1, 1 \\ 1, 1, -1 \end{pmatrix} \frac{2\pi}{a} \tag{18}$$

The high symmetry points are

point	absolute coordinates $\left[\frac{2\pi}{a}\right]$	relative coordinates
Γ	(0,0,0)	(0,0,0)
X	$(1,0,0)$	$(0,\frac{1}{2},\frac{1}{2})$
W	$(1,\frac{1}{4},0)$	$(\frac{1}{8},\frac{1}{2},\frac{5}{8})$
L	$(\frac{1}{2},\frac{1}{2},\frac{1}{2})$	$(\frac{1}{2},\frac{1}{2},\frac{1}{2})$
K	$(\frac{1}{2},\frac{1}{2},0)$	$(\frac{1}{4},\frac{1}{4},\frac{1}{2})$
U	$(1,\frac{1}{2},\frac{1}{2})$	$(\frac{1}{2},\frac{3}{4},\frac{3}{4})$

(see https://en.wikipedia.org/wiki/Brillouin_zone)

The high-symmetry path drawn is

Γ	X	W	L	Γ	K=U	X
0	1	$\frac{3}{2}$	2.20711	3.07313	4.13379	4.48735

The path Gamma-K, U-X is identical ot the straight line from Gamma to an X-point of the neighboring cell.

```
!bcntl
  !BANDSTRUCTURE MODE='DIAGONALISATION'
    !line comment='gamma-X'
          file='Dos_A/bands.dat' nk=30 NB=8
          xk1=0. 0. 0. xk2=0.0 0.5 0.5  !end
    !line comment='X-W'
          file='Dos_A/bands.dat' nk=30 NB=8 tappend=T
          xk1=.0 0.5 0.5 xk2= 0.125 0.5 0.625  !end
    !line comment='W-L'
          file='Dos_A/bands.dat' nk=30 NB=8 tappend=T
          xk1=0.125 0.5 0.625 xk2= .5 .5 .5   !end
    !line comment='L-Gamma'
          file='Dos_A/bands.dat' nk=30 NB=8 tappend=T
          xk1= .5 .5 0.5 xk2=0. 0. 0.  !end
    !line comment='Gamma-K'
          file='Dos_A/bands.dat' nk=30 NB=8 tappend=T
          xk1=0. 0. 0. xk2=.25 .25  .5 !end
    !line comment='U-X'
          file='Dos_A/bands.dat' nk=30 NB=8 tappend=T
          xk1=0.5 0.75 0.75 xk2= 0.0 0.5 0.5 !end
  !end
!end
!eob
```

15.4 Argument description for the control input file

15.4.1 !BCNTL

Rules: optional
Description: defines the operations done on the system; largely independent of the system

15.4.2 !BCNTL!INPUTFILE

Rules: optional
Description: Defines banddata file.

NAME

file name of banddata file from paw calculation.
Type: character
Rules: mandatory

15.4.3 !BCNTL!BANDSTRUCTURE

Rules: optional
Description: defines the bandstructure to be computed and plotted including fatbands

mode

defines the mode of the calculation of the bandstructure. The two possible values are 'LINEARINTERPOLATION" and 'DIAGONALISATION', which are described below.
Type: character
Rules: optional
Default: LINEARINTERPOLATION

method_diag

defines the method used for the diagonalisation of the paw hamiltonian (generalized eigenvalue problem). Currently the two Lapack-functions zhegvd (method_diag=1) and zhegv (method_diag=2) can be used.
Type: integer
Rules: optional
Default: 1

EPWPSI

defines the planewave cutoff in Rydberg for the calculation of the eigenstates of the paw hamiltonian. The default is to use the same value as in the paw calculation, which is stored in the banddata-file. Details are described below.
Type: real
Rules: optional
Default: value from paw calculation (from banddata-file)

15.4.4 !BCNTL!BANDSTRUCTURE!LINE

Rules: optional,multiple
Description: defines one line seqment in k-space, i.e. a part of a bandstructure

XK1

First point of the line segment in relative coordinates of the reciprocal lattice vectors. kvec1=GBAS*xk1
Type: real(3)
Rules: mandatory (xk1 or kvec1)
Default: none

KVEC1

First point of the line segment in absolute coordinates.
Type: real(3)
Rules: mandatory (xk1 or kvec1)

Default: none

XK2

Last point of the line segment in relative coordinates of the reciprocal lattice vectors. kvec2=GBAS*xk2
Type: real(3)
Rules: mandatory (xk2 or kvec2)
Default: none

KVEC2

Last point of the line segment in absolute coordinates.
Type: real(3)
Rules: mandatory (xk2 or kvec2)
Default: none

KVECSCALE

scale factor for kvec1 and kvec2 so that they can be given in common coordinates, i.e. $\vec{k} = \frac{2\pi}{kvecscale}\vec{k}_{input}$.
Type: real
Rules: optional
Default: 2π

SPIN

spin direction. In a spin-polarized collinear calculation the parameter can have the values 1 or 2.
Type: integer
Rules: optional
Default: 1

NK

number of grid points along the line segment.
Type: integer
Rules: optional

Default: 10

NKDIAG

number of grid points along the line segment for diagonalisation. IF NKDIAG<NK the bands obtained from the diagonalisation of the paw hamiltonian will be interpolated using a 1D-FFT. NKDIAG>NK causes an error.
Type: integer
Rules: optional
Default: NK

NB

number of bands
Type: integer
Rules: optional
Default: 20

XKPROJECT

direction for the projection for 2-d band structures in relative coordinates of the reciprocal lattice vectors. Not yet implemented for METHOD=1.
Type: real(3)
Rules: optional
Default: (0.,0.,0.)

NPROJECT

number of displaced band line segments for the projection for 2-d band structures. Not yet implemented for METHOD=1.
Type: integer
Rules: optional, must be an odd number
Default: 1

FILE

Output file.
Type: character
Rules: mandatory
Default: none

TAPPEND

If TAPPEND is true, then the output data from the current line segment will be appended at the end of the file FILE. The default behaviour, i.e. when TAPPEND is false or not given, an existing file FILE will be overwritten.
Type: logical
Rules: optional
Default: F

15.4.5 !BCNTL!BANDSTRCUTURE!LINE!FATBAND

Rules: optional,multiple
Description: defines an orbital projected band structure (fat bands). This rudimentary interface will be extended in the future. Currently only projections $|\langle\tilde{p}_{IPRO}|\tilde{\psi}_i\rangle|^2$ are used, which are not comparable in magnitude between different IPRO, i.e. fatbands of different projectors should not be compared with respect to their relative magnitude.

ATOM

name of an atom.
Type: character
Rules: mandatory
Default: none

IPRO

number of projector for the calculation of the projected band structure.
Type: integer

Rules: mandatory
Default: none

LINESPERBAND

number of lines per band to draw in band structure in order to represent the value of the projection.
Type: integer
Rules: optional
Default: 10

MAXWIDTH

maximal width in eV of bands in projected band structure.
Type: real
Rules: optional
Default: 1.0

FILE

Output file.
Type: character
Rules: mandatory
Default: data are appended to previously specified file.

15.4.6 !BCNTL!PDOS

Rules: optional
Description: defines the calculation of energy eigenvalues and projections, i.e. the calculation of a new PDOS-file, which is compatible with the one produced from the main paw program.

NKDIV

fractions of reciprocal lattice vectors defining the reciprocal sub-lattice for the k-points. See also !STRUCTURE!KPOINTS!DIV
Type: integer(3)

Rules: mandatory

PDOSINFILE

filename of the PDOS-file to be read.
Type: character
Rules: optional
Default: $ROOT.pdos

PDOSOUTFILE

filename of the new PDOS-file to be written.
Type: character
Rules: optional
Default: $ROOT.pdosout

NB

number of bands. This parameter does not influence the computational complexity, because the full spectrum of the paw hamiltonian is calculated. It only determines the number of bands in the output files.
Type: integer
Type: integer
Rules: optional
Default: 20

method_diag

see !BCNTL!BANDSTRUCTURE!method_diag
Type: integer
Rules: optional
Default: 1

EPWPSI

see !BCNTL!BANDSTRUCTURE!EPWPSI
Type: real

Rules: optional
Default: value from paw calculation (from banddata-file)

SPACEGROUP

Symmetry of the crystal used for reducing the number of k-points that have to be calculated. Sequential number according to International Tables for Crystallography, Vol. A. See description below.
Type: integer
Rules: optional
Default: 1

TSHIFT

setting TSHIFT to true excludes the Gamma point from the set of k-points.
Type: logical
Rules: optional
Default: F

ISHIFT

defines a shift of the k-point grid.
Type: integer(3)
Rules: optional
Default: (0,0,0)

15.5 Theoretical background

15.6 Diagonalisation

The banddata-file contains all necessary information to build up the paw hamiltonian $\hat{\tilde{H}}$ in the plane wave basis. To calculate the eigenvalues ϵ_n and eigenstates $|\tilde{\psi}_n\rangle$, which are necessary for the projections $|\langle\tilde{p}_{IPRO}|\tilde{\psi}_i\rangle|^2$, we have to solve the

Kohn-Sham equation for the pseudo wave functions:

$$\hat{\tilde{H}}|\tilde{\psi}_n\rangle = \epsilon_n \hat{\tilde{S}}|\tilde{\psi}_n\rangle, \tag{19}$$

Here the hamilton operator is given as

$$\hat{\tilde{H}} = \frac{\hat{\tilde{p}}^2}{2} + \tilde{v} + \sum_{i,j} |\tilde{p}_i\rangle \left(\langle \phi_i | \frac{\hat{\tilde{p}}^2}{2} + v^1 | \phi_j \rangle - \langle \tilde{\phi}_i | \frac{\hat{\tilde{p}}^2}{2} + \tilde{v}^1 | \tilde{\phi}_j \rangle \right) \langle \tilde{p}_j | \tag{20}$$

and the overlap operator can be written as

$$\hat{\tilde{S}} = 1 + \sum_{i,j} |\tilde{p}_i\rangle [\langle \phi_i | \phi_j \rangle - \langle \tilde{\phi}_i | \tilde{\phi}_j \rangle] \langle \tilde{p}_j |. \tag{21}$$

(For details please refer to Peter Bloechls main publication about the paw formalism.)

For the calculation of the eigenvalues and eigenstates we build up the operators $\hat{\tilde{H}}$ and $\hat{\tilde{S}}$ is a finite plane wave basis and then solve the resulting generalized eigenvalue problem, where $\tilde{H}$ is hermitian and $\tilde{S}$ is positive definite.

Matrix dimensions The main part of the computational demand arises from the need to solve the generalized eigenvalue problem in the plane waves basis. The number of plane waves, i.e. the dimension N of the matrices of the paw hamilton operator and the overlap operator scale like $E_{max}^{\frac{3}{2}}$, where E_{max} is the plane wave energy cutoff (EPWPSI). Also the volume of the simulation cell has a strong influence. The number of plane waves grows linearly with the real space volume V, i.e. cubic with the lattice parameter.

Computational demand The computational effect for the full exact diagonalisation of the generalized eigenvalue problem goes as N^3, so that we get an overall runtime of

$$T \in \Theta(E_{max}^{4.5} V^3) \tag{22}$$

That means that reducing the plane wave cutoff can dramatically reduce the computational effort at the expense of accuracy.

Memory demand The amount of computer memory need is dominated by the storage requirements for the matrices, which is N^2, because we are using a dense storage format. So we get for the memory needed:

$$M \in \Theta(E_{max}^3 V^2). \tag{23}$$

15.7 Symmetry

To reduce the computational effort for the k-point calculations we use the symmetry of the system, so that only irreducible k-points have to be calculated. The symmetry group is determined by the parameter SPACEGROUP given in the PDOS-block, which denotes the three-dimesnional spacegroup according to International Tables for Crystallography, Vol. A.
A table of space group-definitions and the corresponding bravais lattice vectors can be found in appendix B.

16 The Setup-Analysis program: "paw_stpa.x"

Before the simulation starts atomic information is calculated and the augmentation is constructed. Earlier this was done in an external program and the information was passed in a file to the simulation code.

During this initialization step a lot of information is created that is meant to control and to monitor the setup-construction step. This information is written into a single file in an xml-like format. The tool "paw_stpa.x", which stands for "SeTuP-Analysis", allows to visualize this information.

16.1 Command

The calling sequence is

`paw_stpa.x` -s *selection* -o *outfile* *file*

where "*selection*" is a keyword selecting the information to be visualized. The parameter "*outfile*" specifies the file to which the information for the preceeding selection shall be written. The result is written to standard out, if it is omitted. The sequence "-s *selection* -o *outfile*" can be repeated several times. "*file*" is the input file, which has been written by the paw code. It has the ending "`stpforz`*Z*`.myxml`", where *Z* is the atomic number. The `paw_stpa.x` tool writes the data either in free format onto single line, or, for array data, the format is line-by-line free format "x y1 y2 ...". It can be visualized for example with the xy-plotting tool xmgrace.

`xmgrace -nxy` *out*

Here a list of the parameter "*selection*":

scattering	phase shifts
nb	nr. of wave functions
nc	nr. of core wave functions
atom.l	main angular momenta of wave functions
atom.e	energy eigenvalues in h of wave functions
atom.f	occupations of wave functions
npro	nr. of projector functions
lpro	main angular momenta of projector functions
id	id of this setup construction
z	atomic number
zv	nr. of valence electrons
aephi	all-electron partial waves
psphi	auxiliary partial waves
nlphi	node-less partial waves
qphi	core-less partial waves
pro	projector functions
aephidot	all-electron scattering partial waves
psphidot	auxiliary scattering partial waves
nlphidot	node-less scattering partial waves
pawvalencepsi	valence wave functions calculated from paw
aevalencepsi	all-electron valence wave functions
upsi	node-less wave functions
upsism	small component of node-less wave functions
aepsi	all-electron wave functions
aepsism	small component of all-electron wave functions
parms.psi.rcl	method for constructing auxiliary partial waves
parms.psi.type	method for constructing auxiliary partial waves
parms.psi.lambda	decay parameter for constructing auxiliary partial waves
parms.core.rc	cutoff radius for pseudized core
parms.core.pow	leading power at the origin of the pseudized core
parms.core.val0	value at the origin of the pseudized core
parms.pot.rc	cutoff radius for pseudized potential
parms.pot.pow	leading power at the origin of the pseudized potential
parms.pot.val0	value at the origin of the pseudized potential
parms.rcsm	narrow compensation gaussian
parms.rbox	rbox
pot	potentials [ae,ps,v(psrho)]
prog	projector functions in g-space
vaddg	vadd in g-space

17 The Structure Pre-Optimization Tool: paw_preopt

The paw_preopt tool allows to perform a fast but inaccurate structure pre-optimization based on force-field molecular dynamics. It reads the structure input file (`case.strc`) and tries to find bonds in the given structure using the covalent radii of the atoms. Then it tries to find suitable parameterizations for the atoms using the universal force field (UFF) [52]. (See also section 22.) Bonds and force field parameterizations can also be provided via the input files. Atoms to be frozen can be specified.

Using the parameterization it optimizes the geometry (currently) using the conjugate gradient line search algorithm which is also used in QM-MM coupling in the main simulation program. Atom positions after optimization are printed to the protocol file and may be used as starting point for a PAW simulation.

Atoms can be frozen during the optimization. No other constraints can be applied. Freezing can either be done via the keywords !PCNTL!FREEZEONLY and !PCNTL!MOVEONLY or via the keyword FREEZE=T in the !ATOM block of the structure input file. The latter will be ignored by the main PAW simulation program.

17.1 Command

The calling sequence is

`paw_preopt.x` *controlfile*

where *controlfile* is the file name of the control input file for the paw_preopt tool. I recommend the extension ".pcntl".

17.2 Example for the control input file

```
!PCNTL
  !FILES
    !FILE ID='xyz' NAME='optimized.xyz' !END
  !END
  !GENERIC
    TRACE=F
    TOL=0.0001
    NSTEPS=10000
  !END
```

```
  !OUTPUT
     PRINTATOMS=T
     PRINTBONDS=T
     WARNFF=T
     XYZOUT=T
  !END
  !FREEZEONLY
    COMMENT='Not used if MOVEONLY is present.'
    !ATOM NAME='MO1' !END
    !ATOM PART='FE' !END
  !END
  !MOVEONLY
     !ATOM_OFF PART='H_nh' !END
     !ATOM NAME='N_nh41' !END
     !ATOM NAME='H_nh43' !END
     !ATOM NAME='H_nh44' !END
     !ATOM NAME='H_nh45' !END
  !END
!END
!EOB
```

Please note that a keyword COMMENT is ignored by all PAW programs. Thus it can be used to add comments to the input files.

17.3 Argument description for the control input file

17.3.1 !PCNTL

Rules: mandatory
Description: defines the operations done on the system; largely independent of the system

17.3.2 !PCNTL!GENERIC

Rules: optional
Description: defines optimization parameters (and use of the trace)

TRACE

writes information on the calls of subroutines to standard output
Type: logical
Rules: optional
Default: F

TOL

tolerance of the maximal component of the force on atoms in the convergence cycles
Type: real
Rules: optional
Default: 10^{-4}

NSTEPS

maximal number of steps for the convergence. After that number the structure optimization is stopped no matter if the forces have become smaller than TOL. A statement on the convergence will be written to the protocol.
Type: integer
Rules: optional
Default: 1000

17.3.3 !PCNTL!FILES

Rules: optional
Description: specifies the file names that deviate from the standard values

17.3.4 !PCNTL!FILES!FILE

Rules: optional, multiple
Description: Specifies one file

ID

identifier for the file; options are:

'PROT' protocol file
Standard extension:'.pprot'

'STRC' The structure file as used by the simulation code. Used for the input structure.
Standard extension:'.strc'

'XYZ' data file with the resulting optimized structure
Standard extension:'.xyz'

Type: character
Rules: mandatory
Default: none

NAME

filename. Can be the relative file name or an extension to the PAW "root". Standard output can be specified by NAME='stdout' and EXT=.false.
Type: character
Rules: mandatory
Default: none

EXT

T: NAME specifies the extension only. F: full name
Type: logical
Rules: optional
Default: F (false)

17.3.5 !PCNTL!OUTPUT

Rules: optional
Description: specifies which information should be written to the protocol

PRINTATOMS

information on the used atom parameterization and freezing is written to the protocol
Type: logical
Rules: optional
Default: T

PRINTBONDS

information on the used bonds is written to the protocol
Type: logical
Rules: optional
Default: T

WARNFF

paw_preopt tries to find best-suiting force fields for the atoms. However, in some cases it cannot decide which parameterization will be the best. If T in these cases a warning will be printed to the protocol.
Type: logical
Rules: optional
Default: T

XYZOUT

should an xyz-file of the resulting structure be created?
Type: logical
Rules: optional
Default: T

17.3.6 !PCNTL!FREEZEONLY

Rules: optional, not used if !PCNTL!MOVEONLY is present
Description: sets freezing of all atoms but those specified in this block to false. May be overwritten for single atoms by !STRC!ATOM:FREEZE

NAME

specifies an atom name
Type: character
Rules: optional, multiple
Default: none

PART

Specifies a part of an atom name. All atoms containing this string are used. Case is ignored.
Type: character
Rules: optional, multiple
Default: none

17.3.7 !PCNTL!MOVEONLY

Rules: optional, overwrites !PCNTL!FREEZEONLY
Description: sets freezing of all atoms but those specified in this block to true. May be overwritten for single atoms by !STRC!ATOM:FREEZE

NAME

specifies an atom name
Type: character
Rules: optional, multiple
Default: none

PART

Specifies a part of an atom name. All atoms containing this string are used. Case is ignored.

Type:	character
Rules:	optional, multiple
Default:	none

17.4 Argument description for the structure input file

The atomic structure is read from a structure input file which has the same syntax as for the main simulation code. However, a few keywords are used which are ignored by the main simulation code. The keyword !STRC!GENERIC:LUNIT is mandatory.

17.4.1 !STRC!ATOM

Rules: mandatory,multiple

Description: usage is same as in the main simulation code. Keywords R, SP and NAME are used. Additional keywords are:

FFTYPE

Force field type used for this atom. This keyword can be used to overwrite the suggestion of the program. Possible force fields can be found in section 22 on page **??**.

Type:	character
Rules:	optional
Default:	depending on the atom type and the number of bonds to that atom

FREEZE

freeze this atom

Type:	logical
Rules:	optional
Default:	F

17.4.2 !STRC!BOND

Rules: optional,multiple
Description: Overwrite the suggestion of the program for bonds between the atoms. Not yet implemented.

18 Scripts

The scripts are small bash shell scripts that simplify recurring operations with the CP-PAW code.

The following table provides an overview:

`paw_do`	frontend for paw calculations and tools.
`paw_show`	show energies vs iterations.
`paw_collect`	collect total energies.
`paw_get`	extract the last value of certain data from simulation.
`paw_copy`	copy files into ones with a different root name.
`paw_ext_copy`	copies files into ones with a different extension
`paw_resolve`	expand text files according to replacement rules.
`paw_scan`	perform sequence paw calculation
`paw_scanlat`	scan lattice constants.

18.1 Options and *getopts*

While there are currently a few exceptions, the options are scanned by the bash command *getopts*. This implies that the options have the form of a single letter followed by an optional one-word argument. If the one-word argument is a sequence, enclose them by apostrophes. The string "word1 word2 word3" is treated as one-word argument. The option id's can be arranged in arbitrary order and they can be grouped together. The optional argument may directly follow its option id or it may be separated from it by spaces.

18.2 The Frontend Tool: "paw_do"

The tool paw_do simplifies a number of standard operations. It It assumes that there is exactly one PAW project in the current directory. It exploits the notion of standard extensions and can already process a number of commands. By changing a one-letter optional argument, various tasks can be done with the same argument.

18.2.1 Command

The calling sequence is

`paw_do` *option*

-h print this help message

-A paw_show -ce ROOT

-B paw_bands.x ROOT.bcntl; xmgrace -nxy bands.dat

-F paw_fast.x ROOT.cntl

-D paw_dos.x ROOT.dcntl; paw_dosplot.x ROOT.dpcntl; xmgrace -batch ROOT.bat

-K touch ROOT.exit (regular stop of simulation)

-P doppaw ROOT 4 (parallel ppaw_fast.x with 4 tasks)

-S paw_strc.x -c ROOT

-T paw_tra.x ROOT.tcntl

-W paw_wave.x ROOT.wcntl

-Z tail -f ROOT.prot

-0 dry-run (creates files but does not run jobs)

-v verbose

ROOT is the root name of the current project. It is extracted from a earch of all files with the required extension in the current directory. The tool assumes that the ROOT is unambiguous in the current directory.

18.3 The Energy Analysis Tool: "paw_show"

The tool paw_show allows one to inspect the protocol file. It plots the total energy, the instantaneous temperature of the atoms, the fictitious kinetic energy of the wave functions, the conserved energy versus the time step in picoseconds. This tool requires the xmgr plot library[14].

18.3.1 Command

The calling sequence is

`paw_show` *option rootname file*

where *rootname* is the name of the protocol file without the ".prot" ending and *option* is one of the following:

-f fictitious kinetic energy of the wave functions

-e the static total energy

-c the conserved energy

-t temperature of the atoms in Kelvin

-ar friction parameter for the atoms

-ap friction parameter for the wave functions

-s use time steps instead of time in ps

-u unit select unit (h, ry, ev, kj/mol, kcal/mol)

-h prints information about usage of the tool

? prints information about usage of the tool

The optional parameter *file* the file to which the data are written. If *file* is absent, the data are viewed.

18.4 Copy a project: "paw_copy"

paw_copy makes a copy of a cp-paw project with a new rootname.

18.4.1 Command

The calling sequence is

`paw_copy` *root1 root2*

where *root1* is the root name of the project to be copied and root2 is the target foot name.

Specifically, it takes all files beginning with *root1* in the current directory and copies the files with the same name but *root1* replaced by *root2*.

18.5 change the extension of a et of files: “paw_ext_copy”

paw_ext_copy makes a copies of a set of files with a given ending (extension) into files with the same beginning, but a different extension.

18.5.1 Command

The calling sequence is

`paw_ext_copy` *ext1 ext2*

where *ext1* is the extension of the files to be copied and *ext2* is the target extension.

Specifically, it takes all files ending with *ext1* in the current directory and copies the files with the same name but *ext1* replaced by *ext2*.

18.6 Collect energies “paw_collect”

paw_collect collects the last total energies from all protocoll files in the current directory.

18.6.1 Command

The calling sequence is

`paw_collect`

18.7 Collect energies “paw_get”

paw_get extracts the final result of various energies from the protocoll file.

18.7.1 Command

The calling sequence is

`paw_get` *options ROOTNAME*

Options:

-h: print help message

-w whatid: Specify the quantity to be extracted. whatid may be one of

- **etot:** total energy
- **gap:** fundamental band gap
- **homo:** energy of the highest occupied state
- **lumo:** energy of the lowest unoccupied state
- **efermi:** energy of the fermi level
- **volume:** unit cell volume
- **epw:** wave function plane wave cutoff
- **epwrho:** density plane wave cutoff
- **nkpt:** number of k-points

-u: unit (1, ev, kj/mol, kcal/mol , H, angstrom$\hat{3}$, aa$\hat{3}$, cubangstrom, a0$\hat{3}$, abohr$\hat{3}$, cubabohr) case insensitive. Unit must be consistsnet with quantity to be retrieved

-n: prints the result as number

-t: prints the result as text

-v: verbose

The command scans the file *rootname*`.prot` and prints the last occurance of the corresponding variable

Remark: -gap does not work with variable occupations and will provide an error message. -lumo and -homo will report the chemical potential with variable occupations.

18.8 Collect energies "paw_resolve"

paw_resolve constructs *outfile* from *infile* by replacing specific placeholders either by a specific value or by the contents of a specified file.

It is typically used within shell scripts to scan certain variable sets in the structure or control files of a PAW calculation. For example a string "EPW=@X1@" will be changed with the replacement rule "X1=30" to "EPW=30". Several such

rules can be specified.

There are also replacement rules that insert complete files. Here one introduces a single line containing the id of the file-replacement rule. This line will be replaced by the content of the file. This allows one to exchange for example the setup information consistently in a group of projects.

18.8.1 Command

The calling sequence is

`paw_resolve` *options infile* >*outfile*

Options:

-h: print help message

-p prefix: prefix to be added to the filename in @file@

-r id=value: rule to convert @id@ into value

-f id=file: rule to replace lines containing @id@ by the contents of file

18.9 Scan parameters "paw_scan"

A typical application is to scan a certain parameter value

18.9.1 Command

The calling sequence is

`paw_scan` *options* SRCROOT

Options:

-h: print help message

-r "name value1 value2 ...": replacement rule

-w *whatid*: *whatid*=run: perform CP-PAW simulation. Requires .cntl and .strc files

***whatid*=fast:** perform CP-PAW simulation Requires .cntl and .strc files

***whatid*=dos:** make density-of-states. Requires .dcntl file.

-n *nnodes*: submit parallel job with the number of processes specified by *nnodes*.

-m *hostname*: limit execution to the specified host *hostname*

-0: dry-run (creates files but does not run jobs)

-b *directory*: directory containing the paw executables

-v: verbose

the argument SRCROOT is the rootname of the source files and the common part of the root name for all projects. It must contain the a relative or absolute path (i.e. one "/").. The root names of individual projects will be formed as TARGET-ROOT=$SRCROOT_$VALUE/$PROJECT_$VALUE. where PROJECT is $SRCROOT stripped of the directory part and VALUE is assigned by the rule specified by option -r.
If the target directory does not contain a restart file, a source restart file $SRCROOT.rstrt, if present and needed, will be copied into the target directory.
Example: paw_scan -w run -r "EPW 20 30 40" ./si2

Preconditions:

- projectname.cntl
- projectname.strc

18.10 Scan lattice constants "paw_scanlat"

A typical application is to calculate the energy versus volume dependence, which yields the equilibrium lattice constant and the bulk modulus.

18.10.1 Command

The calling sequence is

`paw_scanlat` *options*

Options:

-h: print help message

-p *rootname*: rootname of the project to be considered (mandatory)

-l *list*: list of percent changes of the lattice constant (enter in apostrophes "...")

-b *directory*: directory containing the paw executables (binaries)

-e *executable*: name of the executable

-0: dry run

Preconditions:

- projectname.cntl
- projectname.strc

18.11 Murnaghan fit "paw_murnagan.x"

`paw_murnaghan.x` extracts the minimum energy, equilibrium volume, bulk modulus and its pressure derivative from a energy-vs-volume data sequence. The equation of state proposed by Murnaghan[53] is based on the assumption that the bulk modulus depends linearly on pressure.

The calling sequence is

```
paw_murnaghan.x <in.dat
```

where `in.dat` is an input file holding the energy-versus-volume data. The first column of `in.dat` holds the volumes and the second the corresponding energies, both given in Hartree atomic units. In addition a file "`murn.dat`" is produced holding the interpolated data covering the range of the input volumes +10%.

18.12 "paw_waittilempty"

paw_waittillempty is a helper script for self-written shell scripts.

If one submits several calculations via a shell script to a computer, the computer may run out of memory and become very slow. This script introduces a breakpoint in your own shell script that is passed only when your computer has resources available.

18.12.1 Command

The calling sequence is

`paw_waittillempty` *options*

Options:

-h: print help message

-n *njobx*: paw_waittillempty pauses until the number of paw-related calculations on the computer is lower than *njobx*.

18.13 PAW-snapshot "paw_snapshot"

`paw_snapshot` is a shell script that collects all raw files of the PAW disttribution, i.e. source code, configuration files, parameter files etc. in a compresses tar file paw_snapshot.tgz

The calling sequence is

`paw_snapshot` [`-i` *dir*] [`-o` *snapshot*]

where

- -i *dir* is the path of the PAW distribution. The default is the directory specified by the environment variable `PAWDIR`.

- -o `snapshot` is a compressed tar file with the snapshot. The default is `$PAWDIR/paw_snapshot.tgz`. Write permission for the file `paw_snapshot.tgz` in the directory $PAWDIR is required.

The program creates a directory `/tmp/paw_snapshot`*YYMMDDHHMMSS*. This file is not automatically removed for security reasons. It may be removed however after operation. Write permission is required.

19 Specifications of formatted and unformatted output data files

19.1 Protocoll

One-particle energies: The eigenvalues of the Hamiltonian in the basis of the wave function are printed, unless dynamical occupations are selected (i.e. `!CONTROL!MERMIN`). In this latter case, the diagonal elements of the Hamiltonian are printed. The one-particle energies are printed in eV. The occupations are printed without the geometric weight, but include spin degeneracy, so that for a non-spin polarized calculation the maximum allowed occupation is two.

19.2 waveplot

The file waveplot is used to provide a wave function or a density on a real space grid. From there it has to be converted into a format ready for visualization, which can be done using the "waveplot" tool. A waveplot file is created by selecting "!CNTL!ANALYSE!WAVE" in the control input file.

The file is written as follows:

```
CHARACTER(*) :: TITLE     ! user defined comment
INTEGER(4)   :: NAT       ! number of atoms
REAL(8)      :: RBAS(3,3) ! lattice vectors (xyzxyzxyz)
REAL(8)      :: POS(3,NAT)! atomic positions
REAL(8)      :: Z(NAT)    ! atomic numbers
REAL(8)      :: Q(NAT)    ! point charges
CHARACTER(32):: NAME(NAT) ! atom name (see file strc_out)
                          ! number of grid points along
INTEGER(4)   :: NR1       !        first direction
INTEGER(4)   :: NR2       !        second
INTEGER(4)   :: NR3       !        third
REAL(8)      :: WAVE(NR1,NR2,NR3) ! density or wave function
                                  ! on the real space grid
WRITE(UNIT)'WAVEPLOT',LEN(TITLE)
WRITE(UNIT)TITLE
WRITE(UNIT)RBAS,NAT
WRITE(UNIT)NR1,NR2,NR3
WRITE(UNIT)NAME
WRITE(UNIT)Z
```

```
WRITE(UNIT)POS
WRITE(UNIT)Q
WRITE(UNIT)WAVE
WRITE(UNIT)'END OF FILE'
```

The coordinates of the density or wave with (i_1, i_2, i_3) are

$$x_i = \sum_{i=1}^{3} \mathrm{RBAS}_{i,j} \frac{i_i - 1}{\mathrm{NR}i} \tag{24}$$

19.3 Projected density of states files

```
INTEGER(4)  :: NAT   !#(ATOMS)
INTEGER(4)  :: NSP   !#(ATOM TYPES)
INTEGER(4)  :: NKPT  !#(K-POINTS)
INTEGER(4)  :: NSPIN !#(SPINS)
INTEGER(4)  :: NDIM  !#(SPINOR COMPONENTS)
INTEGER(4)  :: NPRO  !#(PROJECTIONS PER STATE)
INTEGER(4)  :: LNXX  !MAX#(PARTIAL WAVES PER ATOM (L,N))
INTEGER(4)  :: LNX(NSP)       !#(PARTIAL WAVES PER ATOM (L,N))
INTEGER(4)  :: LOX(LNXX,NSP) !MAIN QUANTUM NUMBER
INTEGER(4)  :: ISPECIES(NAT) !ATOM TYPE
REAL(8)     :: RBAS(3,3)     !LATTICE VECTORS
REAL(8)     :: R0(3,NAT)     !ATOMIC POSITIONS
INTEGER(4)  :: IZ            !ATOMIC NUMBER
REAL(8)     :: RAD           !``ASA RADIUS''
REAL(8)     :: VAL(LNXX)     ! AEPHI(RAD)
REAL(8)     :: DER(LNXX)     ! DAEPHI/DR(RAD)
REAL(8)     :: OV(LNXX,LNXX) ! <AEPHI|AEPHI> WITHIN RAD
REAL(8)     :: XK(3,NKPT)    ! K-POINT IN RELATIVE COORDINATES
INTEGER(4)  :: NB            ! #(BANDS)
REAL(8)     :: EIG(NB,NSPIN,NKPT)               ! EIGENVALUES
COMPLEX(8)  :: PROJ(NDIM,NPRO,NB,NSPIN,NKPT)    ! PROJECTIONS
!************************************************************
WRITE(NFIL)NAT,NSP,NKPT,NSPIN,NDIM,NPRO,LNXX
WRITE(NFIL)LNX(:),LOX(:,:),ISPECIES(:)
WRITE(NFIL)RBAS(:,:),R0(:,:)
DO ISP=1,NSP
   WRITE(NFIL)IZ,RAD,VAL(1:LNX(ISP)),DER(1:LNX(ISP)) &
&                                   ,OV(1:LNX(ISP),1:LNX(ISP))
ENDDO
```

```
DO IKPT=1,NKPT
  DO ISPIN=1,NSPIN
    WRITE(NFIL)XK(:,IKPT),NB
    DO IB=1,NB
      WRITE(NFIL)EIG,PROJ(:,:)
    ENDDO
  ENDDO
ENDDO
```

19.4 Trajectory files

Trajectory files are used to monitor variables step by step.

The file is written as follows:

```
INTEGER(4)  :: ISTEP1    !
INTEGER(4)  :: ISTEP2
INTEGER(4)  :: LEN
REAL(8)     :: TIME(ISTEP1:ISTEP2)
REAL(8)     :: ARRAY(LEN,ISTEP1:ISTEP2)
!
DO ISTEP=ISTEP1,ISTEP2
  WRITE(UNIT)ISTEP,TIME(ISTEP),LEN,ARRAY(:,ISTEP)
ENDDO
```

Here "istep" is the number of the time step, consistent with the protocol file.

19.4.1 Position Trajectory

In the position trajectory "*root*_r.tra". The length of each item is $LEN = 9+8*N$, where N is the number of atoms. The data on "ARRAY" are arranged as follows:

1. $\mathbf{T}$: lattice vectors (3×3 data)
2. $\vec{R}$: atomic positions ($3 \times N$ data)
3. Q: point charges (N) as obtained from Gaussian fit (N data)
4. $q, \vec{m}$ Charges and moments as obtained from integrating the one-center density over a "atomic sphere" radius. ($4 \times N$ data)

20 Units and constants

The program works in Cartesian coordinates using Hartree atomic units

$$\hbar = e = m_e = 4\pi\epsilon_0 = 1\,. \tag{25}$$

Angles are given in radian (2π radian $= 360$ deg).

The following list is produced by the CONSTANTS object. The data are based on a values recommended by The Commitee on Data for Science and Technology (CODATA) [12].

```
UNITS AND CONSTANTS IN ATOMIC HARTREE UNITS:
--------------------------------------------
     NAME           VALUE        :DESCRIPTION
PI.............=    3.141593     :PI
HBAR...........=    1.000000     :A.U. FOR ANGULAR MOMENTUM
E..............=    1.000000     :ELEMENTARY CHARGE
ME.............=    1.000000     :ELECTRON MASS
EPSILON0.......=    7.957747E-02 :PERMITIVITY OF VACUUM
ALPHA..........=    7.297353E-03 :FINE STRUCTURE CONSTANT
ABOHR..........=    1.000000     :BOHR RADIUS
TAU0...........=    1.000000     :A.U. FOR TIME
HARTREE........=    1.000000     :HARTREE ENERGY UNIT
C..............=  137.035989     :SPEED OF LIGHT
MU0............=    6.691764E-04 :PERMEABILITY OF VACCUM
BOHRMAGNETON...=    0.500000     :BOHR MAGNETON
NUCLEARMAGNETON=    2.723085E-04 :NUCLEAR MAGNETON
GE.............=    2.002319     :ELECRON GYROMAGNETIC RATIO
KG.............=    1.097768E+30 :KILOGRAMM
SECOND.........=    4.134137E+16 :SECOND
METER..........=    1.889726E+10 :METER
AMPERE.........=  150.974819     :AMPERE
MOL............=    6.022137E+23 :MOLE
KB.............=    3.166679E-06 :BOLTZMANN CONSTANT IN A.U.
NEWTON.........=    1.213779E+07 :NEWTON
JOULE..........=    2.293710E+17 :JOULE
COULOMB........=    6.241506E+18 :COULOMB
VOLT...........=    3.674931E-02 :VOLT
TESLA..........=    4.254381E-06 :TESLA
GAUSS..........=    4.254381E-10 :GAUSS
RY.............=    0.500000     :RYDBERG ENERGY UNIT
```

```
EV.............=    3.674931E-02 :ELECTRON VOLT
ANGSTROM.......=    1.889726     :ANGSTROM
KJ/MOL.........=    3.808798E-04 :KILOJOULE PER MOLE
KCAL/MOL.......=    1.593601E-03 :KILOCALORIE PER MOLE
U..............=    1.822889E+03 :atomic MASS UNIT
DEBYE..........=    0.393430     :DEBYE


CONVERSION FACTORS AN CONSTANTS:
--------------------------------
--------- ATOMIC UNITS ---------------------------------
QE.....................=    1.000000     :A.U.
HBAR...................=    1.000000     :A.U.
ME.....................=    1.000000     :A.U.
HARTREE................=    1.000000     :A.U.
ABOHR..................=    1.000000     :A.U.
1/ALPHA................=  137.035989     :
EPSILON0...............=    7.957747E-02 :
MU0....................=    6.691764E-04 :
C......................=  137.035989     :ABOHR/TAU0
KB.....................=    3.166679E-06 :HARTREE
--------- CONVERSION FROM SI TO ATOMIC UNITS -----------
MOL....................=    6.022137E+23 :
METER..................=    1.889726E+10 :ABOHR
SECOND.................=    4.134137E+16 :TAU0
JOULE..................=    2.293710E+17 :HARTREE
KG.....................=    1.097768E+30 :ME
C......................=    2.997925E+08 :METER/SECOND
--------- CONVERSION FROM ATOMIC TO SI UNITS -----------
ABOHR..................=    5.291772E-11 :METER
TAU0...................=    2.418884E-17 :SECOND
ME.....................=    9.109390E-31 :KG
E......................=    1.602177E-19 :COULOMB
HBAR...................=    1.054573E-34 :JOULE*SECOND
--------- LENGTH CONVERSIONS ---------------------------
ANGSTROM...............=    1.889726     :ABOHR
ABOHR..................=    0.529177     :ANGSTROM
ABOHR..................=    5.291772E-11 :METER
METER..................=    1.889726E+10 :ABOHR
--------- TIME CONVERSIONS -----------------------------
```

```
TAU0...................=    2.418884E-17 :SECOND
TAU0...................=    2.418884E-02 :FEMTO SECOND
PICO SECOND............=    4.134137E+04 :TAU0
--------- ENERGY CONVERSIONS ---------------------------
273.15 KB..............=    2.353727E-02 :EV
EV.....................=   96.485309     :KJ/MOL
HARTREE................=    2.625500E+03 :KJ/MOL
EV.....................=   23.060542     :KCAL/MOL
HARTREE................=  627.509556     :KCAL/MOL
KJ/MOL.................=    1.036427E-02 :EV
KCAL/MOL...............=    4.336411E-02 :EV
HARTREE................=   27.211396     :EV
RY.....................=    0.500000     :HARTREE
--------- MASS CONVERSIONS ----------------------------
U......................=    1.660540E-27 :KG
U......................=    1.822889E+03 :ME
--------- CHARGE CONVERSIONS --------------------------
QE.....................=    1.602177E-19 :COULOMB
COULOMB................=    6.241506E+18 :QE
--------- OTHER CONVERSIONS ---------------------------
DEBYE..................=    0.393430     :E*ABOHR
E*ABOHR................=    2.541748     :DEBYE
E_ZPV/T[PSEC]..........=    7.599149E-05 :HARTREE
E_ZPV/T[PSEC]..........=    2.067835E-03 :EV
E_ZPE/(1/LAMBDA)[CM**-1=    6.199212E-05 :EV
E_ZPE/(1/LAMBBDA)[CM**-=    2.278168E-06 :HARTREE
(1/LAMBDA).............=   33.356410     :CM**-1/T[PSEC]

Scale factors :
---------------
FEMTO..........=    1.000000E-15 :FEMTO=1.D-15
PICO...........=    1.000000E-12 :PICO=1.D-12
NANO...........=    1.000000E-09 :NANO=1.D-9
MICRO..........=    1.000000E-06 :MICRO=1.D-6
MILLI..........=    1.000000E-03 :MILLI=1.D-3
KILO...........=    1.000000E+03 :KILO=1.D+3
MEGA...........=    1.000000E+06 :MEGA=1.D+6
GIGA...........=    1.000000E+09 :GIGA=1.D+9
TERA...........=    1.000000E+12 :TERA=1.D+12
PETA...........=    1.000000E+15 :PETA=1.D+15
```

20.1 Vibrations

Vibrational freqencies and optical absorption energies are frequenty measured via the frequency or wave-length of the absorbed light. The frequency or energy is measured in wave-numbers $\bar{\nu} = 1/L$ defined as the inverse of the wavelength L. (The wave number is the number of times an oscillation of a light wave fitswithin a given length interval.) It is measured in units of cm^{-1}. Here we provide the conversion of wave numbers into energies, frequencies, etc.

First we convert the wave number into a frequency $\omega = 2\pi/T$ where T is the time of a full oscillation.

We use the dispersion relation of light $\omega = c|k|$, where c is the speed of light. $\omega = \frac{2\pi}{T}$ is the frequency and $|\vec{k}| = \frac{2\pi}{L}$ is the absolute value of the wave vector. T is the time for one full period at a given position and L is the length for one full period at a given time.

From the dispersion relation we calculate the the frequency $\omega = 2\pi/T$ and the

$$
\begin{aligned}
|k| &= \frac{2\pi}{\bar{\nu}} && (26)\\
L &= \frac{1}{\bar{\nu}} && (27)\\
\omega &= c|k| = c2\pi/L = 2\pi c\bar{\nu} && (28)\\
T &= \frac{2\pi}{\omega} = \frac{1}{c\bar{\nu}} && (29)
\end{aligned}
$$

The connection to the energy is given by $E = \hbar\omega$, where E is the energy of the absorbed or emmited photon with frequency ω. It is the energy difference between the energy levels of the corresponding transition.

$$
E = \hbar\omega = 2\pi\hbar c\bar{\nu} \qquad (30)
$$

Finally we would like to know the force constant C for a harmonic oscillator producing these level spacings.

$$
\begin{aligned}
M\ddot{x} &= -CX \qquad \Rightarrow \qquad \omega = \sqrt{\frac{C}{M}}\\
C &= M\omega^2 = M4\pi^2c^2\bar{\nu}^2 && (31)
\end{aligned}
$$

Let us now evaluate the conversion factors

$$\begin{aligned}
T[ps] &\overset{\text{Eq. 29}}{=} \frac{1}{c[a_0/\tau_0]\bar{\nu}[cm^{-1}]}\frac{\tau_0}{ps}\cdot\frac{cm}{a_0} \\
&= \left(\frac{1}{137.035989}\right)\cdot\left(\frac{1}{10^{-12}\cdot 4.134137\times 10^{16}}\right)\cdot\left(10^{-2}\cdot 1.889726\times 10^{10}\right)\frac{1}{\bar{\nu}[cm^{-1}]} \\
&= \frac{1.889726}{1.37035989\cdot 4.134137}\cdot 10^{-2+12-16-2+10}\frac{1}{\bar{\nu}[cm^{-1}]} \\
&= 0.3335640485\times 10^2\frac{1}{\bar{\nu}[cm^{-1}]} = 33.35640485\frac{1}{\bar{\nu}[cm^{-1}]} \\
T[a.u.] &= 1.379\times 10^6\frac{1}{\bar{\nu}[cm^{-1}]}
\end{aligned}$$

Note that we can read $T[ps]$ as T/ps if we ps is the value of a pico-second in the unit system of choice.

The energy is obtained from Eq. 30 as

$$\begin{aligned}
E[a.u.] &= \hbar\omega = 2\pi\hbar[a.u.]c[a.u.]\frac{a_0}{m}10^2\bar{\nu}[cm^{-1}] \\
&= 2\pi\times 137.035989\frac{1}{1.889726\times 10^{10}}10^2\bar{\nu}[cm^{-1}] \\
&= 2\pi\times 137.035989\frac{1}{1.889726\times 10^{10}}10^2\bar{\nu}[cm^{-1}] \\
&= 4.556335\times 10^{-6}\bar{\nu}[cm^{-1}] \\
E[eV] &= 1.23984242\times 10^{-4}\bar{\nu}[cm^{-1}]
\end{aligned}$$

As crosscheck we may use that the ground state of the hydrogen atom lies at $-\frac{1}{2}$ H=-13.6 eV corresponds to $\bar{\nu}$=-109678 cm^{-1}. This number is the Rydberg constant. (see Demtröder, Experimentalphysik 3; Springer Verlag p.102)

Here i summarize the main vibrational modes

Vibr. Mode	$\bar{\nu}[cm^{-1}]$
H-H Stretch	≈ 4000
C-H Stretch	2800-3999
C-H Bend	1400-1500
C-C Stretch	1500-1750

The hydrogen stretch vibration has a period of about 350 τ_0 or 8.3 fs. The optical phonon in bulk silicon has a frequency of about 14 THz, which corresponds to $\bar{\nu} = 450 - 500$ cm^{-1}.

20.2 Magnetic hyperfine parameters

An electronic magnetization produces a magnetic field $\vec{B}^N$, the hyperfine field, at the nucleus, which results in splittings of the nuclear energy levels. In electron spin resonance (ESR), it is the electron spin that is flipped by a microwave radiation, while the nuclear spin remains unaffected.

The energy of a nucleus in the magnetic field is

$$E = \vec{m}^N \vec{B}^N$$

where m^N is the nuclear magnetic moment. The nuclear magnetic moment is proportional to the nuclear spin $\vec{S}^N$, where the proportionality constant is the gyromagnetic ratio, defined as the ratio of magnetic moment and spin, i.e. $\vec{m}_N = \gamma_N \vec{S}_N$.

The nuclear magnetic moment is typically given units of nuclear magnetons $\mu_N = (m_e/m_p)\mu_B$, where m_e is the electron mass, m_p is the proton mass, and $\mu_B = \frac{e\hbar}{2m_e}$ is the Bohr magneton. Values for the magnetic moment and the nuclear spin of the isotopes can be obtained from the "Green Book"[54].

The Hamiltonian of the electron-nucleus interaction can be cast in the form

$$\hat{H} = \hat{\vec{S}}_N \mathbf{A} \hat{\vec{S}}_e$$

where $\mathbf{A}$ is the hyperfine tensor and $\hat{\vec{S}}_N$ is the nuclear spin operator and $\hat{\vec{S}}_e$ is the electron-spin operator. The isotropic interaction, also called Fermi-contact interaction, is defined as $a = \frac{1}{3}\mathrm{Tr}[\mathbf{A}]$. The anisotropic hyperfine interaction is governed by $\mathbf{A} - a\mathbf{1}$.

When the electron spin is flipped by an electronic transition, the transition energy is $\hbar\omega = \hbar\omega_0 + 2m^N B^N$, where $\hbar\omega_0$ is, for example, the Zeeman splitting. For different spin quantum numbers S_z^N, we obtain transitions

$$\hbar\omega(S_z^N) = \hbar\omega_0 + 2\frac{m^N}{|S^N|} B^N S_z^N \tag{32}$$

from which follows the splitting of neighboring levels

$$\Delta E = \Delta\hbar\omega(S_z^N) = 2\hbar\frac{m^N}{S^N} B^N \tag{33}$$

with S^N being the nuclear spin with values $0, 1/2\hbar, \hbar, ..$, depending on the nucleus.

In the literature the hyperfine splitting is often converted into a frequency ν using the relation $\Delta E = h\nu$. The frequency is expressed typically in MHz. Another term used for

MHz is Mc/s, i.e. megacycles per second.

$$\begin{aligned}\nu &= \frac{\Delta E}{2\pi\hbar} = \frac{m^N B^N}{\pi S^N} \\ &= \frac{m^N[\mu_N] B^N[\mathrm{T}]}{S^N[\hbar]} \times \frac{\mu_N \mathrm{T}}{\pi\hbar \mathrm{MHz}}\ \mathrm{MHz} \\ &= \frac{m^N[\mu_N] B^N[\mathrm{T}]}{S^N[\hbar]} 15.24518048\ \mathrm{MHz}\end{aligned}$$

The frequency is often converted in wavenumbers using the relation $\frac{1}{l} = \frac{\nu}{c}$.

$$\begin{aligned}\frac{1}{l} &= \frac{m^N[\mu_N] B^N[\mathrm{T}]}{S^N[\hbar]} \times \frac{\mu_N \mathrm{T} 10^{-2}\mathrm{m}}{\pi c\hbar}\ \mathrm{cm}^{-1} \\ &= \frac{m^N[\mu_N] B^N[T]}{S^N[\hbar]} \times 5.085135181 \times 10^{-4}\ \mathrm{cm}^{-1}\end{aligned}$$

The energy of the magnetic hyperfine splitting is often converted into a effective magnetic field B^e acting on the electrons via

$$\Delta E = g_e \mu_B B^e$$

where μ_B is the Bohr magneton, g_e is the g-factor of the free electron. While the g-factor of the electron is slightly larger than 2, its value is set exactly to 2. The term $g_e\mu_B$ is twice the magnetic moment of the electron.

$$\begin{aligned}B^e &= \frac{\Delta E}{g_e \mu_B} = \frac{2\hbar m_N B_N}{S_N g_e \mu_B} \\ &= \frac{m_N[\mu_N] B_N[\mathrm{T}]}{S_N[\hbar]} \times \frac{2\hbar\mu_N T}{\hbar 2\mu_B \mathrm{T}}\ \mathrm{T} \\ &= \frac{m_N[\mu_N] B_N[\mathrm{T}]}{S_N[\hbar]} \times 0.544617\ \mathrm{mT}\end{aligned}$$

The magnetic field is either given in milliTesla, in Oersted or in Gauss (1 Oe=1 gauss=10^{-4} T).

21 Periodic Table

Atomic masses according to IUPAC 1985 (Pure Appl. Chem. 1985, 58, 1677-1692). Length units are in atomic units. "R(COV)" is the covalent radius. "R(ASA)" is the covalent radius scaled up by about 10%, consistent with the scaling between touching and volume filling spheres in an fcc solid. "R(VDW)" is the Van der Waals radius [52].

SY	Z	MASS[U]	R(COV)	R(ASA)	R(VDW)	CONFIGURATION	#(NODES)
H	1	1.008	0.605	0.668	5.454	[0]S1	
HE	2	4.003	1.757	1.943	4.464	[0]S2	
LI	3	6.941	2.324	2.569	4.632	[HE]S1	S1
BE	4	9.012	1.701	1.880	5.187	[HE]S2	S1
B	5	10.811	1.550	1.713	7.716	[HE]S2P1	S1
C	6	12.110	1.455	1.608	7.277	[HE]S2P2	S1
N	7	14.007	1.417	1.567	6.916	[HE]S2P3	S1
O	8	15.999	1.380	1.525	6.614	[HE]S2P4	S1
F	9	18.998	1.361	1.504	6.357	[HE]S2P5	S1
NE	10	20.180	1.342	1.483	6.128	[HE]S2P6	S1
NA	11	22.990	2.910	3.217	5.637	[NE]S1	S2P1
MG	12	24.305	2.570	2.841	5.709	[NE]S2	S2P1
AL	13	26.982	2.230	2.465	8.502	[NE]S2P1	S2P1
SI	14	28.086	2.098	2.319	8.116	[NE]S2P2	S2P1
P	15	30.974	2.003	2.214	7.837	[NE]S2P3	S2P1
S	16	32.066	1.928	2.131	7.625	[NE]S2P4	S2P1
CL	17	35.453	1.871	2.068	7.459	[NE]S2P5	S2P1
AR	18	39.948	1.852	2.047	7.309	[NE]S2P6	S2P1
K	19	39.098	3.836	4.240	7.204	[AR]S1	S3P2
CA	20	40.078	3.288	3.634	6.423	[AR]S2	S3P2
SC	21	44.956	2.721	3.008	6.227	[AR]S2D1	S3P2
TI	22	47.880	2.494	2.757	6.000	[AR]S2D2	S3P2
V	23	50.942	2.305	2.548	5.941	[AR]S2D3	S3P2
CR	24	51.996	2.230	2.465	5.713	[AR]S1D5	S3P2
MN	25	54.938	2.211	2.444	5.595	[AR]S2D5	S3P2
FE	26	55.847	2.211	2.444	5.503	[AR]S2D6	S3P2
CO	27	58.933	2.192	2.423	5.427	[AR]S2D7	S3P2
NI	28	58.340	2.173	2.402	5.355	[AR]S2D8	S3P2
CU	29	63.546	2.211	2.444	6.605	[AR]S1D10	S3P2
ZN	30	65.390	2.362	2.611	5.221	[AR]S2D10	S3P2
GA	31	60.723	2.381	2.632	8.283	[AR]S2P1D10	S3P2
GE	32	72.610	2.305	2.548	8.088	[AR]S2P2D10	S3P2

SY	Z	MASS[U]	R(COV)	R(ASA)	R(VDW)	CONFIGURATION	#(NODES)
AS	33	74.922	2.268	2.507	7.994	[AR]S2P3D10	S3P2
SE	34	78.960	2.192	2.423	7.946	[AR]S2P4D10	S3P2
BR	35	79.904	2.154	2.381	7.916	[AR]S2P5D10	S3P2
KR	36	83.800	2.116	2.339	7.825	[AR]S2P6D10	S3P2
RB	37	85.468	4.082	4.512	7.774	[KR]S1	S4P3D1
SR	38	87.620	3.609	3.990	6.880	[KR]S2	S4P3D1
Y	39	88.906	3.061	3.384	6.321	[KR]S2D1	S4P3D1
ZR	40	91.224	2.740	3.029	5.904	[KR]S2D2	S4P3D1
NB	41	92.906	2.532	2.799	5.981	[KR]S1D4	S4P3D1
MO	42	95.940	2.457	2.715	5.767	[KR]S1D5	S4P3D1
TC	43	97.907	2.400	2.653	5.665	[KR]S2D5	S4P3D1
RU	44	101.070	2.362	2.611	5.599	[KR]S1D7	S4P3D1
RH	45	102.906	2.362	2.611	5.535	[KR]S1D8	S4P3D1
PD	46	106.420	2.419	2.674	5.478	[KR]D10	S4P3D1
AG	47	107.868	2.532	2.799	5.949	[KR]S1D10	S4P3D1
CD	48	112.411	2.797	3.091	5.382	[KR]S2D10	S4P3D1
IN	49	114.818	2.721	3.008	8.434	[KR]S2P1D10	S4P3D1
SN	50	118.710	2.665	2.945	8.300	[KR]S2P2D10	S4P3D1
SB	51	121.757	2.646	2.924	8.353	[KR]S2P3D10	S4P3D1
TE	52	127.600	2.570	2.841	8.447	[KR]S2P4D10	S4P3D1
I	53	126.904	2.513	2.778	8.504	[KR]S2P5D10	S4P3D1
XE	54	131.290	2.476	2.736	8.322	[KR]S2P6D10	S4P3D1
CS	55	132.905	4.441	4.909	8.536	[XE]S1	S5P4D2
BA	56	137.327	3.742	4.136	6.998	[XE]S2	S5P4D2
LA	57	138.906	3.194	3.530	6.656	[XE]S2D1	S5P4D2
CE	58	140.115	3.118	3.446	6.720	[XE]S2D1F1	S5P4D2
PR	59	140.908	3.118	3.446	6.814	[XE]S2F3	S5P4D2
ND	60	144.240	3.099	3.426	6.756	[XE]S2F4	S5P4D2
PM	61	145.000	3.080	3.405	6.703	[XE]S2F5	S5P4D2
SM	62	150.360	3.061	3.384	6.652	[XE]S2F6	S5P4D2
EU	63	151.965	3.496	3.864	6.601	[XE]S2F7	S5P4D2
GD	64	157.250	3.042	3.363	6.365	[XE]S2F8	S5P4D2
TB	65	158.925	3.005	3.321	6.521	[XE]S2F9	S5P4D2
DY	66	162.500	3.005	3.321	6.478	[XE]S2F10	S5P4D2
HO	67	164.930	2.986	3.300	6.442	[XE]S2F11	S5P4D2
ER	68	167.260	2.967	3.279	6.408	[XE]S2F12	S5P4D2
TM	69	168.934	2.948	3.259	6.376	[XE]S2F13	S5P4D2

SY	Z	MASS[U]	R(COV)	R(ASA)	R(VDW)	CONFIGURATION	#(NODES)
YB	70	173.040	3.288	3.634	6.340	[XE]S2F14	S5P4D2
LU	71	174.967	2.948	3.259	6.879	[XE]S2D1F14	S5P4D2
HF	72	178.490	2.721	3.008	5.936	[XE]S2D2F14	S5P4D2
TA	73	180.948	2.532	2.799	5.990	[XE]S2D3F14	S5P4D2
W	74	183.840	2.457	2.715	5.800	[XE]S2D4F14	S5P4D2
RE	75	186.207	2.419	2.674	5.582	[XE]S2D5F14	S5P4D2
OS	76	190.230	2.381	2.632	5.896	[XE]S2D6F14	S5P4D2
IR	77	192.220	2.400	2.653	5.367	[XE]S2D7F14	S5P4D2
PT	78	195.080	2.457	2.715	5.204	[XE]S1D9F14	S5P4D2
AU	79	196.967	2.532	2.799	6.223	[XE]S1D10F14	S5P4D2
HG	80	200.590	2.816	3.112	5.112	[XE]S2D10F14	S5P4D2
TL	81	204.383	2.797	3.091	8.215	[XE]S2P1D10F14	S5P4D2
PB	82	207.200	2.778	3.071	8.120	[XE]S2P2D10F14	S5P4D2
BI	83	208.980	2.759	3.050	8.258	[XE]S2P3D10F14	S5P4D2
PO	84	208.982	2.759	3.050	8.899	[XE]S2P4D10F14	S5P4D2
AT	85	209.987	2.740	3.029	8.976	[XE]S2P5D10F14	S5P4D2
RN	86	222.018	2.721	3.008	9.005	[XE]S2P6D10F14	S5P4D2
FR	87	223.020	4.724	5.222	9.260	[RN]S1	S6P5D3F1
RA	88	226.025	3.779	4.178	6.949	[RN]S2	S6P5D3F1
AC	89	227.028	3.118	3.446	6.572	[RN]S2D1	S6P5D3F1
TH	90	232.038	3.118	3.446	6.418	[RN]S2D2	S6P5D3F1
PA	91	231.036	3.118	3.446	6.470	[RN]S2D1F2	S6P5D3F1
U	92	238.029	2.683	2.966	6.416	[RN]S2D1F3	S6P5D3F1
NP	93	237.048	3.080	3.405	6.470	[RN]S2D1F4	S6P5D3F1
PU	94	244.064	3.061	3.384	6.470	[RN]S2F6	S6P5D3F1
AM	95	243.061	3.496	3.864	6.389	[RN]S2F7	S6P5D3F1
CM	96	247.070	3.042	3.363	6.285	[RN]S2D1F7	S6P5D3F1
BK	97	247.070	3.005	3.321	6.310	[RN]S2F9	S6P5D3F1
CF	98	251.080	3.005	3.321	6.261	[RN]S2F10	S6P5D3F1
ES	99	252.083	2.986	3.300	6.234	[RN]S2F11	S6P5D3F1
FM	100	257.095	2.967	3.279	6.210	[RN]S2F12	S6P5D3F1
MD	101	258.099	2.948	3.259	6.187	[RN]S2F13	S6P5D3F1
NO	102	259.101	3.288	3.634	6.138	[RN]S2F14	S6P5D3F1
LR	103	260.105	2.948	3.259	6.115	[RN]S2D1F14	S6P5D3F1
RF	104	261.109	2.721	3.008	6.614	[RN]S2D2F14	S6P5D3F1
HA	105	262.114	2.532	2.799	6.614	[RN]S2D3F14	S6P5D3F1
CP	106	1.000	0.000	0.000	0.000	[0]	

22 The Universal Force Field (UFF)

The "Universal Force Field"(UFF)[52] is used in the QM-MM coupling and in the pre-optimization tool paw_preopt. It has been published in [52]. Parts of that paper most important for calculations are given here.

The original UFF has 126 atom types, however in the CP-PAW implementation there are currently 141 and occasionally some more may be added. In case of uncertainty have a look into the code: subroutine `UFFTABLE_INI` in the file `paw_classical.f`. A five-character mnemonic label is used to describe the atom types. The first two characters correspond to the chemical symbol; an underscore appears in the second column if the symbol has one letter. The third column describes the hybridization or geometry: 1=linear, 2=trigonal, R=resonant, 3=tetrahedral, 4=square planar, 5=trigonal bipyramidal, 6=octahedral. Thus N_3 is tetrahedral nitrogen, while Rh6 is octahedral rhodium. The fourth and fifth columns are used as indicators for alternate parameters such as formal oxidation state: Rh6+3 indicates an octahedral rhodium formally in the +3 oxidation state. H___B indicates a bridging hydrogen as in B_2H_6. Some parameters of the implementation of UFF in the CP-PAW code are given in the following table. `bond` is the bond radius in Å and `angle` the bond angle in degrees.

FFTYPE	BOND	ANGLE	FFTYPE	BOND	ANGLE
H_	0.354	180.000	AG1+1	1.386	180.000
H___B	0.460	83.500	CD3+2	1.403	109.471
HE4+4	0.849	90.000	IN3+3	1.459	109.471
LI	1.336	180.000	SN3	1.398	109.471
BE3+2	1.074	109.471	SB3+3	1.407	91.600
B_3	0.838	109.471	TE3+2	1.386	90.250
B_2	0.828	120.000	I_	1.382	180.000
C_3	0.757	109.471	XE4+4	1.267	90.000
C_R	0.729	120.000	CS	2.570	180.000
C_2	0.732	120.000	BA6+2	2.277	90.000
C_1	0.706	180.000	LA3+3	1.943	109.471
N_3	0.700	106.700	CE6+3	1.841	90.000
N_R	0.699	120.000	PR6+3	1.823	90.000
N_2	0.685	111.300	ND6+3	1.816	90.000
N_1	0.656	180.000	PM6+3	1.801	90.000
O_3	0.658	104.510	SM6+3	1.780	90.000
O_3_Z	0.528	145.500	EU6+3	1.771	90.000
O_R	0.680	110.300	GD6+3	1.735	90.000
O_2	0.634	120.000	TB6+3	1.732	90.000
O_1	0.639	180.000	DY6+3	1.710	90.000
F_	0.668	180.000	HO6+3	1.696	90.000
NE4+4	0.920	90.000	ER6+3	1.673	90.000
NA	1.539	180.000	TM6+3	1.660	90.000
MG3+2	1.421	109.471	YB6+3	1.637	90.000
AL3	1.244	109.471	LU6+3	1.671	90.000
SI3	1.117	109.471	HF3+4	1.611	109.471
P_3+3	1.101	93.800	TA3+5	1.511	109.471

P_3+5	1.056	109.471	W_6+6	1.392	90.000
P_3+Q	1.056	109.471	W_3+4	1.526	109.471
S_3+2	1.064	92.100	W_3+6	1.380	109.471
S_3+4	1.049	103.200	RE6+5	1.372	90.000
S_3+6	1.027	109.471	RE3+7	1.314	109.471
S_R	1.077	92.200	OS6+6	1.372	90.000
S_2	0.854	120.000	IR6+3	1.371	90.000
CL	1.044	180.000	PT4+2	1.364	90.000
AR4+4	1.032	90.000	AU4+3	1.262	90.000
K_	1.953	180.000	HG1+2	1.340	180.000
CA6+2	1.761	90.000	TL3+3	1.518	120.000
SC3+3	1.513	109.471	PB3	1.459	109.471
TI3+4	1.412	109.471	BI3+3	1.512	90.000
TI6+4	1.412	90.000	PO3+2	1.500	90.000
V_3+5	1.402	109.471	AT	1.545	180.000
CR6+3	1.345	90.000	RN4+4	1.420	90.000
MN6+2	1.382	90.000	FR	2.880	180.000
FE3+2	1.412	109.470	RA6+2	2.512	90.000
FE6+2	1.335	90.000	AC6+3	1.983	90.000
CO6+3	1.241	90.000	TH6+4	1.721	90.000
NI4+2	1.164	90.000	PA6+4	1.711	90.000
CU3+1	1.302	109.471	U_6+4	1.684	90.000
ZN3+2	1.193	109.471	NP6+4	1.666	90.000
GA3+3	1.260	109.471	PU6+4	1.657	90.000
GE3	1.197	109.471	AM6+4	1.660	90.000
AS3+3	1.211	92.100	CM6+3	1.801	90.000
SE3+2	1.190	90.600	BK6+3	1.761	90.000
BR	1.192	180.000	CF6+3	1.750	90.000
KR4+4	1.147	90.000	ES6+3	1.724	90.000
RB	2.260	180.000	FM6+3	1.712	90.000
SR6+2	2.052	90.000	MD6+3	1.689	90.000
Y_3+3	1.698	109.471	NO6+3	1.679	90.000
ZR3+4	1.564	109.471	LW6+3	1.698	90.000
NB3+5	1.473	109.471	CPR	0.551	90.000
MO6+6	1.467	90.000	CPR_B	0.340	90.000
MO3+6	1.484	109.471	CIR	0.616	90.000
TC6+5	1.322	90.000	PIR	0.616	90.000
RU6+2	1.478	90.000			
RH6+3	1.332	90.000			
PD4+2	1.338	90.000			

23 Methods

23.1 Friction dynamics and annealing schedules

The time evolution of most quantities is done as friction dynamics. This friction itself may be time dependent for example in optimization procedures or via a thermostat. Here we shall describe the main features, and provide guidelines for the selction of the free parameters.

The general equation of motion is

$$m\ddot{x} = F(x) - m\alpha\dot{x}$$

$m\alpha$ is the friction coefficient. The mass has been integrated to be consistent with the formulation of the thermostats.

In practice this equation is discretized.

$$m\frac{x(t+\Delta) - 2x(t) + x(t-\Delta)}{\Delta^2} = F(x) - m\alpha\frac{x(t+\Delta) - x(t-\Delta)}{2\Delta}$$

which we resolve for $x(t+\Delta)$ as

$$x(t+\Delta) = \frac{2}{1+\frac{\alpha\Delta}{2}}x(t) - \frac{1-\frac{\alpha\Delta}{2}}{1+\frac{\alpha\Delta}{2}}x(t-\Delta) + \frac{1}{m}F(x)\frac{\Delta^2}{1+\frac{\alpha\Delta}{2}}$$

We define a new friction coefficient $a = \frac{\alpha\Delta}{2}$, which is the parameter specified as input to the program, so that

$$x(t+\Delta) = \frac{2}{1+a}x(t) - \frac{1-a}{1+a}x(t-\Delta) + \frac{1}{m}F(x)\frac{\Delta^2}{1+a}$$

There are two important special cases:

- Undamped dynamics results from $a = 0$. Here the friction is zero, and we obtain the well known Verlet algorithm

  $$x(t+\Delta) = 2x(t) - x(t-\Delta) + \frac{1}{m}F(x)\Delta^2$$

  for a second order differential equation. Over long times, the dynamics is energy conserving independent of the step size as result of the time inversion symmetry of the verlet algorithm.

- steepest descent is obtained with the choice $a = 1$.

$$x(t + \Delta) = x(t) - \frac{1}{m} F(x) \frac{\Delta^2}{2}$$

  Steepest descent corresponds to a dynamics with infinite friction. The motion does not come to rest even in this limit because the the time step is scaled inversely proportional to the friction. The product $\Delta^2/(2m)$ plays the role of the mixing parameter.

During optimization, that is with the options !AUTO switched on, the system can choose between two friction values. If the total energy is lowered the lower fiction value is chosen, and if the energy goes up the higher. Both are scaled by a factor, which allows to start with a high friction in order to take out energy from high frequency, that is high energy, modes. The reduced friction allows then to work on the low frequency modes.

If we know the frequencies $\omega = \frac{2\pi}{T}$, where T is the period of one oscillation we can provide some guidelines for the friction parameters.

- The dynamics becomes unstable for a time step $\Delta \geq 2/\omega_0 \approx T/3$. For timesteps $\Delta \leq T/10$ the error in the frequency is less than 1%. The frequency ω of the discretized undamped dynamics increases with the time step.

- The most rapid convergence rate is obtained for a friction lying just at the boundary between damped and overdamped dynamics, namely $a = 2\pi\frac{\Delta}{T}$. For frequencies with a shorter period than $\frac{2\pi\Delta}{a}$ the the variables converge exponentially with a rate $2\Delta/a$, which makes a large friction a appear favorable. However at the same time, oscilations with a period $T > 2\pi\Delta/a$ do not converge due to overdamping. (In order to converge bond stretch frequencies, which typically lie above 500 cm^{-1}, the friction parameter should not be higher than $a = 0.025$. This also indicates the inefficiency of the steepest descent approach which uses $a = 1$.)

23.2 Optimum friction scheme

The optimum friction is estimated from the forces and velocities of two subsequent time steps. These data are used to estimate the effective curvature and the effective mass along the line of propagation.

The optimum friction factor is estimated according to

$$a_{opt} = \Delta \sqrt{\frac{\dot{R}\dot{F}}{\dot{R}\mathbf{m}\dot{R}}}$$

Numerator and Denominator can oszillate rapidly. Therefore we use a running average, which is calculated accoring to

$$\langle a_{opt} \rangle = \beta a_{opt} + (1-\beta)\langle a_{opt}$$

The running average approaches the actual value exponentially. The deviation approaches $\frac{1}{2}$ of the original value after n time steps of

$$\beta = 1 - 2^{-\frac{1}{n}}$$

23.3 Wave function dynamics

Here we start with a total energy

$$E_{tot} = \sum_n f_n \langle \dot{\tilde{\Psi}}_n | m_\Psi | \dot{\tilde{\Psi}}_n \rangle + E_{DFT} - \sum_{n,m} \langle \tilde{\Psi}_n | \tilde{O} | \tilde{\Psi}_m \rangle \Lambda_{n,m} \tag{34}$$

The $\tilde{\Psi}$ are the pseudowave functions as defined by the PAW method, $\Lambda_{n,m}$ are the Lagrange parameters corresponding to the constraint of orthonormal wave functions and

$$\begin{aligned} E_{DFT} &= \sum_n \langle \Psi_n | -\frac{1}{2}\nabla^2 | \Psi_n \rangle \\ &+ \frac{1}{2} \int dr \int dr' \frac{(n(r)+n^Z(r))(n(r')+n^Z(r'))}{|r-r'|} \\ &+ \int dr n(r) \epsilon_{xc}(n_\sigma(r), \nabla n_\sigma(r)) \end{aligned}$$

Here we denote with $n^Z(r) = -\sum_R Z_R \delta(r-R)$ the density of the point charges of the nuclei.

$$m_\Psi |\ddot{\tilde{\Psi}}_n\rangle = -\tilde{H}|\tilde{\Psi}_n\rangle + \sum_m \tilde{O}\Lambda_{m,n} - m_\Psi |\dot{\tilde{\Psi}}_n\rangle f_\Psi \tag{35}$$

23.4 Nuclear dynamics

In order to move also the nuclei we add the kinetic energy of the atoms

$$\Delta E_{tot} = \sum_R \frac{1}{2} M_R \dot{R}^2 - \frac{1}{2} C_R \dot{R}^2 \tag{36}$$

Here the M_R are the true nuclear masses and C_R are the effective masses of the wave functions. The second term aims at subtracting the fictitious kinetic energy of the wave functions in a parameterized form.

$$(M_i - C_i)\ddot{R}_i \;=\; F_i - (M_i - C_i)\dot{R}_i f_R$$

23.5 Thermostats

Thermostats can be linked to both atoms and wave functions by adding

$$\Delta E_{tot} = \frac{1}{2} Q_\Psi \dot{x}_\Psi^2 + \frac{1}{2} Q_R \dot{x}_R^2 + g k_B T x_R \tag{37}$$

The equations of motion do not directly follow from the total energy as there is no Lagrangian formulation of the two thermostat method.

$$\begin{aligned}
m_\Psi |\ddot{\tilde{\Psi}}_n\rangle &= -\tilde{H}|\tilde{\Psi}_n\rangle + \sum_m \tilde{O}|\tilde{\Psi}_n \Lambda_{m,n} - m_\Psi |\dot{\tilde{\Psi}}_n\rangle \dot{x}_\Psi \\
(M_i - C_i)\ddot{R}_i &= F_i - (M_i - C_i)\dot{R}_i \dot{x}_R + C_i \dot{R}_i \dot{x}_\Psi \\
Q_\Psi \ddot{x}_\Psi &= 2\sum_n \langle \dot{\tilde{\Psi}}_n | m_\Psi | \dot{\tilde{\Psi}}_n \rangle - \sum_i C_i \dot{R}^2 \\
Q_R \ddot{x}_\Psi &= \sum_i (M_i - C_i)\dot{R}^2 - g k_B T
\end{aligned}$$

As an option one can also use only the atom thermostat x_R and replace the wave function thermostat friction $\dot{x}_\Psi$ by a constant friction f_Ψ, in the equation for atom and wave function thermostat.

23.6 Occupations

$$\begin{aligned}\Delta E_{tot} &= \sum_{n,\sigma} m_X \dot{X}^2_{n,\sigma} + k_B T \sum_{n,\sigma} \Big[f_{n,\sigma} \ln[f_{n,\sigma}] + (1 - f_{n,\sigma}) \ln[1 - f_{n,\sigma}] \Big] \\ &+ \mu \Big[\sum_{n,\sigma} f_{n,\sigma} - N \Big] + \mu \Big[\sum_{n,\sigma} f_{n,\sigma} (\delta_{\sigma,\uparrow} - \delta_{\sigma,\downarrow}) - S \Big] B\end{aligned}$$

Here B is the magnetic Field acting on the electron spins, μ is the electron chemical potential, S is the total spin and N is the total number of electrons.

23.7 Sawtooth behavior of the total energy.

If we change the volume of the unit cell, the number of basis functions changes abruptly always when a new star of G-vectors enters or leaves the sphere defined by the plane wave cutoff. This abrupt change of the basis set results in abrupt changes of the total energy. Thus we obtain a typical saw-tooth behavior of the energy as function of volume as shown in Fig. 23.7.

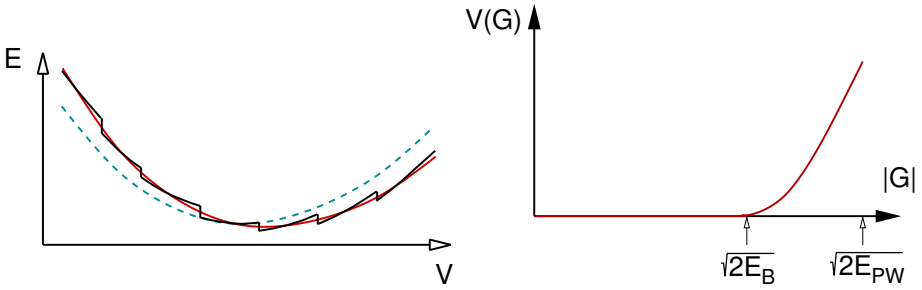


Figure 2: Left: Sawtooth behavior of the total energy as function of volume with a fixed plane wave cutoff (black) with a fixed number of plane waves (green dashed) and the correct result with a fully converged calculation (red). By adding a G-dependent potential the plane wave convergence can be artyificially accelerated.

The steps in the total energy vanish if the calculation is fully converged in the basis set size, because additional basis functions do not lower the total energy. If the function is not converged, the best result is obtained by keeping the same plane wave cutoff. However, if a the energy obtained at a few points is then fitted to, for example, a parabola or the Murnaghan equation of state[53], one has to be careful with the steps. A step can considerably mess up the fit. In these cases it is better to use a larger step size in the lattice constant.

This sawtooth behavior can be suppressed by including an additional G-dependent bucket potential that ensures that the wave function coefficients at the plane wave cutoff vanish strictly.

23.8 Overcompleteness problem when using too many projector functions

When testing the convergence with respect to the number of projector functions one usually runs into instabilities. I believe this problem is a sort of overcompleteness problem. There is no rigorous investigation so far.

The problem is the following: One cannot map a number of N projector functions onto less than N real space grid points. On the grid they are no more distingishable. Thus there are combinations of partial waves with coefficients that are completely decoupled from the plane wave part.

To illustrate the problem let us estimate the number of degrees of freedom on a grid with a given plane wave cutoff in spheres of different radius r.

r	20 Ry	40 Ry	60 Ry
1.5 a_0	5	14	26
2 a_0	12	34	62
2.5 a_0	23	66	122
3 a_0	40	115	211

A semi-core setup with two shells of s- and p-character and, in addition one shell for d and f electrons yields 20 projector functions.

23.9 Constraints acting on atoms

23.10 Electrostatic decoupling and coupling of point charges

23.11 Non-collinear densities

In the current implementation the non-collinear spin density is first transformed into a locally collinear spin density, from which the exchange functional is evaluated.

$$n_s(\vec{r}) = |\vec{\sigma}(\vec{r})|$$

The choice is non-unique regarding gradient corrections because the gradient of the spin density is a tensor.

$$\partial_u \sigma_j(\vec{r})$$

Our choice

$$\vec{\nabla} n_s(\vec{r}) = \vec{\nabla} |\vec{\sigma}(\vec{r})|$$

is only one of the possible choices.

For the one-center densities, we evaluate

$$\begin{aligned} n_s &= \sqrt{Q^2 + (\vec{\sigma}^2 - Q^2)} = Q\sqrt{1 + \left(\frac{\vec{\sigma}^2}{Q^2} - 1\right)} \\ &= Q\Big[1 + \frac{1}{2}\left(\frac{\vec{\sigma}^2}{Q^2} - 1\right) - \frac{1}{8}\left(\frac{\vec{\sigma}^2}{Q^2} - 1\right)^2 + \ldots \end{aligned}$$

where Q is defined as

$$Q = \sqrt{C + (\hat{P}_s \vec{\sigma})^2}$$

and $\hat{P}_s$ is an operator the projects out nonspherical terms from the spin density. The constant is a small value that avoids a divide-by-zero.

Note, that these approximations are not necessary for collinear densities. Therefore the results of collinear and non-collinear densities differ even though the density may be collinear!

There is a second implementation, which is even more approximate, but it provides the exact result for strictly collinear densities. This option must be enabled in the code.

24 Troubleshooting

24.1 Errors in the input data

- A common mistake is to enter data with the wrong data type. In this case you will receive a message as follows, which should help to locate the problem.

```
TYPE INCONSISTENT
VARIABLE ID HAS THE VALUE DT
VARIABLE NTH HAS THE VALUE          1
VARIABLE TYPE EXPECTED HAS THE VALUE R(8)
VARIABLE ACTUAL TYPE HAS THE VALUE I(4)
STOP IN LINKEDLIST_GETGENERIC
```

  The first line says that there is a type mismatch. The last line says that the problem happened in method of the linkedlist object, which is also responsible for linked list/tree data structures such as block-structured input files. In particular the program attempts to collect a data item from this tree structure. The second line gives you the name of the variable, in this example "DT", which is the key word for the time step. It cannot tell in which file or which block the data is located. The third line tells you that it is the first data with this name in the block. The next two lines say that the program expects a data with type "real(8)", but finds one of type "integer(4)". It appears that the program has mistaken your data for DT as an integer, because the user forgot to type the decimal point.

- The program does not do what you want it to. You may have mistyped keywords, or mixed bup the tree structure. This renders a lot of data invisible, and the program uses the default values. Try to consult the protocol of the program. It should report all options selected. If options are not listed, the option is not selected, or the corresponding value is zero. (Of course, it can also be that it is not reported. In this case see whether you get a statement in the protocol if you select the option. If not, please report this to the author.)

- If you did not provide, or mistype, a keyword that does not have a default, you may receive a message like this:

```
KEYWORD T= IS MANDATORY
STOP IN STRCIN; !STRUCTURE!LATTICE
```

24.2 Runtime errors

This is a list of nontrivial user errors from which the program is not sufficiently protected and loose ends of the code where particular caution is needed. It is my goal to adjust the program to keep this list as short as possible.

1. Sometimes the program stops with an error message saying "LOOP FOR ORTHOGONALIZATION NOT CONVERGED". This is an indication of extremely large changes of the wave functions, which can no longer be orthogonalized. There are several possible problems that can lead to this, and several possible remedies:

   - The atomic positions have been chosen with unreasonably short distances, or with atoms lying on top of each other.
   - Some setups produce this behavior in the first few time steps. In this case reduce the plane wave cutoff to a small value such as 5, 10 or 20 Ry. If this alone does not help, also reduce the time step, but leave the wave function masses MPSI and MPSICG2 constant. It is suffient to do this for the first few (about five or ten) time steps. If you started a new job with gradient corrected DFT functionals, you should try "!CONTROL!FOURIER:CDUAL=4" or reach convergence furst for a truly local functional before switching gradient corrections on. If the kinetic energy for the wave functions decreases to a reasonable value of a few Hartree, one can restart with the original values.
   - If all this does not help, you are in trouble!

2. If the program produces a coredump, increase first the stack size using the "ulimit" command. You can also control the size of the stack array using the compiler option -bmaxstack.

3. The code has a limited size of the send-receive buffer. The program is not guarded against a buffer overflow, even though the MPI Library should give a useful message (affects only the parallel version).

4. Errors occur if the operating system, Fortran runtime environment or SP2 Software are not on the same level, used during compilation.

5. If you find that the program crashes without providing an error message, and if you have the source code, specify `!CONTROL!GENERIC:TRACE=T`.

The program will then continuously write information on subroutines entered and left, which often allows one to locate the problem. Note that, in order to avoid producing files that are too large to be useful, not all routines result in a trace message.

6. The routine "MADELUNG" chooses itself the range of real and reciprocal space summations. It has not been sufficiently tested whether these choices are accurate. It may affect the result of the object "Isolate" that separates isolated molecules from its periodic images.

7. The Becke88 gradient correction[24]for exchange produces unreasonable results if the total density or the spin density is vanishes. In this case small numerical values for the gradient pick up a singularity in the functional form. This may happen for a fully spin polarized system. The cure is to pick another density functional, which does not contain the Becke88 functional.

24.3 Overcompleteness of projector functions

Using too many partial waves leads to so-called **overcompleteness** of projector functions. This is counterintitive because one would think that more terms would make the description more accurate.

The origin of the problem is that, with a finite plane wave cutoff, the wave function has only a limited number of degrees of freedom inside the atomic region. If the numbert of projector functions exceeds this number of degrees of freedom, there is atleast one partial wave with a prefactor that is not fixed by the wave function. If this partial wave increases the energy, it will be projected out. If it, however, lowers the energy the calculation becomes unstable and the results unphysical.

$$
\begin{aligned}
N &= \frac{4\pi}{3}G_x^3\frac{1}{V_G} \overset{V_G=(2\pi)^3/V_R}{=} \frac{4\pi}{3}G_x^3\frac{V_R}{(2\pi)^3} \\
&\overset{E_{PW}=\frac{1}{2}G_x^2}{=} \frac{4\pi}{3}\Big(2E_{PW}\Big)^{\frac{3}{2}}\frac{V_R}{(2\pi)^3} \\
\Rightarrow \quad \frac{N}{V_R} &= \frac{4\pi}{3}\Big(\frac{\sqrt{2E_{PW}}}{2\pi}\Big)^3 \\
\sum_{\ell=0}(2\ell+1)n_\ell &< \frac{4\pi}{3}r_{cov}^3\frac{4\pi}{3}\Big(\frac{\sqrt{2E_{PW}}}{2\pi}\Big)^3 = \underbrace{\frac{2}{9\pi}}_{\approx 1/30}\Big(r_{cov}\sqrt{2E_{PW}}\Big)^3
\end{aligned}
\tag{38}
$$

Here N is the number of plane waves, or the number of real-space grid points respectively. n_ℓ is the number of different radial partial waves for the main angular momentum ℓ. G_x is the maximum size for the wave vectors considered. V_R is the real-space unit-cell volume. V_G is the reciprocal-space unit-cell volume. $E_{PW} = \frac{1}{2}G_x^2$ is the plane wave cutoff and r_{cov} is the covalent radius.

We obtain

	r_{cov}	20 Ry	30 Ry	50 Ry
H	0.32 Å	1.4	2.5	5.5
O	0.73 Å	16.6	30.5	65.6
Si	1.11 Å	58.4	107.2	230.8

On the right-hand side the estimated number of degrees of freedom are listed. It should be considered that the effective augmentation radius is usually smaller than the covalent radius. This ratio enters to the third power. Hence, if the augmentation radius is 20 % smaller than the covalent radius only half the number of degrees of freedom are present.

This table indicates that for hydrogen, only the s-type projector should be used. For second-row elements the choice of the number of projectors appears fairly uncritical. For first-row elements, however, insdtabilities may already be present if only one projector function is used for s, p, and d-type angular momenta.

25 Code documentation

25.1 Strategy

The main documentation for the user is the manual file. It shall contain a complete description of all input parameters and their functionality.

For the algorithms we prepare reports with a detailed description and their theoretical background. These reports are currently only for internal use.

The code is written as self-explanatory as possible. I understand that this looks like an excuse for not documenting the code. The idea behind is to use meaningful variable names, use explicit typing such as declaring each variable explicitely, declaring its intent (in,out,inout), and to use similar structures and conventions throughout the code. This technique avoids the common problem that documentation and code become asynchronous.

25.2 Doxygen

Doxygen is a tool for code analysis and automatic code documentation. It is a free multi-platform tool available at `http://www.stack.nl/~dimitri/doxygen/`.

The CP-PAW code supports analysis with Doxygen. Currently this is in a testing phase and not in its final implementation.

- Only the debug version of the code is analyzed. Therefore this must be compiled before analysing it with `make dbg`. Then you construct the doxygen files with `make doxygen`. The Doxygen data are within a directory `Doxydocs` in the directory alongside with the paw executables.
- one can view the data (on OSXusing

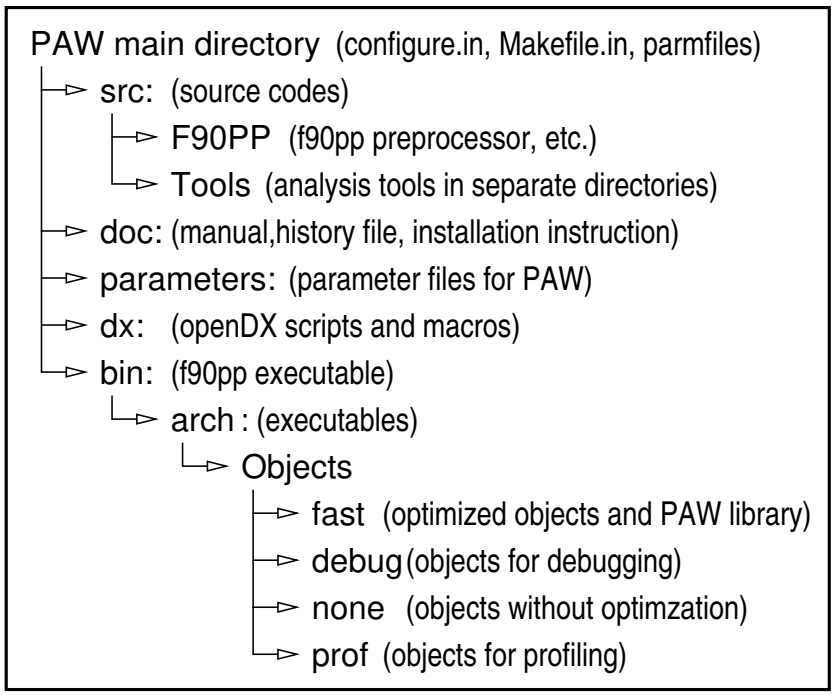
  ```
  A=$(which paw_dbg.x); A=${A%paw_dbg.x}Doxydocs/html/index.html; op
  ```

  The last command opens the Web-browser on the file `$A`, and should be replaced by the corresponding command on your system.

A Installation

The installation is described the in the installation guide also contained in the documentation.

The directory structure will have the following form


PAW main directory (configure.in, Makefile.in, parmfiles)
src: (source codes)
F90PP (f90pp preprocessor, etc.)
Tools (analysis tools in separate directories)
doc: (manual,history file, installation instruction)
parameters: (parameter files for PAW)
dx: (openDX scripts and macros)
bin: (f90pp executable)
arch : (executables)
Objects
fast (optimized objects and PAW library)
debug (objects for debugging)
none (objects without optimzation)
prof (objects for profiling)

B Symmetry tables

B.1 Bravais lattices

Unit cells definitions by Bradley and Cracknell[55]:

Bravais Lattice		$\vec{T}_1$	$\vec{T}_2$	$\vec{T}_3$
triclinic				
primitive	Γ_1			
monoclinic				
primitive	Γ_m	$(0,-b,0)$	$(a\sin(\gamma),-a\cos(\gamma),0)$	$(0,0,c)$
base centered	Γ_m^b	$(0,-b,0)$	$\frac{1}{2}(a\sin(\gamma),-a\cos(\gamma),-c)$	$\frac{1}{2}(a\sin(\gamma),-a\cos(\gamma),c)$
orthorhombic				
primitive	Γ_o	$(0,-b,0)$	$(a,0,0)$	$(0,0,c)$
base centered	Γ_o^b	$(\frac{1}{2}a,-\frac{1}{2}b,0)$	$(\frac{1}{2}a,\frac{1}{2}b,0)$	$(0,0,c)$
body centered	Γ_o^v	$\frac{1}{2}(a,b,c)$	$\frac{1}{2}(-a,-b,c)$	$\frac{1}{2}(a,-b,-c)$
face centered	Γ_o^f	$\frac{1}{2}(a,0,c)$	$\frac{1}{2}(0,-b,c)$	$\frac{1}{2}(a,-b,0)$
tetragonal				
primitive	Γ_q	$(a,0,0)$	$(0,a,0)$	$(0,0,c)$
body centered	Γ_q^v	$\frac{1}{2}(-a,a,c)$	$\frac{1}{2}(a,-a,c)$	$\frac{1}{2}(a,a,-c)$
trigonal				
primitive	Γ_{rh}	$(0,a,c)$	$(\frac{1}{2}\sqrt{3}a,\frac{1}{2}a,c)$	$(-\frac{1}{2}\sqrt{3}a,\frac{1}{2}a,c)$
hexagonal				
primitive	Γ_h	$(0,-1,0)a$	$(\frac{1}{2}\sqrt{3},\frac{1}{2},0)a$	$(0,0,1)c$
cubic				
primitive	Γ_c	$(1,0,0)a$	$(0,1,0)a$	$(0,0,1)a$
face centered	Γ_c^f	$\frac{1}{2}a(0,1,1)$	$\frac{1}{2}a(1,0,1)$	$\frac{1}{2}a(1,1,0)$
body centered	Γ_c^v	$\frac{1}{2}a(-1,1,1)$	$\frac{1}{2}a(1,-1,1)$	$\frac{1}{2}a(1,1,-1)$

B.2 Spacegroups

The translation between the spacegroup number and the international space group symbols is given in tables 1 and 2.

1	P1	2	$P\bar{1}$	3	P2	4	$P2_1$	5	C2
6	Pm	7	Pc	8	Cm	9	Cc	10	P2/m
11	$P2_1/m$	12	C2/m	13	P2/c	14	$P2_1/c$	15	C2/c
16	P222	17	$P222_1$	18	$P2_12_12$	19	$P2_12_12_1$	20	$C222_1$
21	C222	22	F222	23	I222	24	$I2_12_12_1$	25	Pmm2
26	$Pmc2_1$	27	Pcc2	28	Pma2	29	$Pca2_1$	30	Pnc2
31	$Pmn2_1$	32	Pba2	33	$Pna2_1$	34	Pnn2	35	Cmm2
36	$Cmc2_1$	37	Ccc2	38	Amm2	39	Aem2	40	Ama2
41	Aea2	42	Fmm2	43	Fdd2	44	Imm2	45	Iba2
46	Ima2	47	Pmmm	48	Pnnn	49	Pccm	50	Pban
51	Pmma	52	Pnna	53	Pmna	54	Pcca	55	Pbam
56	Pccn	57	Pbcm	58	Pnnm	59	Pmmn	60	Pbcn
61	Pbca	62	Pnma	63	Cmcm	64	Cmce	65	Cmmm
66	Cccm	67	Cmme	68	Ccce	69	Fmmm	70	Fddd
71	Immm	72	Ibam	73	Ibca	74	Imma	75	P4
76	$P4_1$	77	$P4_2$	78	$P4_3$	79	I4	80	$I4_1$
81	$P\bar{4}$	82	$I\bar{4}$	83	P4/m	84	$P4_2/m$	85	P4/n
86	$P4_2/n$	87	I4/m	88	$I4_1/a$	89	P422	90	$P42_12$
91	$P4_122$	92	$P4_12_12$	93	$P4_222$	94	$P4_22_12$	95	$P4_322$
96	$P4_32_12$	97	I422	98	$I4_122$	99	P4mm	100	P4bm
101	$P4_2cm$	102	$P4_2nm$	103	P4cc	104	P4nc	105	$P4_2mc$
106	$P4_2bc$	107	I4mm	108	I4cm	109	$I4_1md$	110	$I4_1cd$
111	$P\bar{4}2m$	112	$P\bar{4}2c$	113	$P\bar{4}2_1m$	114	$P\bar{4}2_1c$	115	$P\bar{4}m2$
116	$P\bar{4}c2$	117	$P\bar{4}b2$	118	$P\bar{4}n2$	119	$I\bar{4}m2$	120	$I\bar{4}c2$

Table 1: Space group number as given in the "International tables for Crystallography, Vol. A" and the corresponding international space group symbol (Herman-Maughin notation). Retrieved from the Bilbao Crystallographic Server http://www.cryst.ehu.es/cryst/text/table.html on Dec. 21, 2013. For further information, see Aroyo, et. al. Zeitschrift für Kristallographie (2006), 221, 1, 15-27.

121	I$\bar{4}$2m	122	I$\bar{4}$2d	123	P4/mmm	124	P4/mcc	125	P4/nbm
126	P4/nnc	127	P4/mbm	128	P4/mnc	129	P4/nmm	130	P4/ncc
131	$P4_2$/mmc	132	$P4_2$/mcm	133	$P4_2$/nbc	134	$P4_2$/nnm	135	$P4_2$/mbc
136	$P4_2$/mnm	137	$P4_2$/nmc	138	$P4_2$/ncm	139	I4/mmm	140	I4/mcm
141	$I4_1$/amd	142	$I4_1$/acd	143	P3	144	$P3_1$	145	$P3_2$
146	R3	147	P$\bar{3}$	148	R$\bar{3}$	149	P312	150	P321
151	$P3_1$12	152	$P3_1$21	153	$P3_2$12	154	$P3_2$21	155	R32
156	P3m1	157	P31m	158	P3c1	159	P31c	160	R3m
161	R3c	162	P$\bar{3}$1m	163	P$\bar{3}$1c	164	P$\bar{3}$m1	165	P$\bar{3}$c1
166	R$\bar{3}$m	167	R$\bar{3}$c	168	P6	169	$P6_1$	170	$P6_5$
171	$P6_2$	172	$P6_4$	173	$P6_3$	174	P$\bar{6}$	175	P6/m
176	$P6_3$/m	177	P622	178	$P6_1$22	179	$P6_5$22	180	$P6_2$22
181	$P6_4$22	182	$P6_3$22	183	P6mm	184	P6cc	185	$P6_3$cm
186	$P6_3$mc	187	P$\bar{6}$m2	188	P$\bar{6}$c2	189	P$\bar{6}$2m	190	P$\bar{6}$2c
191	P6/mmm	192	P6/mcc	193	$P6_3$/mcm	194	$P6_3$/mmc	195	P23
196	F23	197	I23	198	$P2_13$	199	$I2_13$	200	Pm$\bar{3}$
201	Pn$\bar{3}$	202	Fm$\bar{3}$	203	Fd$\bar{3}$	204	Im$\bar{3}$	205	Pa$\bar{3}$
206	Ia$\bar{3}$	207	P432	208	$P4_232$	209	F432	210	$F4_132$
211	I432	212	$P4_332$	213	$P4_132$	214	$I4_132$	215	P$\bar{4}$3m
216	F$\bar{4}$3m	217	I$\bar{4}$3m	218	P$\bar{4}$3n	219	F$\bar{4}$3c	220	I$\bar{4}$3d
221	Pm$\bar{3}$m	222	Pn$\bar{3}$n	223	Pm$\bar{3}$n	224	Pn$\bar{3}$m	225	Fm$\bar{3}$m
226	Fm$\bar{3}$c	227	Fd$\bar{3}$m	228	Fd$\bar{3}$c	229	Im$\bar{3}$m	230	Ia$\bar{3}$d

Table 2: Space group number as given in the "International tables for Crystallography, Vol. A" and the corresponding international space group symbol (Herman-Maughin notation). Retrieved from the Bilbao Crystallographic Server http://www.cryst.ehu.es/cryst/text/table.html on Dec. 21, 2013. For further information, see Aroyo, et. al. Zeitschrift für Kristallographie (2006), 221, 1, 15-27.

C Bands

Here I will list the high-symmetry points of the Brillouin zone, for use with the `paw_bands.x` tool. If the real space lattice vectors are chosen consistent with the convention below, one can directly use the relative coordinates as `xk1` and `xk2` in the `bcntl` file.

See http://lampx.tugraz.at/~hadley/ss1/bzones/

C.1 Face-centered cubic lattice

real space vectors		reciprocal space vectors	
$\vec{T}_1$	$(0, \frac{1}{2}, \frac{1}{2})a$	$\vec{g}_1$	$(-1, 1, 1)\frac{2\pi}{a}$
$\vec{T}_2$	$(\frac{1}{2}, 0, \frac{1}{2})a$	$\vec{g}_2$	$(1, -1, 1)\frac{2\pi}{a}$
$\vec{T}_3$	$(\frac{1}{2}, \frac{1}{2}, 0)a$	$\vec{g}_3$	$(1, 1, -1)\frac{2\pi}{a}$

$\Gamma - X - W - L - \Gamma - K - X'$		
$(0) - (1) - (\frac{3}{2}) - (2.20711) - (3.07313) - (4.13379) - (4.48735)$		
Symbol	absolute coord.	relative coord. `xk1,xk2`
Γ	$(0, 0, 0)\frac{2\pi}{a}$	$(0, 0, 0)$
X	$(1, 0, 0)\frac{2\pi}{a}$	$(0, \frac{1}{2}, \frac{1}{2})$
W	$(1, \frac{1}{2}, 0)\frac{2\pi}{a}$	$(\frac{1}{4}, \frac{1}{2}, \frac{3}{4})$
L	$(\frac{1}{2}, \frac{1}{2}, \frac{1}{2})\frac{2\pi}{a}$	$(\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$
Γ	$(0, 0, 0)\frac{2\pi}{a}$	$(0, 0, 0)$
K	$(\frac{3}{4}, \frac{3}{4}, 0)\frac{2\pi}{a}$	$(\frac{3}{8}, \frac{3}{8}, \frac{3}{4})$
X'	$(1, 1, 0)\frac{2\pi}{a}$	$(\frac{1}{2}, \frac{1}{2}, 1)$

The point K lies on the connection from Γ to X', where X' is the X-point of a nearest neighbor reciprocal lattice vector of the Γ point.

C.2 Body-centered cubic lattice

real space vectors		reciprocal space vectors	
$\vec{T}_1$	$(-\frac{1}{2}, \frac{1}{2}, \frac{1}{2})a$	$\vec{g}_1$	$(0, 1, 1)\frac{2\pi}{a}$
$\vec{T}_2$	$(\frac{1}{2}, -\frac{1}{2}, \frac{1}{2})a$	$\vec{g}_2$	$(1, 0, 1)\frac{2\pi}{a}$
$\vec{T}_3$	$(\frac{1}{2}, \frac{1}{2}, -\frac{1}{2})a$	$\vec{g}_3$	$(1, 1, 0)\frac{2\pi}{a}$

$N-\Gamma-H-N-P-\Gamma$ $(0)-(0.7071)-(1.7071)-(2.5731)-(3.0731)-(3.9392)$		
Symbol	absolute coord.	relative coord. `xk1,xk2`
Γ	$(0,0,0)\frac{2\pi}{a}$	$(0,0,0)$
H	$(1,0,0)\frac{2\pi}{a}$	$(-\frac{1}{2},\frac{1}{2},\frac{1}{2})$
P	$(\frac{1}{2},\frac{1}{2},\frac{1}{2})\frac{2\pi}{a}$	$(\frac{1}{4},\frac{1}{4},\frac{1}{4})$
N	$(\frac{1}{2},\frac{1}{2},0)\frac{2\pi}{a}$	$(0,0,\frac{1}{2})$

References

[1] P.E. Blöchl and A. Togni. First-principles investigation of enantioselective catalysis: Asymmetric allylic amination with pd-complexes bearing p,n-ligands. *Organometallics*, 15:4125, 1996.

[2] P. E. Blöchl. Projector augmented-wave method. *Phys. Rev. B*, 50: 17953–17979, Dec 1994. doi: 10.1103/PhysRevB.50.17953. URL http://link.aps.org/doi/10.1103/PhysRevB.50.17953.

[3] Pierre Hohenberg and Walter Kohn. Inhomogeneous electron gas. *Phys. Rev.*, 136:B864, 1964. doi: 10.1103/PhysRev.136.B864. URL http://link.aps.org/doi/10.1103/PhysRev.136.B864.

[4] Walter Kohn and Lu J. Sham. Self-consistent equations including exchange and correlation effects. *Phys. Rev.*, 140: A1133, 1965. doi: 10.1103/PhysRev.140.A1133. URL http://link.aps.org/doi/10.1103/PhysRev.140.A1133.

[5] A. D. Becke. Density-functional thermochemistry. ii. the effect of the perdew-wang generalized-gradient correlation correction. *J. Chem. Phys.*, 97:9173, 1992.

[6] R.M. Dickson and A.D. Becke. Basis-set ffree local density functional calculations of geometries of polyatomic molecules. *J. Chem. Phys.*, 99:3898, 1993. DFT benchmark.

[7] Roberto Car and Michele Parrinello. Unified approach for molecular dynamics and density-functional theory. *Phys. Rev. Lett*, 55:2471, 1985. doi: 10.1103/PhysRevLett.55.2471. URL http://link.aps.org/doi/10.1103/PhysRevLett.55.2471.

[8] R. Car and M. Parrinello. *Simple Molecular Systems at Very High Density*, chapter A Unified Approach for Molecular Dynamics and Density Functional Theory, page 455. Plenum, NY, 1989.

[9] G. Pastore, E. Smargiassi, and F. Buda. Theory of *ab initio* molecular-dynamics calculations. *Phys. Rev. A*, 44:6334–6347, Nov 1991. doi: 10.1103/PhysRevA.44.6334. URL http://link.aps.org/doi/10.1103/PhysRevA.44.6334. Analysis of the Car-Parrinello method.

[10] P.E. Blöchl. Electrostatic decoupling of periodic images of plane wave expanded densities and derived atomic point charges. *J. Chem. Phys.*, 103: 7422, 1995.

[11] Message passing interface. hhttp://www.osc.edu/Lam/lam.html#MPI. Software for interprocessor communication on parallel computers.

[12] P.J. Mohr and B.N. Taylor. Codata recommended values of the fundamental physical constants: 1989. *Rev. Mod. Phys.*, 72:351, 2000.

[13] Open data explorer (opendx). http://www.opendx.org/. a general-purpose 3D visualization software derived from the IBM Dataexplorer.

[14] P.J. Turner. Xmgrace, an xy-plotting tool. http://plasma-gate.weizmann.ac.il/Xmgr/.

[15] B.Lüken. Crymolcad. Planned to be available soon at http://orion.pt.tu-clausthal.de/paw/. Software package for the visualization, analysis and manipulation of Crystal and Molecular Structures as well data-fields.

[16] J.P. Perdew and A. Zunger. Self interaction correction to density-functional approximations for many-electron systems. *Phys. Rev. B*, 23:5048, 1981.

[17] D.M. Ceperley and B.J. Alder. Ground state of the electron gas by a stochastic method. *Phys. Rev. Lett.*, 45:566, 1980.

[18] J. P. Perdew and Y. Wang. Accurate and simple analytic representation of the electron gas correlation energy. *Phys. Rev. B*, 45:13244, 1992.

[19] John P. Perdew, Kieron Burke, and Matthias Ernzerhof. Generalized gradient approximation made simple. *Phys. Rev. Lett*, 77:3865, 1996.

[20] B. Hammer, L.B. Hansen, and J.K. Norskov. Improved adsorption energetics within density-functional theory using revised perdew-burke-ernzerhof functionals. *Phys. Rev. B*, 59:7413, 1999.

[21] J.P. Perdew, J.A. Chevary, S.H. Vosko, K.A. Jackson, M.P. Pederson, D.J. Singh, and C. Fiolhais. Atoms, molecules solids and surfaces: applications of the generalizd gradient approximation for exchange and correlation. *Phys. Rev. B*, 46:6671, 1992.

[22] Y. Zhang and W. Yang. Comment on "generalized gradient approximation made simple". *Phys. Rev. Lett.*, 80:890, 1998.

[23] J. P. Perdew, A. Ruzsinszky, G. I. Csonka, O. A. Vydrov, G. E. Scuseria, L. A. Constantin, X. Zhou, and K. Burke. Restoring the density-gradient expansion for exchange in solids and surfaces. *Phys. Rev. Lett.*, 100:136406, 2008.

[24] A. D. Becke. Density-functional exchange energy with correct asymptotic behavior. *Phys. Rev. A*, 38:3098, 1988.

[25] C. Lee, W. Yang, and R.G. Parr. Development of the colle-salvetti correlation-energy formula into a functional of the electron density. *Phys. Rev. B*, 37:785, 1988.

[26] J. P. Perdew. Density functional approximation for the correlation energy of the inhomogeneous electron gas. *Phys. Rev. B*, 33:8822, 1986.

[27] J. Sun, A. Ruzsinszky, and J. P. Perdew. Strongly constrained and appropriately normed semilocal density functional. *Phys. Rev. Lett.*, 115:036402, 2015.

[28] James W. Furness, Aaron D. Kaplan, Jinliang Ning, John P. Perdew, and Jianwei Sun. Accurate and numerically efficient r^2SCAN meta-generalized gradient approximation. *J. Phys. Chem. Lett.*, 11:8208, 2020. doi: 10.1021/acs.jpclett.0c02405. URL https://doi.org/10.1021/acs.jpclett.0c02405. Erratum: J. Phys. Chem. Lett. **11**, 9248 (2020).

[29] D. Mejía-Rodríguez and S. B. Trickey. Deorbitalization strategies for meta-generalized-gradient-approximation exchange-correlation functionals. *Phys. Rev. A*, 96:052512, 2017.

[30] J. P. Perdew and L. A. Constantin. Laplacian-level density functionals for the kinetic energy density and exchange-correlation energy. *Phys. Rev. B*, 75:155109, 2007.

[31] M. Sotoudeh, S. Rajpurohit, P. Blöchl, D. Mierwaldt, J. Norpoth, V. Roddatis, S. Mildner, B. Kressdorf, B. Ifland, and C. Jooss. Electronic structure of $Pr_{1-x}Ca_xMnO_3$. *Phys. Rev. B*, 95:235150, 2017.

[32] S. Grimme, J. Antony, St.Ehrlich, and H. Krieg. A consistent and accurate ab initio parametrization of density functional dispersion correction (dft-d) for the 94 elements h-pu. *J. Chem. Phys.*, 132:154104, 2010.

[33] L. Verlet. Computer "experiments" on classical fluids i. thermodynamical properties of lennard-jones molecules. *Phys. Rev.*, 159:98, 1967.

[34] F. Tassone, F. Mauri, and R. Car. Acceleration schemes for ab initio molecular-dynamics simulations and electronic-structure calculations. *Phys. Rev. B*, 50:10561, 1994.

[35] Peter E. Blöchl. Second-generation wave-function thermostat for ab-initio molecular dynamics. *Phys. Rev. B*, 65:104303, 2002.

[36] S. Nose. A molecular dynamics method for the simulation in the canonical ensemble. *Mol. Phys.*, 52:255, 1984. Original publication on the Nose thermostat.

[37] S. Nose. A unified formulation of the constant temperature molecular dynamics methods. *J. Chem. Phys.*, 81:511, 1984.

[38] William G. Hoover. Canonical dynamics: Equilibrium phase space distributions. *Phys. Rev. A*, 31:1695, 1985.

[39] N.D. Mermin. Thermal properties of the inhomogeneous electron gas. *Physical Review*, 137:A1441, 1965. doi: 10.1103/PhysRev.137.A1441. URL http://link.aps.org/doi/10.1103/PhysRev.137.A1441.

[40] O. Jepsen and O.K. Andersen. The electronic structure of h.c.p. ytterbium. *Sol. St. Commun.*, 9:1763, 1971.

[41] G. Lehmann and M. Taut. On the numerical calculation of the density of states and related properties. *Phys. Stat. Sol. B*, 54:469, 1972.

[42] Peter E. Blöchl, O. Jepsen, and O. K. Andersen. Improved tetrahedron method for brillouin-zone integrations. *Phys. Rev. B*, 49: 16223–16233, Jun 1994. doi: 10.1103/PhysRevB.49.16223. URL http://link.aps.org/doi/10.1103/PhysRevB.49.16223.

[43] H.M. Senn, P.M. Margl, R. Schmid, T. Ziegler, and P.E. Blöchl. Ab initio molecular dynamics with a continuum solvation model. *J. Chem. Phys.*, 118: 1089, 2003.

[44] Frederick Wooten. *Optical Properties of Solids*. Academic Press, 1972.

[45] R.F. Egerton. Limits to the spatial, energy and momentum resolution of electron energy-loss spectroscopy. *Ultramicroscopy*, 107:565, 2007.

[46] M.P. Seah and W.A. Dench. Quantitative electron spectroscopy of surfaces: A standard data base for electron inelastic mean free paths in solids. *Surf. Interface Anal.*, 1:2, 1979. URL http://dx.doi.org/10.1002/sia.740010103.

[47] P. E. Blöchl and C. Först. Node-less atomic wave functions, pauli repulsion and systematic projector augmentation. *Arxiv*, 1210.5937, 2012. URL http://arxiv.org/abs/1210.5937.

[48] Jean-Paul Ryckaert, Giovanni Ciccotti, and Herman J. C. Berendsen. Numerical integration of cartesian equations of motion of a system with constraints: molecular dynamics of alkenes. *J. Comput. Phys.*, 23:327, 1977.

[49] First-Principles Calculations of Diffusion Coefficients: Hydrogen in Silica. P.e. blöchl and c.g. van de walle and s.t. pantelides. *Phys. Rev. Lett*, 64:1401, 1990.

[50] Kenichi Fukui. Formulation of the reaction coordinate. *The Journal of Physical Chemistry*, 74(23):4161–4163, 1970. doi: 10.1021/j100717a029. URL http://pubs.acs.org/doi/abs/10.1021/j100717a029.

[51] Kenichi Fukui. The path of chemical reactions - the irc approach. *Accounts of Chemical Research*, 14(12): 363–368, 1981. doi: 10.1021/ar00072a001. URL http://pubs.acs.org/doi/abs/10.1021/ar00072a001.

[52] A.K. Rappe, C.J. Casewit, K.S. Colwell, W.A. Goddard III, and W. M. Skiff. Uff, a full periodic table force field for molecular mechanics and molecular dynamics simulations. *J. Am. Chem. Soc.*, 114:10024–10035, 1992.

[53] F.D. Murnaghan. The compressibility of media under extreme pressures. *Proc. Natl. Acad. Sci. U.S.A.*, 30:244, 1944.

[54] I. Mills and T. Cvitaš, editors. *Quantities, Units and Symbols in Physical Chemistry*. Blackwell Scientific Publications, Oxford, 1993.

[55] Christopher. J. Bradley and Arthur P. Cracknell. *The Mathematical Theory of Symmetry in Solids*. Oxford University Press, 1972. ISBN 0199582580, 9780199582587. URL http://books.google.de/books?id=lMdNv_wbu2IC.

Index

atom name
    extended notation, 8

comments
    input files, 6
control input file, 10

extended notation atom name, 8

gyromagnetic ratio, 236

Hartree atomic units, 4
hyperfine field, 236

input file
    control, 10
    structure, 10

optimum friction, 244
    in COSMO, 59
overcompleteness, 252

structure input file, 10